UNIVERSIDAD PRIVADA DEL NORTE CIENCIAS

Departamento de Ingeniería

“Sistema integrado de base de datos segura, automatizada e inteligente para la DIRESA La Libertad”

Integrantes:

Chávez Rojas, Leonardo Josué	**N00478030@upn.pe**	**Ing. de Sistemas**
Jimenez Montes Samantha Luciana	**N00493055@upn.pe**	**Ing. de Sistemas**
Medina Manrique Dylan Daniel	**N00465903@upn.pe**	**Ing. de Sistemas**
Parra Huatuco, Britanny Karolin	**N00477820@upn.pe**	**Ing. de Sistemas**

Docente: JORGE LUIS MUÑOZ DIAZ

Sede San Juan de Lurigancho

2026_2

ÍNDICE

Contenido

Sede San Juan de Lurigancho 1
Relevamiento del dominio y selección del dataset MINSA 5
1.1. Descripción del reto y dataset 5
1.1.1. Dataset 1: Afiliados activos con diagnóstico de Diabetes Mellitus 5
1.1.2. Dataset 2: Consumo de medicamentos en recién nacidos 6
1.2. Diagnóstico de calidad de los datos 6
1.3. Alcance y requerimientos funcionales 8
1.4. Supuestos del proyecto y justificación ética 9
1.5. Planificación y ruta del proyecto integrador 9
1.5.2. Ruta crítica y dependencias 11
1.5.3. Recursos por bloque 11
1.5.4. Contingencias ante posibles retrasos 12
Diseño e implementación de objetos de automatización 13
2.1. Diseño de la base de datos DIRESA_PERU 13
2.2. Descripción de las tablas creadas 13
2.3. Enfoque de automatización seleccionado 15
2.4. Comparación de alternativas técnicas 15
2.5. Justificación del enfoque seleccionado 16
2.6. Implementación de procedimientos almacenados 16
2.6.1. Procedimiento SP_INGRESAR_PRESTACION_RN 16
2.6.2. Procedimiento SP_INGRESAR_AFILIADO_DIABETES 16
2.7. Control de transacciones y manejo de excepciones 17
2.7.1. Prueba de ROLLBACK ante una excepción 17
2.7.2. Prueba de SAVEPOINT y ROLLBACK parcial 17
2.8. Implementación de auditoría 17
2.8.1. Tabla de log de auditoría 17
2.8.2. Trigger TRG_AUDITORIA_PRESTACIONES 17
2.8.3. Validación del trigger de auditoría 18
2.9. Implementación de mecanismos de integridad 18
2.9.1. Trigger TRG_VALIDAR_PRESTACIONES 18
2.9.2. Prueba del trigger de integridad 18
2.10. Implementación de función en SQL Server 18
2.10.1. Función FN_CANTIDAD_DIAGNOSTICOS 18
2.10.2. Prueba de funcionamiento 18
2.10.2. Prueba de funcionamiento 19
Implementación de políticas de seguridad y cumplimiento normativo 19
3.1. Diseño de seguridad y segregación de funciones 19

3.2. Implementación de roles y privilegios 19
3.2.1. Creación y asignación de usuarios y roles 20
3.2.2. Asignación de privilegios 20
3.3. Pruebas de acceso y mínimo privilegio 20
3.3.1. Acceso permitido para UsuarioAnalista 20
3.3.2. Acceso denegado para UsuarioAnalista 20
3.3.3. Restricción de modificación para UsuarioAuditor 20
3.3.4. Verificación del principio de mínimo privilegio 21
3.4. Política de respaldo y recuperación 21
3.5. Prueba de restauración 21
3.6. Implementación de índice para optimización 22
3.7. Comparación de rendimiento antes y después del índice 22
3.7.1. Rendimiento antes de implementar el índice 22
3.7.2. Rendimiento después de implementar el índice 22
3.7.3. Comparación de resultados 22
3.7.4. Análisis del plan de ejecución 23
3.8. Cumplimiento de la Ley N.° 29733 e ISO/IEC 27001 23
3.8.1. Checklist de seguridad y cumplimiento 24
3.9. Impacto de las decisiones de seguridad sobre los pacientes 25
Exploración de paradigma NoSQL Y decisión de integración 26
4.1. Identificación y objetivo de la prueba 26
4.2. Procedimiento y resultados 26
4.2.1 Diseño del documento 26
4.2.2 Migración de datos 26
4.2.3 Flexibilidad de esquema entre documentos 27
4.2.4 Validación de operaciones CRUD sobre la colección. 27
4.2.5 Comparación técnica SQL Server vs. MongoDB. 29
4.3. Veredicto de viabilidad y ajustes al proyecto. 29
Diseño del Data Warehouse y procesos ETL 30
5.1. Enfoque dimensional 30
5.2. Tablas de hechos y granularidad 30
5.2.1. FactPrestacionesRN 31
5.2.2. FactAtencionDiabetes 31
5.3. Implementación del modelo dimensional 31
5.4. Proceso ETL 31
5.4.1. Extracción 31
5.4.2. Transformación 31
5.4.3. Carga 32
5.5. Control y trazabilidad del ETL 32
5.6. Validación del Data Warehouse 32
5.7. Justificación de Kimball frente a Inmon 33

Construcción de dashboard bi y análisis OLAP 34
6.1. Implementación del modelo dimensional en Power BI 34
6.2. Definición e implementación de KPIs 34
6.2.1. KPI 1: Total de prestaciones de recién nacidos 34
6.2.2. KPI 2: Total de afiliados con diabetes 35
6.2.3. KPI 3: Peso promedio de recién nacidos 35
6.2.4. KPI 4: Porcentaje de registros de diabetes con diagnóstico de hipertensión 35
6.3. Construcción del dashboard BI 35
6.4. Metadatos y gobernanza de los indicadores 36
6.5. Análisis OLAP 36
6.6. Preguntas de negocio y análisis de resultados 37
6.6.1. Pregunta de negocio 1 37
6.6.2. Pregunta de negocio 2 37
6.7. Validación de los resultados del dashboard 37
Análisis Big Data mediante Apache Spark y reflexión de escalabilidad 38
7.1. Análisis Big Data mediante Apache Spark 38
7.1.1. Objetivo del análisis 38
7.1.2. Entorno tecnológico y metodología 38
7.1.3. Procesamiento y operaciones analíticas 38
7.1.4. Comparación de tiempos de ejecución 39
7.2. Análisis de eficiencia, escalabilidad y adecuación 39
7.2.1. Evaluación de eficiencia 39
7.2.2. Evaluación experimental de escalabilidad 40
7.2.3. Adecuación a la naturaleza de los datos 40
7.2.4. Limitaciones del experimento 41
7.2.5. Conclusiones y recomendaciones 41
Integración, reflexión ética y articulación curricular. 41
8.1. Integración de los componentes del proyecto 41
8.2. Repositorio Git y organización de los artefactos 42
8.3. Reflexión ética y responsabilidad profesional 42
8.3.1. Dilema ético 1: disponibilidad de información frente a privacidad de los pacientes 43
8.3.2. Dilema ético 2: facilidad de operación frente a integridad y seguridad de los datos 43
8.3.3. Posición ética del equipo 43
8.4. Articulación curricular 44
8.4.1. Saberes previos del curso Base de Datos (ciclo 3) 44
8.4.2. Proyección hacia el curso Ciencia de Datos 44
8.4.3. Proyección hacia el curso Ingeniería de Software 44
8.5. Conclusiones de la integración 45
CAPÍTULO VII 87
Evidencias gráficas 87
Código y documentación complementaria 89

CAPÍTULO I

Relevamiento del dominio y selección del dataset MINSA

1.1. Descripción del reto y dataset

El proyecto DataSalud Perú tiene como propósito diseñar una solución integrada para la gestión de datos abiertos del sector salud, considerando su calidad, organización, seguridad, automatización y análisis. Para ello, se seleccionaron dos conjuntos de datos publicados por el Seguro Integral de Salud (SIS) en la Plataforma Nacional de Datos Abiertos: afiliados activos con diagnóstico de Diabetes Mellitus y consumo de medicamentos en recién nacidos atendidos en Hospitalización y Emergencia.

Ambos conjuntos representan dominios diferentes, por lo que no se establece una relación clínica directa entre sus registros. Su integración se realiza a nivel de arquitectura mediante procesos comunes de carga, validación, almacenamiento, seguridad, auditoría y análisis. En el contexto de la **DIRESA La Libertad**, la solución permitirá analizar la información según criterios territoriales, temporales y características disponibles en cada fuente.

1.1.1. Dataset 1: Afiliados activos con diagnóstico de Diabetes Mellitus

Nombre: Afiliados activos en el Seguro Integral de Salud con diagnóstico de Diabetes Mellitus - [SIS].

Fuente: Seguro Integral de Salud (SIS), publicado mediante la Plataforma Nacional de Datos Abiertos del Estado Peruano.

URL:
https://www.datosabiertos.gob.pe/dataset/afiliados-activos-en-el-seguro-integral-desalud-con-diagn%C3%B3stico-de-diabetes-mellitus-sis

Fecha de descarga: 02 de septiembre de 2026

Formato y archivo utilizado: Archivo Microsoft Excel (.xlsx), denominado *Afiliados activos con DM 202512.xlsx*.

Volumen: 850 366 registros, distribuidos en 24 campos.

El conjunto contiene información de afiliados activos del SIS con diagnóstico de Diabetes Mellitus, incluyendo datos anonimizados, demográficos y geográficos, así como información sobre diagnóstico, comorbilidades, atenciones y hospitalizaciones.

1.1.2. Dataset 2: Consumo de medicamentos en recién nacidos

Nombre: Consumo de medicamentos en recién nacidos asegurados al Seguro Integral de Salud – Hospitalización y Emergencia - [SIS].

Fuente: Seguro Integral de Salud (SIS), publicado mediante la Plataforma Nacional de Datos Abiertos del Estado Peruano.

URL:
https://www.datosabiertos.gob.pe/dataset/consumo-de-medicamentos-en-reci%C3%A9nnacidos-asegurados-al-seguro-integral-de-salud

Fecha de descarga: 2 de septiembre de 2026

Formato y archivos utilizados: El conjunto de datos se encuentra distribuido en cuatro archivos Microsoft Excel (.xlsx): *1 Prestaciones*, *2 Diagnósticos*, *3 Medicamentos* y *4 Catálogos*. Este último contiene dos hojas: *Diagnóstico* y *Medicamentos*.

Volumen: El conjunto presenta la siguiente distribución de registros:

- Prestaciones: 290469 ● Diagnósticos: 374083
- Medicamentos: 931763
- Catálogo de diagnósticos: 13360
- Catálogo de medicamentos: 11543

En conjunto, las estructuras analizadas comprenden 1621218 registros.

El dataset contiene información asociada a las prestaciones de salud y al consumo de medicamentos en recién nacidos asegurados al SIS atendidos en los servicios de Hospitalización y Emergencia.

1.2. Diagnóstico de calidad de los datos

Se realizó el perfilado de los archivos originales para identificar problemas de calidad y cuantificar los registros afectados antes de su incorporación al sistema. Se evaluó principalmente la presencia de valores faltantes y el nivel de completitud de los campos relevantes.

Dataset / archivo	**Problema de calidad**	**Campo(s) afectado(s)**	**Registros afectados**	**Tratamiento realizado**
Diabetes Mellitus – Afiliados activos	Valores faltantes en variables monetarias	VALOR_NETO, VALOR_NETO_HOSP	19	Eliminación de los registros afectados.
Diabetes Mellitus – Afiliados activos	Alta incompletitud en información de última atención	UBIGEO_ULT_ATE, DEPARTAMENTO_ULT_ATE, PROVINCIA_ULT_ATE, DISTRITO_ULT_ATE, NIVEL_ULT_ATE	625 827 (73,59 %)	Exclusión de los campos en la versión limpia.
Recién Nacidos – Prestaciones	Valores faltantes	PESO	41 188	Conservación como valores nulos para evitar imputar información no disponible.
Recién Nacidos –	Valores faltantes	DIAS_HOSP	27 050	Conservación como valores nulos debido a que el dato puede no

Prestaciones				corresponder a prestaciones sin hospitalización.
Recién Nacidos – Medicamentos	Código de medicamento faltante	COD_MEDICAMENTO	2	Eliminación de los registros afectados.
Recién Nacidos – Catálogo Medicamentos	Valores faltantes	PRESENTACIÓN	4 853	Conservación como valores nulos, sin imputar información no disponible.
Recién Nacidos – Catálogo Medicamentos	Valores faltantes	CONCENTRACIÓN	1 069	Conservación como valores nulos, sin imputar información no disponible.

Ver evidencia en ANEXOS — Capítulo I.

Fuente: Elaboración propia a partir de los datos abiertos del SIS.

Las figuras anteriores presentan evidencias representativas del proceso de perfilado y tratamiento de los datos.

Los resultados muestran que no todos los valores faltantes requieren el mismo tratamiento. Los registros cuya ausencia compromete la identificación o procesamiento del dato fueron eliminados, mientras que aquellos cuya información no se encontraba disponible en la fuente se conservaron como valores nulos para evitar introducir datos artificiales. Asimismo, los campos de última atención del dataset de Diabetes Mellitus fueron excluidos debido a su elevado nivel de incompletitud.

1.3. Alcance y requerimientos funcionales

A partir de los problemas identificados en los datasets seleccionados y de las necesidades planteadas en el reto DataSalud Perú, se definieron los siguientes requerimientos funcionales:

N.°	Requerimiento Funcional	Criterio de éxito verificable
RF01	El sistema deberá permitir la ingesta y validación de los datos provenientes de los datasets de Diabetes Mellitus y Recién Nacidos antes de incorporarlos a las tablas finales.	La carga se ejecuta mediante procedimientos almacenados y los registros que incumplen las validaciones definidas son identificados antes de su incorporación a las tablas finales.
RF02	El sistema deberá controlar la calidad de los datos, considerando valores nulos en campos relevantes, formatos inválidos y registros que incumplan las reglas establecidas.	Las validaciones generan un resultado cuantificable de registros válidos y registros observados o rechazados durante cada proceso de carga.

RF03	El sistema deberá registrar las operaciones realizadas sobre los datos para mantener trazabilidad de los cambios.	Las operaciones definidas para auditoría generan automáticamente un registro que permita identificar la operación realizada, la fecha y el objeto afectado.
RF04	El sistema deberá permitir la consulta de información consolidada para apoyar el análisis de los datos de salud seleccionados.	Se obtienen consultas y resultados agregados a partir de los datos almacenados, utilizables posteriormente en el modelo analítico y la visualización de indicadores.
RF05	El sistema deberá gestionar el acceso a la base de datos DIRESA_PERU mediante usuarios, roles y permisos, aplicando el principio de mínimo privilegio.	Los usuarios autorizados pueden ejecutar únicamente las operaciones asignadas a su rol y los accesos no autorizados son rechazados.
RF06	El sistema deberá proporcionar información para el análisis mediante indicadores y consultas multidimensionales sobre los datos procesados.	El dashboard presenta al menos tres KPI y permite realizar al menos una operación OLAP sobre la información almacenada en el Data Warehouse.

1.4. Supuestos del proyecto y justificación ética

Para el diseño e implementación de la solución se establecieron los siguientes supuestos, considerando las características de los datasets seleccionados y el alcance del proyecto.

Código	Supuesto	Justificación técnica	Impacto en el diseño
S01	Los datasets de Diabetes Mellitus y Recién Nacidos se gestionan dentro de una misma arquitectura, sin establecer una relación clínica directa entre ambas poblaciones.	Los conjuntos de datos pertenecen al sector salud y provienen del SIS, pero representan poblaciones y procesos diferentes. Relacionarlos directamente podría generar asociaciones sin sustento en los datos.	La base DIRESA_PERU almacenará ambos dominios mediante estructuras independientes, compartiendo mecanismos de carga, validación, seguridad y auditoría.
S02	Los valores nulos que no impidan identificar o procesar un registro podrán conservarse cuando la fuente no proporcione información suficiente para completarlos.	La imputación de valores sin evidencia podría introducir información artificial. Se identificaron, por ejemplo, valores ausentes en PESO, PRESENTACIÓN y CONCENTRACIÓN.	El modelo permitirá valores NULL en los atributos correspondientes y las consultas deberán considerar su presencia durante el análisis.

S03	Los códigos anonimizados proporcionados por las fuentes serán utilizados como identificadores de referencia, sin intentar reconstruir la identidad de los asegurados.	Los datasets entregan información anonimizada y el proyecto no dispone de información adicional que permita identificar directamente a las personas.	El diseño conservará los identificadores anonimizados y evitará incorporar datos personales que no sean necesarios para los objetivos analíticos.
S04	El análisis se realizará utilizando la información disponible en los archivos descargados, manteniendo trazabilidad respecto a su fecha de corte.	Los conjuntos de datos representan información correspondiente a determinados períodos y pueden ser actualizados posteriormente por la entidad de origen.	Las estructuras de almacenamiento conservarán campos temporales como FECHA_CORTE, permitiendo diferenciar el periodo al que corresponde la información procesada.

1.5. Planificación y ruta del proyecto integrador

Para organizar el desarrollo de DataSalud Perú se estableció un cronograma de hitos que comprende las etapas de gestión de datos, implementación de bases de datos y análisis de información. Esta planificación permite establecer dependencias entre actividades, distribuir responsabilidades y anticipar posibles retrasos.

Hito	Actividad principal	Fecha	Bloque temático	Responsables	Dependencia
H1	Selección y descarga de los datasets del SIS	29/08/2026	Gestión y calidad de datos	Samantha Jimenez y Leonardo Chavez	—
H2	Perfilado, limpieza y diagnóstico de calidad de los datos	05/09/2026	Gestión y calidad de datos	Leonardo Chavez, Samantha Jimenez, Dylan Medina y Britanny Parra	H1
H3	Modelado e implementación de la base de datos DIRESA_PERU	12/09/2026	Bases de datos avanzadas	Leonardo Chavez	H2

H4	Desarrollo de procedimientos almacenados, validaciones, transacciones, seguridad y auditoría	19/09/ 2026	Bases de datos avanzadas	Britanny Parra	H3
H5	Implementación y evaluación de datos semiestructurados en MongoDB	22/09/ 2026	Bases de datos avanzadas / Big Data	Dylan Medina	H2
H6	Diseño del Data Warehouse bajo metodología Kimball y ejecución del proceso ETL	24/09/ 2026	Inteligencia de negocios	Samantha Jimenez	H3 y H4
H7	Construcción de KPI, análisis OLAP y dashboard en Power BI	25/09/ 2026	Inteligencia de negocios	Britanny Parra	H6
H8	Integración, pruebas finales, documentación y preparación de la presentación	09/10/ 2026	Integración del proyecto	Leonardo Chavez, Samantha Jimenez, Dylan Medina y Britanny Parra	H5 y H7

1.5.2. Ruta crítica y dependencias

La ruta crítica definida para el proyecto es:

H1 → H2 → H3 → H4 → H6 → H7 → H8

Esta secuencia comprende la selección y limpieza de los datos, implementación de la base de datos operacional, automatización y seguridad, desarrollo del Data Warehouse, análisis en Power BI e integración final. Cada etapa depende de los resultados obtenidos en la anterior, por lo que cualquier retraso puede afectar la culminación del proyecto.

El **H5 (MongoDB)** se desarrolla en paralelo y converge en H8, por lo que no forma parte de la ruta crítica principal.

1.5.3. Recursos por bloque

Para el cumplimiento de los hitos se consideraron los recursos humanos, tecnológicos y de información necesarios para cada bloque del proyecto.

Bloque temático	Recursos humanos	Recursos tecnológicos y de información
Gestión y calidad de datos	Equipo del proyecto, con responsables de limpieza y documentación	Datasets abiertos del SIS, Microsoft Excel y documentación de las fuentes
Bases de datos avanzadas y Big Data	Responsable de base de datos y equipo de apoyo	SQL Server, SQL Server Management Studio, MongoDB Atlas/Compass y scripts SQL/NoSQL
Inteligencia de negocios	Equipo y responsable del componente analítico	Data Warehouse, herramientas ETL, Python/Pandas y Power BI

1.5.4. Contingencias ante posibles retrasos

Riesgos	Contingencia
Retraso en la limpieza o procesamiento de los datasets debido a su volumen o problemas de calidad.	Distribuir el procesamiento entre los integrantes y priorizar las variables necesarias para la implementación y el análisis.
Errores durante la implementación o integración en SQL Server, MongoDB o el proceso ETL.	Mantener copias de respaldo de los scripts y datasets, realizar pruebas por etapas y conservar una versión funcional previa antes de aplicar cambios.
Retraso en la elaboración del Data Warehouse o dashboard en Power BI.	Priorizar los KPI y visualizaciones obligatorias establecidos en el proyecto y desarrollar primero los componentes necesarios para cumplir los requerimientos mínimos.

CAPÍTULO II

Diseño e implementación de objetos de automatización en SQL Server

2.1. Diseño de la base de datos DIRESA_PERU

La base de datos DIRESA_PERU integra los dos conjuntos de datos del proyecto mediante estructuras independientes, sin establecer una relación clínica directa entre ambos dominios. Para la carga inicial se utilizaron tablas temporales, permitiendo validar y transformar los registros antes de incorporarlos a las tablas definitivas.

Para el dataset 1: El modelo de Diabetes Mellitus está conformado principalmente por AFILIADOS, UBIGEO y ATENCIONES_DIABETES, relacionadas mediante claves primarias y foráneas.

Para el Dataset 2: El modelo de Recién Nacidos comprende PRESTACIONES,

MEDICAMENTOS, DIAGNOSTICOS, CATALOGO_MEDICAMENTOS, CATALOGO_DIAGNOSTICOS y UBIGEO_2, manteniendo relaciones entre prestaciones, diagnósticos, medicamentos y sus respectivos catálogos.

En ambos modelos se establecieron relaciones mediante claves primarias y foráneas para mantener la integridad referencial. Las tablas temporales se utilizaron como apoyo durante los procesos de carga, validación y transformación antes de incorporar los registros a las estructuras definitivas.

2.2. Descripción de las tablas creadas

Las tablas de DIRESA_PERU fueron organizadas según la función que cumplen dentro de cada dataset. Algunas representan las entidades principales, otras almacenan información de detalle o catálogos y las tablas temporales apoyan el proceso inicial de importación y transformación.

Tabla	Dataset	Propósito	Clase principal	Relaciones principales
UBIGEO	Diabetes Mellitus	Almacenar departamento, provincia y distrito.	UBIGEO	AFILIADOS
AFILIADOS	Diabetes Mellitus	Almacenar los datos principales de los afiliados.	ID	UBIGEO y ATENCIONES_DIABETES
ATENCIONES_DIABETES	Diabetes Mellitus	Registrar diagnóstico, comorbilidades, atenciones y hospitalizaciones.	ID	AFILIADOS mediante ID_AFILIADO

AFILIADOS_DIABETESDM_TEMP	Diabetes Mellitus	Recibir temporalmente los datos originales.	ID	Apoyo para importación y validación
PRESTACIONES	Recién Nacidos	Almacenar las prestaciones de salud.	ID	MEDICAMENTOS, DIAGNOSTICOS y UBIGEO_2
MEDICAMENTOS	Recién Nacidos	Registrar medicamentos asociados a cada prestación.	ID_REGISTRO_REL + CODDIA + COD_MEDICAMENTO	PRESTACIONES y CATALOGO_MEDICAMENTOS
DIAGNOSTICOS	Recién Nacidos	Registrar diagnósticos asociados a cada prestación.	ID_REGISTRO_REL + CODDIA	PRESTACIONES y CATALOGO_DIAGNOSTICOS
CATALOGO_MEDICAMENTOS	Recién Nacidos	Almacenar información descriptiva de medicamentos.	COD_MEDICAMENTO	MEDICAMENTOS
CATALOGO_DIAGNOSTICOS	Recién Nacidos	Almacenar códigos y nombres de diagnósticos.	CODDIA	DIAGNOSTICOS
UBIGEO_2	Recién Nacidos	Almacenar información territorial.	UBIGEO	PRESTACIONES
PRESTACIONES_TEMPORAL	Recién Nacidos	Recibir registros originales antes de su incorporación al modelo definitivo.	ID	Apoyo para importación y transformación

La estructura mantiene separados los dos dominios analizados y organiza la información mediante tablas principales, de detalle y catálogos.

2.3. Enfoque de automatización seleccionado

La automatización en DIRESA_PERU se implementó mediante procedimientos almacenados, triggers, una función reutilizable y la tabla LOG_AUDITORIA, permitiendo controlar el ingreso y validación de datos, mantener la trazabilidad y asegurar la integridad de la información.

Además, se utilizaron TRY/CATCH, COMMIT, ROLLBACK y SAVE TRANSACTION para gestionar excepciones y garantizar la consistencia de las operaciones.

2.4. Comparación de alternativas técnicas

Para seleccionar el enfoque de automatización se compararon los objetos programables de SQL Server, la lógica implementada mediante una aplicación externa y los procesos ETL externos.

Criterio	SQL Server	Aplicación externa	ETL externo
Rendimiento	Ejecuta operaciones directamente sobre los datos almacenados.	Puede requerir transferencia de información entre aplicación y BD.	Adecuado para cargas y transformaciones masivas.
Mantenibilidad	Centraliza la lógica relacionada directamente con los datos.	Mantiene reglas en la aplicación, separadas de la BD.	Mantiene las transformaciones dentro del proceso ETL.
Integridad	Permite aplicar transacciones, restricciones y triggers directamente.	Depende de las validaciones implementadas por la aplicación.	Puede validar durante la transformación y carga.
Trazabilidad	Triggers y tablas de auditoría permiten registrar operaciones.	Requiere mecanismos adicionales desde la aplicación.	Permite generar logs de ejecución del proceso.

La comparación demuestra que SQL Server es una alternativa adecuada para automatizar las validaciones, transacciones y auditorías directamente en la base de datos. Aunque las aplicaciones externas y herramientas ETL ofrecen otras posibilidades, requieren componentes adicionales. Por ello, se priorizaron los procedimientos almacenados y triggers, manteniendo los procesos analíticos de forma independiente.

2.5. Justificación del enfoque seleccionado

Se optó por implementar los principales controles en SQL Server para centralizar las reglas de integridad, transacciones y auditoría, independientemente de las aplicaciones que consuman la información. La lógica de presentación, interacción con el usuario y visualización analítica se mantiene separada y puede ser gestionada mediante aplicaciones o herramientas de Business Intelligence.

2.6. Implementación de procedimientos almacenados

Se implementaron dos procedimientos almacenados para automatizar el ingreso y validación de información correspondiente a los datasets utilizados.

2.6.1. Procedimiento SP_INGRESAR_PRESTACION_RN

Propósito: automatizar el registro de una prestación correspondiente al dataset de Recién Nacidos, aplicando las validaciones necesarias antes de incorporar la información en PRESTACIONES. El procedimiento recibe los principales datos de la prestación, como fecha de corte, identificador del registro, código anonimizado, ubigeo, servicio prestacional, días de hospitalización, edad en días, fecha de atención, sexo, peso y modalidad de atención.

Se verifican los datos requeridos y las reglas establecidas para evitar valores inconsistentes. Además, utiliza **TRY/CATCH** y control transaccional: ante una ejecución satisfactoria se realiza COMMIT y, si ocurre una excepción, se ejecuta ROLLBACK.

La prueba realizada confirmó la incorporación correcta del registro en PRESTACIONES.

Script SQL del procedimiento SP_INGRESAR_PRESTACION_RN.

Código: ver ANEXOS — Capítulo II.

2.6.2. Procedimiento SP_INGRESAR_AFILIADO_DIABETES

Propósito: automatizar conjuntamente el registro de un afiliado y su información asociada de Diabetes Mellitus, manteniendo la relación entre AFILIADOS y ATENCIONES_DIABETES.

El procedimiento valida la existencia del ubigeo, evita duplicar códigos anonimizados y comprueba que cantidades, valores monetarios y días de hospitalización no sean negativos. Primero registra al afiliado en AFILIADOS y recupera mediante SCOPE_IDENTITY() el identificador generado. Posteriormente, utiliza este valor para insertar la información relacionada en ATENCIONES_DIABETES. Ambas operaciones se ejecutan dentro de una misma transacción. Si finalizan correctamente se realiza COMMIT; ante una excepción, el bloque CATCH ejecuta ROLLBACK.

Script SQL del procedimiento SP_INGRESAR_AFILIADO_DIABETES.

USE DIRESA_PERU;

GO

---Creamos el procedimiento

Código: ver ANEXOS — Capítulo II.

FROM dbo.AFILIADOS

WHERE CODIGO_ANONIMIZADO = 'PRUEBA_ROLLBACK02';

2.7. Control de transacciones y manejo de excepciones

Los procedimientos implementados utilizan mecanismos de control transaccional y manejo de excepciones mediante TRY/CATCH, COMMIT y ROLLBACK. Adicionalmente, se realizó una prueba mediante SAVE TRANSACTION para comprobar una reversión parcial.

2.7.1. Prueba de ROLLBACK ante una excepción

Para comprobar el funcionamiento del ROLLBACK, se provocó intencionalmente un error durante la segunda inserción de SP_INGRESAR_AFILIADO_DIABETES, utilizando un valor que excedía la longitud permitida en TIPO_DIABETES.

SQL Server generó el error 2628 y el bloque CATCH ejecutó el ROLLBACK. Posteriormente, se consultó el afiliado utilizado en la prueba y se obtuvieron cero registros, comprobándose que la primera inserción tampoco permaneció almacenada.

2.7.2. Prueba de SAVEPOINT y ROLLBACK parcial

Se insertó el afiliado de prueba PRUEBA_SAVEPOINT01 con una edad inicial de 30 años. Posteriormente, se creó un punto de guardado mediante SAVE TRANSACTION, se modificó temporalmente la edad a 99 y se ejecutó ROLLBACK TRANSACTION hasta el punto de guardado.

Finalmente, se confirmó la transacción mediante COMMIT. La consulta posterior mostró que el afiliado permaneció registrado con una edad de 30 años, comprobando que únicamente se revirtió la modificación realizada después del punto de guardado.

2.8. Implementación de auditoría

Para mantener la trazabilidad de las operaciones se implementó la tabla LOG_AUDITORIA y el trigger TRG_AUDITORIA_PRESTACIONES.

2.8.1. Tabla de log de auditoría

LOG_AUDITORIA almacena información sobre las operaciones auditadas, incluyendo fecha y hora, usuario, tabla afectada, tipo de operación, identificador del registro y descripción.

Script SQL de LOG_AUDITORIA.

Código: ver ANEXOS — Capítulo II.

2.8.2. Trigger TRG_AUDITORIA_PRESTACIONES

El trigger se ejecuta **AFTER INSERT** sobre PRESTACIONES y utiliza la tabla lógica inserted para obtener el identificador del nuevo registro. La información correspondiente se registra automáticamente en LOG_AUDITORIA.

Para identificar al usuario que realizó la operación se utiliza SUSER_SNAME().

Script SQL de TRG_AUDITORIA_PRESTACIONES.

Código: ver ANEXOS — Capítulo II.

2.8.3. Validación del trigger de auditoría

Después de insertar una prestación de prueba se consultó LOG_AUDITORIA, verificándose la generación automática de un registro con la operación **INSERT**, usuario, fecha, identificador y descripción correspondiente.

Ver evidencia en ANEXOS — Capítulo II.

2.9. Implementación de mecanismos de integridad

Se implementó el trigger **TRG_VALIDAR_PRESTACIONES** para impedir el almacenamiento de determinados valores inválidos en PRESTACIONES.

2.9.1. Trigger TRG_VALIDAR_PRESTACIONES

El trigger actúa ante operaciones **INSERT** y **UPDATE** y verifica las siguientes reglas:

- EDAD_DIAS no puede ser negativa.
- DIAS_HOSP no puede ser negativo.
- PESO, cuando se encuentre registrado, debe ser mayor que cero.

El trigger analiza los registros mediante la tabla lógica inserted. Si detecta un valor inválido, utiliza RAISERROR, ejecuta ROLLBACK TRANSACTION y cancela la operación.

Script SQL de TRG_VALIDAR_PRESTACIONES.

Código: ver ANEXOS — Capítulo II.

2.9.2. Prueba del trigger de integridad

Se intentó modificar una prestación estableciendo EDAD_DIAS = -5. El trigger rechazó la operación y una consulta posterior mostró nuevamente el valor original de **12 días**, demostrando que la modificación inválida no fue almacenada.

Ver evidencia en ANEXOS — Capítulo II.

2.10. Implementación de función en SQL Server

Para reutilizar consultas sobre la información almacenada se implementó una función escalar.

2.10.1. Función FN_CANTIDAD_DIAGNOSTICOS

Propósito: obtener la cantidad de diagnósticos asociados a un determinado ID_REGISTRO_REL.

La función recibe @ID_REGISTRO_REL como parámetro y retorna un valor entero correspondiente al número de registros encontrados en DIAGNOSTICOS para el identificador proporcionado.

Script SQL de FN_CANTIDAD_DIAGNOSTICOS.

Código: ver ANEXOS — Capítulo II.

2.10.2. Prueba de funcionamiento

Para verificar el funcionamiento de la función FN_CANTIDAD_DIAGNOSTICOS, se ejecutó una consulta utilizando el identificador 1018096043. La función devolvió un total de 9 diagnósticos asociados, resultado que coincidió con el conteo realizado directamente sobre la tabla DIAGNOSTICOS.

Esta prueba permitió comprobar que la función obtiene correctamente la cantidad de diagnósticos relacionados con una prestación y puede reutilizarse en consultas posteriores.

Ver evidencia en ANEXOS — Capítulo II.

CAPÍTULO III

Implementación de políticas de seguridad y cumplimiento normativo

3.1. Diseño de seguridad y segregación de funciones

La seguridad de la base de datos DIRESA_PERU se implementa bajo el principio de mínimo privilegio, de manera que cada usuario disponga únicamente de los permisos necesarios para realizar las actividades correspondientes a su función. Para ello, se establecen roles con privilegios diferenciados, evitando otorgar acceso general a todas las operaciones de la base de datos. Esta separación permite reducir el riesgo de modificaciones no autorizadas, accesos innecesarios o eliminación accidental de información.

La segregación de funciones considera como mínimo tres perfiles: uno orientado a la consulta de información, otro encargado de las operaciones sobre los datos y un perfil de administración, responsable de las tareas que requieren mayores privilegios. La implementación de estos roles permite aplicar controles de acceso sobre la información almacenada y comprobar, mediante pruebas, que las operaciones autorizadas puedan ejecutarse correctamente y que aquellas no permitidas sean rechazadas por SQL Server.

3.2. Implementación de roles y privilegios

Para controlar el acceso a la base de datos DIRESA_PERU, se implementaron perfiles con diferentes niveles de privilegio. La asignación de permisos se realizó considerando las responsabilidades de cada usuario y aplicando el principio de mínimo privilegio.

La distribución de permisos contempla tres niveles principales: un perfil orientado a la consulta de información, un perfil encargado de realizar operaciones autorizadas sobre los datos y un perfil de administración, responsable de las tareas que requieren mayores privilegios.

Perfil	Función	Privilegios principales
Consulta / Analista	Consultar información sin modificar los registros.	SELECT
Operador	Consultar y registrar información necesaria para las operaciones del sistema.	SELECT, operaciones autorizadas y ejecución de procedimientos
Administrador	Gestionar objetos, permisos y operaciones administrativas de la base de datos.	Privilegios administrativos

Esta separación evita que los usuarios dispongan de permisos superiores a los necesarios para sus funciones y permite mantener diferenciadas las actividades de consulta, operación y administración.

3.2.1. Creación y asignación de usuarios y roles

La seguridad se configura directamente en SQL Server mediante la creación de usuarios y roles, seguida de la asignación de los permisos correspondientes.

Script SQL:

Código: ver ANEXOS — Capítulo III.

3.2.2. Asignación de privilegios

Los permisos se asignan de acuerdo con las responsabilidades establecidas para cada perfil. De esta manera, las operaciones de consulta, modificación y administración permanecen separadas.

En el caso del usuario **UsuarioAnalista**, se permite la consulta de información sobre la tabla PRESTACIONES, mientras que las operaciones de eliminación se encuentran restringidas.

Script SQL de asignación de privilegios.

Código: ver ANEXOS — Capítulo III.

3.3. Pruebas de acceso y mínimo privilegio

Para comprobar la correcta aplicación de los permisos establecidos, se realizaron pruebas utilizando usuarios con privilegios diferenciados sobre la base de datos **DIRESA_PERU**.

3.3.1. Acceso permitido para UsuarioAnalista

Se ejecutó una consulta SELECT mediante EXECUTE AS USER = 'UsuarioAnalista'. La operación se realizó correctamente y permitió visualizar registros de la tabla PRESTACIONES, comprobando que el usuario dispone de permisos de lectura.

Ver evidencia en ANEXOS — Capítulo III.

3.3.2. Acceso denegado para UsuarioAnalista

Posteriormente, se intentó ejecutar una operación DELETE sobre la tabla PRESTACIONES utilizando el mismo usuario. SQL Server rechazó la operación mostrando que el permiso DELETE se encontraba denegado.

Ver evidencia en ANEXOS — Capítulo III.

3.3.3. Restricción de modificación para UsuarioAuditor

Como prueba adicional de segregación de funciones, se utilizó UsuarioAuditor para intentar modificar directamente un registro de la tabla LOG_AUDITORIA mediante una operación UPDATE.

SQL Server rechazó la operación, demostrando que este usuario no dispone de permisos para alterar directamente los registros de auditoría.

Ver evidencia en ANEXOS — Capítulo III.

3.3.4. Verificación del principio de mínimo privilegio

Las pruebas realizadas evidencian la aplicación de privilegios diferenciados. UsuarioAnalista puede consultar la información necesaria para sus funciones, pero no eliminar registros de PRESTACIONES; asimismo, UsuarioAuditor no puede modificar directamente los registros almacenados en LOG_AUDITORIA.

Estas restricciones reducen el riesgo de modificaciones no autorizadas y permiten aplicar los principios de mínimo privilegio y segregación de funciones dentro de DIRESA_PERU.

3.4. Política de respaldo y recuperación

Con el objetivo de garantizar la disponibilidad y recuperación de la información almacenada en **DIRESA_PERU**, se establece una política de respaldo basada en la combinación de copias completas, diferenciales y del registro de transacciones.

La estrategia propuesta permite limitar la pérdida potencial de información y reducir el tiempo necesario para recuperar la base de datos ante fallos, corrupción de datos o eliminación accidental.

Tipo de respaldo	Frecuencia propuesta	Finalidad
Completo	Una vez al día	Mantener una copia íntegra de la base de datos.
Diferencial	Cada 6 horas	Registrar los cambios realizados desde el último respaldo completo.
Log de transacciones	Cada 30 minutos	Reducir la cantidad de información que podría perderse ante una falla.

A partir de esta estrategia se establece un RPO (Recovery Point Objective) de 30 minutos, debido a la frecuencia propuesta para los respaldos del log de transacciones. Asimismo, se plantea un RTO (Recovery Time Objective) máximo de 2 horas, periodo dentro del cual se espera recuperar la disponibilidad de la base de datos ante una contingencia.

3.5. Prueba de restauración

Para verificar que los respaldos generados puedan utilizarse efectivamente ante una contingencia, se realizó una prueba de restauración sobre una base independiente denominada **DIRESA_PERU_RESTORE_TEST**.

La restauración se realizó utilizando los archivos de respaldo generados previamente. SQL Server confirmó la ejecución exitosa de la operación mediante el mensaje RESTORE DATABASE successfully processed.

Ver evidencia en ANEXOS — Capítulo III.

La prueba realizada permite comprobar que el mecanismo de respaldo implementado puede utilizarse para recuperar la información almacenada ante una contingencia.

3.6. Implementación de índice para optimización

Con el propósito de mejorar el rendimiento de las consultas realizadas sobre la tabla **PRESTACIONES**, se evaluó una consulta que filtra los registros mediante el campo FECHA_ATENCION.

Para optimizar esta operación se implementó un índice no agrupado (NONCLUSTERED) sobre dicha columna. Además, se incluyeron las columnas utilizadas por la consulta con el objetivo de reducir accesos adicionales a la tabla.

Script SQL:

Código: ver ANEXOS — Capítulo III.

La consulta utilizada para realizar la comparación recupera las prestaciones correspondientes al intervalo comprendido entre el 1 de junio y el 1 de julio de 2022.

3.7. Comparación de rendimiento antes y después del índice

Para evaluar el efecto del índice se utilizaron SET STATISTICS IO ON y SET STATISTICS TIME ON. Se ejecutó la misma consulta antes y después de crear el índice, manteniendo las mismas condiciones de filtrado.

3.7.1. Rendimiento antes de implementar el índice

Antes de crear el índice, la consulta presentó un mayor número de lecturas y un mayor tiempo de ejecución.

Ver evidencia en ANEXOS — Capítulo III.

Fuente: Elaboración propia.

3.7.2. Rendimiento después de implementar el índice

Después de implementar IX_PRESTACIONES_FECHA_ATENCION, se ejecutó nuevamente la misma consulta. Los resultados evidenciaron una reducción considerable en las operaciones de lectura y en el tiempo de ejecución.

Consulta SQL:

Código: ver ANEXOS — Capítulo III.

3.7.3. Comparación de resultados

Los resultados obtenidos antes y después de implementar el índice se resumen en la siguiente tabla:

Métrica	Antes	Después	Cambio
Scan count	9	1	↓ 88.9 %
Logical reads	8,809	39	↓ 99.56 %
Physical reads	271	2	↓ 99.26 %
Read-ahead reads	8,371	37	↓ 99.56 %
CPU	169 ms	0 ms	↓ ~100 %
Elapsed time	3,738 ms	130 ms	↓ 96.52 %
Filas devueltas	7,276	7,276	Sin variación

La comparación evidencia una reducción importante en las lecturas realizadas por SQL Server y en el tiempo de ejecución. Asimismo, la cantidad de registros obtenidos permaneció en **7,276 filas** antes y después de la optimización, lo que demuestra que la implementación del índice mejoró el rendimiento sin modificar el resultado de la consulta.

3.7.4. Análisis del plan de ejecución

Finalmente, se revisó el plan de ejecución generado por SQL Server después de implementar el índice. En este se observa la utilización de una operación Index Seek (NonClustered) sobre el índice creado para PRESTACIONES.

Ver evidencia en ANEXOS — Capítulo III.

Fuente: Elaboración propia.

El uso de Index Seek evidencia que SQL Server puede localizar los registros correspondientes al rango de fechas utilizando el índice, evitando realizar una exploración completa de la tabla y contribuyendo a la reducción observada en las lecturas y el tiempo de ejecución.

3.8. Cumplimiento de la Ley N.° 29733 e ISO/IEC 27001

La protección de la información utilizada en DIRESA_PERU se consideró tomando como referencia la Ley N.° 29733, Ley de Protección de Datos Personales, y su Reglamento vigente, aprobado mediante el Decreto Supremo N.° 016-2024-JUS.

La Ley N.° 29733 establece la necesidad de adoptar medidas técnicas, organizativas y legales que permitan proteger los datos personales frente a riesgos como la alteración, pérdida, tratamiento o acceso no autorizado. Asimismo, establece el deber de confidencialidad para quienes intervienen en el tratamiento de dicha información.

En el proyecto se implementaron controles técnicos relacionados con estos principios, complementados con buenas prácticas de seguridad de la información consideradas por ISO/IEC 27001, principalmente en materia de control de acceso, trazabilidad, integridad, disponibilidad y recuperación de la información.

3.8.1. Checklist de seguridad y cumplimiento

Aspecto evaluado	Medida aplicada en DIRESA_PERU	Evidencia del proyecto	Estado
Control de acceso	Usuarios y privilegios diferenciados	UsuarioAnalista y UsuarioAuditor	Implementado
Mínimo privilegio	Restricción de operaciones no autorizadas	DELETE y UPDATE rechazados	Implementado
Confidencialidad	Acceso limitado según las funciones del usuario	Roles y permisos de SQL Server	Implementado
Integridad	Validaciones y restricciones sobre modificaciones	Permisos y triggers de validación	Implementado
Trazabilidad	Registro automático de operaciones	LOG_AUDITORIA y trigger de auditoría	Implementado
Disponibilidad	Generación de respaldos de la base de datos	Política y archivos de respaldo	Implementado
Recuperación	Restauración de una copia de seguridad	DIRESA_PERU_RESTORE_TEST	Verificado
Protección frente a pérdida	Política de respaldo periódico	Respaldo completo, diferencial y log de transacciones	Implementado

Los controles implementados proporcionan evidencia técnica de medidas orientadas a proteger la confidencialidad, integridad y disponibilidad de la información. Sin embargo, el cumplimiento integral de la Ley N.° 29733 y de un sistema de gestión basado en ISO/IEC 27001 también requiere medidas organizativas, legales y de gestión que exceden el alcance técnico del presente proyecto académico.

3.9. Impacto de las decisiones de seguridad sobre los pacientes

Las decisiones de seguridad implementadas en DIRESA_PERU buscan reducir los riesgos asociados al acceso, modificación, pérdida o indisponibilidad de la información relacionada con los asegurados y las prestaciones de salud.

Un acceso no autorizado podría comprometer la confidencialidad de la información, mientras que una modificación indebida podría afectar su integridad y generar resultados incorrectos durante su análisis. Asimismo, la pérdida o indisponibilidad de los registros podría limitar temporalmente el acceso a información necesaria para los procesos de gestión y análisis.

Para reducir estos riesgos, se implementaron controles de acceso mediante usuarios con privilegios diferenciados, restricciones basadas en el principio de mínimo privilegio, mecanismos de auditoría, validaciones de integridad y una estrategia de respaldo y recuperación.

Riesgo	Posible consecuencia	Medida implementada
Acceso no autorizado	Exposición de información	Roles y privilegios diferenciados
Modificación indebida	Alteración de la integridad de los registros	Restricciones de permisos y triggers
Eliminación de información	Pérdida de registros necesarios para el análisis	Mínimo privilegio y respaldos
Operaciones no trazables	Dificultad para identificar acciones realizadas	LOG_AUDITORIA
Falla o pérdida de la base de datos	Indisponibilidad de información	Política de respaldo y restauración
Consultas de bajo rendimiento	Mayor tiempo para obtener información	Implementación de índices

De esta manera, las medidas adoptadas contribuyen a preservar la confidencialidad, integridad, disponibilidad y trazabilidad de la información utilizada en el proyecto. Esto resulta especialmente relevante en un contexto relacionado con información de salud, donde una gestión inadecuada de los datos puede afectar la privacidad de las personas y la confiabilidad de los análisis realizados.

CAPÍTULO IV

Exploración de paradigma NoSQL Y decisión de integración

4.1. Identificación y objetivo de la prueba

Se planteó evaluar la viabilidad de representar información del proyecto DataSalud Perú mediante un modelo NoSQL orientado a documentos, como alternativa complementaria al modelo relacional implementado en SQL Server.

Para esta prueba se seleccionó el Dataset 2: Consumo de medicamentos en recién nacidos, en lugar del dataset de Diabetes Mellitus, debido a que su estructura presenta una relación jerárquica de tres niveles (una prestación puede tener varios diagnósticos, y cada diagnóstico puede tener varios medicamentos asociados). Esta característica lo convierte en un caso más representativo para evaluar las ventajas del modelo de documentos, en el cual las relaciones anidadas pueden representarse dentro de una misma estructura, a diferencia del dataset de Diabetes Mellitus, cuya relación es más simple (un afiliado con una atención asociada).

El objetivo de la prueba fue determinar si MongoDB permite representar de forma coherente y consultable las relaciones PRESTACIONES → DIAGNOSTICOS → MEDICAMENTOS ya modeladas relacionalmente en SQL Server, evaluando además la facilidad de consulta resultante frente al modelo normalizado con claves foráneas.

4.2. Procedimiento y resultados

4.2.1 Diseño del documento

Se definió una colección “Recien_Nacidos”, donde cada documento representa un episodio de atención (equivalente a una fila de la tabla PRESTACIONES). Se utilizó “ID_REGISTRO_REL” como _id del documento, dado que este campo ya es único por episodio en la fuente original, evitando así generar un identificador adicional. Los diagnósticos asociados a cada episodio se representaron como un arreglo anidado (diagnosticos), y dentro de cada diagnóstico se anidó a su vez un arreglo de medicamentos (medicamentos), respetando la relación real de los datos, en la cual un medicamento se asocia a un diagnóstico específico dentro de un mismo episodio (ID_REGISTRO_REL + CODDIA).

Con fines de eficiencia de consulta, se optó por desnormalizar la información de los catálogos: En lugar de almacenar únicamente los códigos (CODDIA, COD_MEDICAMENTO) y requerir una operación de unión ($lookup) en cada consulta, se incorporó directamente el nombre del diagnóstico y los atributos descriptivos del medicamento (nombre, presentación, concentración, forma farmacéutica) dentro de cada documento.

4.2.2 Migración de datos

Se desarrolló un script en Python (openpyxl + pymongo) que:

1) Carga en memoria los catálogos de diagnósticos y medicamentos.
2) Agrupa el archivo de medicamentos por la clave compuesta (ID_REGISTRO_REL, CODDIA), enriqueciendo cada registro con los datos del catálogo correspondiente.
3) Agrupa el archivo de diagnósticos por ID_REGISTRO_REL, anidando en cada uno los medicamentos ya agrupados en el paso anterior.
4) Recorre el archivo de prestaciones y construye el documento final completo, aplicando conversiones de tipo necesarias para mantener consistencia con el modelo relacional (por ejemplo, ATE_CAPITA de texto 'S'/'N' a valor booleano, y normalización del código de medicamento al formato de cinco dígitos utilizado en el catálogo).

5) Inserta los documentos en MongoDB mediante operaciones upsert por lotes, y crea un índice sobre afiliado.codigo_anonimizado para optimizar consultas por paciente.

La carga produjo 290,469 documentos, cifra que coincide exactamente con el número de registros del archivo de Prestaciones, confirmando que ningún episodio quedó excluido del proceso.

Contraste manual, campo por campo, entre documentos individuales de la colección y sus filas de origen en los cuatro archivos Excel, confirmando correspondencia exacta en los datos del episodio, del diagnóstico y de los medicamentos, incluyendo el enriquecimiento correcto desde ambos catálogos.

Ejecución del filtro { "diagnosticos": { $size: 0 } } sobre la colección, el cual retornó 0 documentos, confirmando que la totalidad de los episodios migrados conservó al menos un diagnóstico asociado tras el proceso de unión.

Confirmando que la estructura resultante es consultable tanto desde la interfaz gráfica de Compass como desde línea de comandos.

4.2.3 Flexibilidad de esquema entre documentos

Una de las características distintivas del modelo NoSQL orientado a documentos frente al modelo relacional es la posibilidad de que distintos documentos de una misma colección presenten estructuras diferentes, sin necesidad de alterar un esquema declarado previamente. Para comprobar esta propiedad, se identificó un caso real dentro de la colección correspondiente a un recién nacido con estancia en la Unidad de Cuidados Intensivos (UCI) (servicio_prestacional.codigo: "068"), y se le agregó el campo adicional cuidados_intensivos, con información clínica que no resulta relevante para un internamiento sin complicaciones (código 051).

Documento actualizado con campo “cuidados_intensivos”:

Documento sin campo “cuidados_intensivos”:

Esta prueba confirma que MongoDB permite incorporar información específica de determinados casos, por ejemplo, variables propias de cuidados intensivos, procedimientos quirúrgicos u otras especialidades de forma incremental y localizada, sin requerir una operación equivalente a ALTER TABLE ni afectar el resto de los documentos de la colección. En el modelo relacional de SQL Server, representar esta misma variabilidad habría exigido modificar la tabla PRESTACIONES para todos los registros, o bien crear una tabla adicional con relación uno a uno, incrementando la complejidad del diseño para casos que representan una minoría de los datos.

4.2.4 Validación de operaciones CRUD sobre la colección.

Con el fin de comprobar que la colección Recien_Nacidos admite el ciclo completo de operaciones (Create, Read, Update, Delete) y no se limita a una carga estática de datos, se ejecutaron las cuatro operaciones básicas mediante mongosh sobre un documento de prueba (_id: "TEST_CRUD_001"), construido con la misma estructura anidada definida para el resto de la colección

.

CREATE

Primero se insertó el documento de prueba mediante insertOne().

Script:

Código: ver ANEXOS — Capítulo IV.

READ

Posteriormente, se confirmó su existencia mediante una consulta findOne() por _id.

Script:

Código: ver ANEXOS — Capítulo IV.

UPDATE

Se modificaron dos campos del documento (afiliado.sexo y atencion.dias_hospitalizacion) mediante updateOne() con el operador $set, y se verificó el cambio con una consulta posterior.

Script:

Código: ver ANEXOS — Capítulo IV.

DELETE

Finalmente, se eliminó el documento de prueba mediante deleteOne(), confirmando su eliminación mediante una consulta que retornó null.

Script:

Código: ver ANEXOS — Capítulo IV.

Las cuatro operaciones se ejecutaron exitosamente: el documento de prueba fue creado, consultado, modificado y finalmente eliminado, confirmando que la colección soporta el ciclo de vida completo de un documento sin restricciones adicionales de esquema.

4.2.5 Comparación técnica SQL Server vs. MongoDB.

A partir de los resultados obtenidos, se compararon ambos motores utilizando cuatro criterios técnicos aplicados al caso de uso del proyecto:

Criterio	SQL Server (DIRESA_PERU)	MongoDB (Recien_Nacidos)
Modelo de datos	Relacional normalizado (3FN); PRESTACIONES, DIAGNOSTICOS y MEDICAMENTOS relacionadas mediante clave foránea ID_REGISTRO_REL.	Orientado a documentos; diagnósticos y medicamentos anidados dentro de un mismo documento por episodio.

Escalabilidad	SQL Server escala principalmente de forma vertical, aunque soporta particionamiento horizontal.	MongoDB ofrece escalabilidad horizontal nativa mediante sharding, favorecida por el carácter autocontenido de sus documentos.
Consistencia	Fuerte (ACID), garantizada mediante TRY/CATCH, COMMIT/ROLLBACK y triggers de validación.	Ofrece atomicidad a nivel de documento y permite configurar validaciones de esquema. En este proyecto, la validación de los datos migrados se realizó principalmente mediante Python.
Latencia de consulta	Recuperar un episodio completo en SQL Server requiere JOIN entre PRESTACIONES, DIAGNOSTICOS y MEDICAMENTOS	En MongoDB se obtiene con una única operación findOne() sin uniones.

La comparación evidencia que ambos motores cumplen funciones complementarias dentro del proyecto. SQL Server proporciona un modelo relacional normalizado, con controles de integridad, transacciones y auditoría, mientras que MongoDB simplifica la consulta de la estructura jerárquica del dataset de Recién Nacidos mediante documentos anidados, evitando la necesidad de realizar uniones entre tablas. Aunque MongoDB dispone de mecanismos nativos de validación, en esta implementación se priorizó la validación de los datos durante el proceso de migración mediante Python.

4.3. Veredicto de viabilidad y ajustes al proyecto.

A partir de los resultados, se decidió incorporar una arquitectura híbrida: SQL Server y MongoDB como motores complementarios, sin que uno reemplace al otro.

SQL Server garantiza consistencia fuerte de forma nativa (TRY/CATCH, COMMIT/ROLLBACK, triggers), por lo que se mantiene como motor de sistema de registro para ambos dominios, responsable de la integridad y auditoría. MongoDB no valida los datos nativamente en esta implementación (depende del script de migración), por lo que no es apto como repositorio único; sí demostró ventajas concretas: Recupera un episodio completo con findOne() sin uniones entre tablas, y permite agregar campos a un solo documento sin alterar el resto de la colección, algo que en SQL Server exigiría modificar la tabla completa.

Por ello se descartó tanto usar MongoDB de forma exclusiva (sin mecanismos nativos de integridad) como limitar el proyecto a SQL Server. La arquitectura final queda así: SQL Server

como motor operacional y de integridad, y MongoDB como motor analítico complementario para Recien_Nacidos.

CAPÍTULO V

Diseño del Data Warehouse y procesos ETL

Para complementar la base de datos transaccional DIRESA_PERU, se implementó el Data Warehouse DW_DataSalud, orientado al análisis de la información proveniente de los datasets de prestaciones de recién nacidos y afiliados con diagnóstico de diabetes mellitus.

5.1. Enfoque dimensional

Se empleó un enfoque dimensional basado en la metodología Kimball, organizando la información mediante tablas de hechos y dimensiones. La granularidad fue definida antes de realizar la carga para conservar el nivel de detalle de los datos originales.

Se implementaron las siguientes dimensiones principales:

- **DimTiempo:** representa las fechas asociadas a los hechos y permite realizar análisis temporales.
- **DimUbicacion:** almacena la información geográfica utilizada para analizar los registros según ubigeo.
- **DimServicio:** contiene los códigos y descripciones de los servicios prestacionales correspondientes al dataset de recién nacidos.
- **DimRecienNacido:** representa a los recién nacidos mediante su código anonimizado y las características necesarias para el análisis.
- **DimAfiliadoDiabetes:** representa a los afiliados del dataset de diabetes mediante un identificador interno, código anonimizado, edad y sexo.

Las dimensiones utilizan claves sustitutas dentro del Data Warehouse, permitiendo independizar las relaciones analíticas de las claves operacionales de DIRESA_PERU.

5.2. Tablas de hechos y granularidad

Se implementaron dos tablas de hechos debido a que los datasets representan procesos distintos y poseen granularidades diferentes.

5.2.1. FactPrestacionesRN

La tabla FactPrestacionesRN representa una prestación registrada para un recién nacido. Contiene las claves hacia las dimensiones de tiempo, ubicación, servicio y recién nacido, además de atributos cuantitativos y descriptivos necesarios para el análisis, como edad en días, peso y días de hospitalización.

La granularidad corresponde a una prestación registrada para un recién nacido.

La carga final fue de 290 469 registros, coincidiendo con la cantidad de prestaciones válidas del dataset utilizado.

5.2.2. FactAtencionDiabetes

La tabla FactAtencionDiabetes representa la información de atención asociada a cada afiliado con diagnóstico de diabetes. Se relaciona con las dimensiones de afiliado, tiempo y ubicación e incorpora información como tipo de diabetes, comorbilidades, cantidad de atenciones, valores netos y hospitalización.

La granularidad corresponde a un registro de atención asociado a un afiliado con diagnóstico de diabetes.

La carga final fue de 850 366 registros, correspondientes a los registros válidos utilizados para el análisis.

La separación de ambas tablas evita establecer relaciones clínicas artificiales entre dos datasets independientes y permite analizar cada proceso según su propia granularidad.

5.3. Implementación del modelo dimensional

El modelo dimensional se implementó en SQL Server mediante cinco tablas de dimensiones (DimTiempo, DimUbicacion, DimServicio, DimRecienNacido y DimAfiliadoDiabetes) y dos tablas de hechos (FactPrestacionesRN y FactAtencionDiabetes), utilizando claves primarias, foráneas y claves sustitutas para mantener las relaciones del modelo.

Ver evidencia en ANEXOS — Capítulo V.

Fuente: Elaboración propia.

5.4. Proceso ETL

El proceso ETL permitió trasladar la información desde la base operacional DIRESA_PERU hacia DW_DataSalud.

5.4.1. Extracción

Los datos fueron obtenidos desde las tablas operacionales de DIRESA_PERU, principalmente PRESTACIONES, AFILIADOS y ATENCIONES_DIABETES.

5.4.2. Transformación

Antes de la carga se aplicaron reglas destinadas a garantizar la consistencia del modelo dimensional. Entre las principales transformaciones se encuentran:

- Exclusión de registros utilizados únicamente durante las pruebas de implementación.
- Exclusión del código de servicio 001 durante la construcción de DimServicio, de acuerdo con las reglas de transformación aplicadas en el proceso ETL.
- Uso de valores distintos para evitar duplicados en las dimensiones.
- Resolución de 22 casos en los que un mismo código anonimizado de recién nacido presentaba ambos valores de sexo, asignándoles NO DETERMINADO en la dimensión.
- Utilización de claves sustitutas para relacionar dimensiones y hechos.
- Conservación de valores nulos cuando representan ausencia legítima de información, como determinados registros de peso.

5.4.3. Carga

Las dimensiones se cargaron antes que las tablas de hechos para garantizar la existencia de las claves referenciadas. Posteriormente, se cargaron FactPrestacionesRN y FactAtencionDiabetes, verificándose 0 registros sin correspondencia con las dimensiones requeridas.

Figura 19. Ejecución del proceso ETL para la carga de DW_DataSalud.

Fuente: Elaboración propia.

5.5. Control y trazabilidad del ETL

Para controlar las ejecuciones se implementó LogETL, donde se registra el proceso ejecutado, fechas, cantidad de registros, estado y mensaje descriptivo.

La carga inicial registró los siguientes resultados:

Proceso	Registros	Estado
FactPrestacionesRN	290 469	EXITOSO
FactAtencionDiabetes	850 366	EXITOSO

Asimismo, SP_VALIDAR_ETL_DATASALUD verifica las cantidades cargadas y registra el resultado en LogETL mediante control de excepciones. La validación final procesó 1 140 835 registros con estado EXITOSO.

Figura 20. Registro de control y validación del proceso ETL.

Fuente: Elaboración propia.

5.6. Validación del Data Warehouse

Después de la carga se realizaron consultas de control para comprobar que las cantidades almacenadas en el Data Warehouse coincidieran con las fuentes utilizadas.

Tabla de hechos	Registros
FactPrestacionesRN	290 469
FactAtencionDiabetes	850 366
Total validado	**1 140 835**

Los resultados confirmaron que la carga dimensional se realizó correctamente y que las tablas de hechos mantienen la cantidad esperada de registros después de aplicar las reglas de transformación definidas.

Figura 21. Validación de registros cargados en las tablas de hechos.

Fuente: Elaboración propia.

5.7. Justificación de Kimball frente a Inmon

Se seleccionó el enfoque **Kimball** porque el proyecto requiere obtener estructuras analíticas directamente orientadas a procesos concretos y posteriormente utilizarlas para indicadores, consultas OLAP y visualizaciones.

Criterio	Kimball	Inmon
Construcción	Incremental mediante modelos dimensionales	Data Warehouse corporativo centralizado

Orientación	Procesos de negocio y análisis	Integración empresarial global
Complejidad inicial	Menor para el alcance del proyecto	Mayor por el diseño corporativo previo
Aplicación en DataSalud	Permite modelar directamente prestaciones y diabetes	Requeriría una capa corporativa adicional

Para **DataSalud Perú**, Kimball permite desarrollar modelos dimensionales separados para las prestaciones de recién nacidos y las atenciones de diabetes, manteniendo dimensiones reutilizables como tiempo y ubicación. El enfoque Inmon sería más apropiado si el proyecto evolucionara hacia un repositorio corporativo de gran escala que integrara numerosos sistemas heterogéneos del sector salud y necesitara establecer primero un modelo empresarial centralizado antes de construir estructuras analíticas específicas.

Por lo tanto, para el alcance actual se adoptó **Kimball** debido a su adecuación al desarrollo incremental, orientación analítica y compatibilidad directa con las posteriores consultas OLAP e indicadores de Business Intelligence.

CAPÍTULO VI

Construcción de dashboard bi y análisis OLAP

Para la etapa de Business Intelligence se utilizó **Power BI Desktop**, conectándolo directamente con la base de datos **DW_DataSalud** implementada previamente en SQL Server. La conexión se realizó mediante el modo **Importar**, incorporando únicamente las tablas necesarias para el análisis dimensional.

La implementación tiene como finalidad representar mediante indicadores y visualizaciones la información correspondiente a las prestaciones de recién nacidos y a los afiliados con diagnóstico de diabetes mellitus, utilizando como fuente el modelo dimensional desarrollado en el Data Warehouse.

6.1. Implementación del modelo dimensional en Power BI

Se importaron las siguientes tablas de **DW_DataSalud**:

- DimTiempo.
- DimUbicacion.
- DimServicio.
- DimRecienNacido.
- DimAfiliadoDiabetes.
- FactPrestacionesRN.

- FactAtencionDiabetes.

La tabla **LogETL** no fue incorporada al modelo de Power BI debido a que cumple una función de control y seguimiento del proceso ETL y no constituye una tabla destinada al análisis dimensional.

En la Vista de modelo de Power BI se verificaron Relaciones de uno a muchos (1:N) entre las dimensiones y las tablas de hechos. El modelo implementado corresponde a una **constelación de esquemas estrella**, debido a la existencia de dos procesos analíticos independientes que comparten dimensiones conformadas.

FactPrestacionesRN se relaciona con DimTiempo, DimUbicacion, DimServicio y DimRecienNacido. Por su parte, **FactAtencionDiabetes** se relaciona con DimTiempo, DimUbicacion y DimAfiliadoDiabetes. De esta manera, **DimTiempo** y **DimUbicacion** funcionan como dimensiones compartidas entre ambos procesos.

Ver evidencia en ANEXOS — Capítulo VI.

Fuente: Elaboración propia.

6.2. Definición e implementación de KPIs

Sobre el modelo dimensional se implementaron medidas DAX destinadas a obtener indicadores relevantes para el análisis de ambos datasets.

6.2.1. KPI 1: Total de prestaciones de recién nacidos

Este indicador contabiliza los registros existentes en la tabla de hechos correspondiente a las prestaciones de recién nacidos.

Código: ver ANEXOS — Capítulo VI.

Resultado: 290 469 prestaciones.

El indicador permite conocer el volumen total de prestaciones de recién nacidos incorporadas al modelo analítico.

6.2.2. KPI 2: Total de afiliados con diabetes

Se utilizó el conteo distinto del identificador del afiliado para determinar la cantidad de afiliados representados en la tabla de hechos.

Código: ver ANEXOS — Capítulo VI.

Resultado: 850 366 afiliados.

El uso de DISTINCTCOUNT permite contabilizar los identificadores distintos presentes en la tabla de hechos.

6.2.3. KPI 3: Peso promedio de recién nacidos

Este indicador calcula el promedio del peso registrado en las prestaciones de recién nacidos.

Código: ver ANEXOS — Capítulo VI.

Resultado: 3.19 kg.

Los valores nulos de peso no son considerados por la función AVERAGE, evitando interpretarlos incorrectamente como valores iguales a cero.

6.2.4. KPI 4: Porcentaje de registros de diabetes con diagnóstico de hipertensión

Este indicador determina la proporción de registros de la tabla de hechos de diabetes cuyo campo **ConDxHipertension** presenta el valor "SI".

Código: ver ANEXOS — Capítulo VI.

Resultado: 40.3 %.

El indicador representa la proporción de registros analizados que presentan diagnóstico de hipertensión dentro del conjunto de datos de diabetes utilizado en el proyecto.

Los cuatro indicadores permiten resumir información relevante de ambos procesos analíticos y sirven como base para la construcción de las visualizaciones del dashboard.

Ver evidencia en ANEXOS — Capítulo Vi.

Fuente: Elaboración propia.

6.3. Construcción del dashboard BI

A partir de los indicadores implementados se construyó el dashboard de **DataSalud Perú**, integrando visualizaciones orientadas al análisis de los dos procesos representados en el Data Warehouse.

El dashboard incorpora los cuatro KPIs principales junto con gráficos y elementos de filtrado que permiten explorar la información desde diferentes dimensiones del modelo, principalmente tiempo, ubicación y servicio.

Para evitar interpretaciones incorrectas, la información de prestaciones de recién nacidos y la correspondiente a diabetes se analiza respetando las tablas de hechos y dimensiones relacionadas con cada proceso.

Ver evidencia en ANEXOS — Capítulo Vi.

Fuente: Elaboración propia.

6.4. Metadatos y gobernanza de los indicadores

Para mantener una interpretación uniforme de los indicadores utilizados en el dashboard, se documentaron sus principales metadatos, especificando su definición, fuente y criterio de cálculo.

Indicador	Definición	Fuente	Criterio de cálculo
Total de prestaciones RN	Cantidad total de prestaciones de recién nacidos incorporadas al modelo analítico	FactPrestacionesRN	Conteo de filas
Total de afiliados con diabetes	Cantidad de identificadores distintos de afiliados representados en la tabla de hechos	FactAtencionDiabetes	Conteo distinto de IdAfiliadoDiabetes
Peso promedio RN	Promedio del peso disponible en las prestaciones de recién nacidos	FactPrestacionesRN	Promedio de Peso, excluyendo valores nulos

Porcentaje de hipertensión	Proporción de registros de diabetes con diagnóstico de hipertensión	FactAtencionDiabetes	Registros con ConDxHipertension = "SI" / total de registros

La gobernanza de los indicadores se basa en utilizar como fuente única las tablas del Data Warehouse **DW_DataSalud**, evitando realizar cálculos sobre fuentes externas diferentes a las utilizadas durante el proceso ETL. Asimismo, las definiciones y medidas DAX permiten establecer criterios uniformes para interpretar los resultados mostrados en el dashboard.

El uso de códigos anonimizados en las dimensiones relacionadas con los pacientes permite mantener el análisis orientado a información estadística, evitando incorporar identificadores personales directos que no sean necesarios para los objetivos analíticos del proyecto.

6.5. Análisis OLAP

El modelo dimensional implementado permite realizar operaciones OLAP mediante las dimensiones disponibles en Power BI. Estas operaciones permiten analizar la información desde diferentes niveles de detalle sin modificar los datos almacenados en el Data Warehouse.

Para demostrar esta capacidad, se implementó una operación OLAP de tipo drilldown sobre una de las visualizaciones del dashboard.

Se utilizó la dimensión DimTiempo para realizar una operación de drilldown, pasando desde un nivel temporal general hacia niveles progresivamente más detallados.

Jerarquía implementada:

Año → Mes → Día

De esta manera, inicialmente se puede visualizar la cantidad de prestaciones agregada por año y posteriormente descender hacia los meses y días correspondientes.

Ver evidencia en ANEXOS — Capítulo VI.

Fuente: Elaboración propia.

Resultado de la operación:

La operación permitió modificar el nivel de detalle de la información analizada, pasando de las prestaciones agrupadas por año a su distribución por mes y día, sin modificar los registros almacenados en el Data Warehouse.

6.6. Preguntas de negocio y análisis de resultados

El dashboard fue planteado para transformar los datos almacenados en **DW_DataSalud** en información que permita responder preguntas concretas relacionadas con los procesos analizados.

6.6.1. Pregunta de negocio 1

¿Qué provincias concentran la mayor cantidad de prestaciones de recién nacidos?

Para responder esta pregunta se analizó la cantidad de prestaciones registradas en FactPrestacionesRN según provincia. Los resultados muestran una mayor concentración de prestaciones en Santa, seguida por Huaraz, mientras que las demás provincias presentan cantidades considerablemente menores.

Ver evidencia en ANEXOS — Capítulo VI.

Fuente: Elaboración propia.

6.6.2. Pregunta de negocio 2

¿Cómo se distribuyen los registros de diabetes según la presencia de hipertensión?

El análisis muestra que 40.3 % de los afiliados con diabetes presenta diagnóstico de hipertensión. Esto indica que aproximadamente cuatro de cada diez registros analizados presentan esta comorbilidad.

Ver evidencia en ANEXOS — Capítulo VI.

Fuente: Elaboración propia.

6.7. Validación de los resultados del dashboard

Los indicadores obtenidos en Power BI fueron contrastados con las tablas de hechos de DW_DataSalud, verificándose 290 469 prestaciones de recién nacidos y 850 366 registros de afiliados con diabetes.

Esta correspondencia confirma la consistencia entre el Data Warehouse y la información utilizada en el dashboard, permitiendo su análisis mediante indicadores, visualizaciones y operaciones
OLAP.

CAPITULO VII

Análisis Big Data mediante Apache Spark y reflexión de escalabilidad

7.1. Análisis Big Data mediante Apache Spark

7.1.1. Objetivo del análisis

Con el propósito de evaluar la viabilidad de tecnologías Big Data en el proyecto DataSalud Perú, se implementó un entorno de procesamiento con Apache Spark mediante PySpark. El análisis permitió ejecutar operaciones de transformación, filtrado, agregación y combinación de datos provenientes de conjuntos de datos abiertos del sector salud peruano.

Asimismo, se realizó una comparación experimental con SQL Server para identificar diferencias en los tiempos de ejecución y analizar la conveniencia de utilizar un motor de procesamiento distribuido como complemento de la arquitectura de datos propuesta.

7.1.2. Entorno tecnológico y metodología

La implementación se desarrolló en Google Colab, utilizando PySpark en modo local. Se emplearon dos interfaces de procesamiento: DataFrames y SparkSQL, cumpliendo con el requisito de utilizar al menos dos objetos de Apache Spark.

El procedimiento experimental comprendió la preparación de los datos, la ejecución de cinco consultas analíticas equivalentes en PySpark y SQL Server, y el registro de sus tiempos de ejecución.

Para PySpark se distinguió entre una ejecución inicial o fría y ejecuciones posteriores o calientes. Los resultados comparativos utilizaron las medianas de las ejecuciones calientes, con el propósito de disminuir la influencia de variaciones puntuales.

Evidencia: ver ANEXOS — Capítulo VII

7.1.3. Procesamiento y operaciones analíticas

Se implementaron cinco consultas para evaluar diferentes operaciones sobre los conjuntos de datos:

- **Q1:** Estadística descriptiva de los registros de diabetes, agrupados por tipo y sexo, mediante DataFrames.
- **Q2:** Cálculo del porcentaje de registros con diagnóstico de hipertensión mediante SparkSQL.
- **Q3:** Agregación de información por UBIGEO mediante SparkSQL.
- **Q4:** Unión de las tablas de prestaciones, diagnósticos y medicamentos de recién nacidos mediante SparkSQL.
- **Q5:** Filtrado de prestaciones por rango de fechas mediante DataFrames.

Estas operaciones permitieron evaluar distintos patrones de procesamiento, incluyendo agrupaciones, cálculos de indicadores, uniones y filtros.

Evidencia: ver ANEXOS — Capítulo VII

7.1.4. Comparación de tiempos de ejecución

Se registraron los siguientes resultados experimentales:

Consulta	PySpark caliente (ms)	SQL Server (ms)	Motor más rápido
Q1	1020,9	619,5	SQL Server
Q2	252,7	315,5	PySpark
Q3	750,6	510,0	SQL Server
Q4	7594,3	1068,0	SQL Server
Q5	1156,2	86,5	SQL Server

SQL Server obtuvo menores tiempos de ejecución en cuatro de las cinco consultas. Las mayores diferencias se observaron en Q4 y Q5, donde registró tiempos aproximadamente 7,1 y 13,4 veces menores que PySpark, respectivamente.

Por otra parte, PySpark presentó un mejor rendimiento en Q2, con 252,7 ms frente a los 315,5 ms de SQL Server.

Evidencia: ver ANEXOS — Capítulo VII

Los resultados reflejan el comportamiento de ambos motores en los entornos utilizados. Sin embargo, las diferencias de hardware, almacenamiento y optimización impiden interpretar estos tiempos como una comparación universal de rendimiento.

7.2. Análisis de eficiencia, escalabilidad y adecuación

7.2.1. Evaluación de eficiencia

La evaluación de eficiencia permitió identificar diferencias en el comportamiento de Apache Spark y SQL Server frente a distintas operaciones analíticas.

En PySpark se observaron tiempos de ejecución fría superiores a los tiempos de ejecución caliente. Este comportamiento puede asociarse con costos iniciales de planificación, preparación de tareas y lectura de información.

SQL Server presentó un rendimiento favorable en la mayoría de las consultas evaluadas. Sus mecanismos de optimización e indexación pueden beneficiar operaciones específicas, aunque la influencia de cada mecanismo depende del plan de ejecución utilizado.

Apache Spark, por su parte, ofrece herramientas para transformar y analizar datos mediante procesamiento paralelo, además de compatibilidad con formatos columnares como Parquet.

7.2.2. Evaluación experimental de escalabilidad

Para estudiar el comportamiento de ambos motores ante un aumento del volumen lógico de datos, se realizaron pruebas con factores de crecimiento de 1×, 5×, 10× y 20× sobre el conjunto de diabetes.

Factor	Registros lógicos	PySpark (ms)	SQL Server (ms)
1×	850 366	1861,7	452,5
5×	4 251 830	1893,5	1503,5
10×	8 503 660	2224,0	1509,0
20×	17 007 320	3367,6	1543,0

Evidencia: ver ANEXOS — Capítulo VII

PySpark registró un aumento de aproximadamente 1,81 veces en su tiempo de ejecución cuando el volumen lógico se multiplicó por 20. SQL Server presentó un incremento aproximado de 3,41 veces entre ambos extremos, aunque sus tiempos permanecieron relativamente estables entre los factores 5× y 20×.

Este comportamiento puede estar influenciado por las estrategias de optimización de cada motor. Por tanto, la prueba permite estudiar tendencias de rendimiento ante la replicación lógica de registros, pero no constituye una demostración de escalabilidad horizontal real.

7.2.3. Adecuación a la naturaleza de los datos

La adecuación de Apache Spark se analizó considerando las cuatro dimensiones de Big Data: volumen, velocidad, variedad y veracidad.

Volumen: Los conjuntos de datos evaluados comprenden millones de registros, un tamaño que puede ser procesado mediante una infraestructura local. Por ello, el uso de un clúster distribuido no resulta indispensable para el escenario actual.

Velocidad: Los datos corresponden a registros publicados por periodos o cortes, por lo que su procesamiento puede realizarse mediante cargas por lotes, sin requerir procesamiento continuo de eventos.

Variedad: La información se encuentra principalmente en estructuras tabulares, aunque presenta relaciones jerárquicas entre prestaciones, diagnósticos y medicamentos. Apache Spark permite procesar diferentes formatos, incluidos CSV, Parquet y JSON.

Veracidad: La preparación de los datos incluyó operaciones de revisión, limpieza y tratamiento de inconsistencias. Estas actividades son relevantes para garantizar la calidad de los análisis posteriores.

7.2.4. Limitaciones del experimento

Las pruebas se realizaron en entornos distintos: PySpark se ejecutó en Google Colab en modo local y SQL Server en un equipo independiente.

Adicionalmente, las mediciones estuvieron sujetas a diferencias en los mecanismos de caché, formatos de almacenamiento, planes de ejecución y recursos disponibles.

Las pruebas de escalabilidad utilizaron replicación lógica de registros y no incorporaron nodos adicionales ni ampliación física equivalente de los conjuntos de datos. Por ello, los resultados deben interpretarse dentro de las condiciones experimentales descritas.

7.2.5. Conclusiones y recomendaciones

Los resultados obtenidos evidencian que SQL Server fue más rápido en cuatro de las cinco consultas analíticas evaluadas, mientras que PySpark presentó un mejor tiempo en la consulta Q2.

En las pruebas de escalabilidad, ambos motores mostraron comportamientos distintos ante el aumento del volumen lógico de registros. Sin embargo, las condiciones experimentales no permiten determinar de manera concluyente cuál ofrecería mayor rendimiento en una infraestructura distribuida.

Para el volumen actual de DataSalud Perú, se recomienda mantener SQL Server como sistema relacional principal, aprovechando sus mecanismos de integridad, seguridad, auditoría y optimización.

Apache Spark se propone como herramienta complementaria para procesos analíticos, limpieza, transformación e integración de mayores volúmenes de información en futuras ampliaciones del proyecto.

Esta propuesta resulta coherente con la arquitectura híbrida desarrollada, en la que SQL Server gestiona la información relacional y MongoDB permite representar estructuras documentales. La incorporación de PySpark amplía las posibilidades de procesamiento analítico sin reemplazar los componentes existentes.

Ver evidencia en ANEXOS — Capítulo VII.

CAPÍTULO VIII

Integración, reflexión ética y articulación curricular.

8.1. Integración de los componentes del proyecto

El proyecto DataSalud Perú integra diferentes tecnologías y enfoques de gestión de datos con el propósito de construir una solución para el almacenamiento, procesamiento, protección y análisis de información del sector salud.

La arquitectura desarrollada contempla SQL Server como sistema de almacenamiento relacional, MongoDB como alternativa NoSQL orientada a documentos, un Data Warehouse diseñado bajo el enfoque dimensional de Kimball, Power BI para la visualización de indicadores y Apache Spark para la evaluación de operaciones analíticas sobre volúmenes crecientes de información.

La integración se realiza a nivel de arquitectura y procesos, debido a que los conjuntos de datos de Diabetes Mellitus y prestaciones de recién nacidos corresponden a dominios diferentes y no presentan una relación clínica directa.

En conjunto, los componentes permiten abordar diferentes necesidades: SQL Server proporciona mecanismos de integridad, transacciones y auditoría; MongoDB facilita la representación de estructuras jerárquicas; el Data Warehouse organiza la información para

el análisis; Power BI permite interpretar indicadores; y Apache Spark complementa el proyecto mediante pruebas de procesamiento y escalabilidad.

8.2. Repositorio Git y organización de los artefactos

Para facilitar la trazabilidad, reproducibilidad y mantenimiento del proyecto, se establece un repositorio Git como punto central de organización de los artefactos desarrollados por el equipo.

El repositorio está destinado a contener los scripts de creación de tablas, procedimientos almacenados, triggers, índices, políticas de seguridad, respaldos documentados, procesos ETL, consultas MongoDB, medidas DAX, archivos de Power BI y el notebook de Apache Spark.

La documentación se complementa con un archivo README.md que describe el objetivo del proyecto, las tecnologías utilizadas, la estructura de carpetas y las instrucciones generales para consultar o ejecutar los componentes.

Repositorio del proyecto: https://github.com/Dyl4nMD/LosDescendientesdeTutankamon_CIIN1021P_EF_REPO

La publicación de los artefactos debe considerar la protección de los datos personales y la exclusión de credenciales, contraseñas, cadenas de conexión privadas y archivos que no cuenten con autorización para su redistribución.

8.3. Reflexión ética y responsabilidad profesional

El tratamiento de información relacionada con la salud exige que las decisiones técnicas se evalúen también desde una perspectiva ética. El equipo reconoce que la responsabilidad profesional no se limita al funcionamiento del sistema, sino que incluye la protección de la privacidad, la integridad y la disponibilidad de la información.

Esta reflexión se desarrolla tomando como referencia los principios de responsabilidad profesional del Código de Ética del Colegio de Ingenieros del Perú y las medidas de protección de datos personales consideradas en el proyecto.

8.3.1. Dilema ético 1: disponibilidad de información frente a privacidad de los pacientes

La utilización de datos de salud permite desarrollar indicadores útiles para la planificación y el análisis de servicios. Sin embargo, también puede generar riesgos de exposición de información personal cuando los registros contienen identificadores o combinaciones de atributos que facilitan la identificación de personas.

El dilema consiste en determinar cuánta información debe mantenerse disponible para el análisis sin comprometer innecesariamente la privacidad de los pacientes.

Como respuesta, el proyecto contempla controles de acceso diferenciados, aplicación del principio de mínimo privilegio y uso de identificadores anonimizados cuando corresponde. Asimismo, se reconoce la importancia de evitar la publicación de información individual identificable en los repositorios y productos analíticos.

Estas decisiones buscan reducir el riesgo de accesos indebidos, exposición de información sensible y posibles afectaciones a la privacidad de las personas cuyos datos son procesados.

8.3.2. Dilema ético 2: facilidad de operación frente a integridad y seguridad de los datos

Otorgar permisos amplios a todos los usuarios puede facilitar determinadas operaciones de mantenimiento y consulta. Sin embargo, esta práctica incrementa el riesgo de modificaciones no autorizadas, eliminación accidental de registros y pérdida de trazabilidad.

El dilema consiste en equilibrar la facilidad de operación del sistema con la necesidad de proteger la información y garantizar su confiabilidad.

Para afrontar este riesgo, el equipo implementó permisos diferenciados en SQL Server, restricciones de operaciones no autorizadas, procedimientos almacenados con control transaccional, triggers de validación y mecanismos de auditoría.

Adicionalmente, se estableció una política de respaldo y se realizó una prueba de restauración, con el propósito de evaluar la capacidad de recuperación ante una contingencia.

Estas decisiones contribuyen a disminuir la probabilidad de alteraciones indebidas y pérdidas de información. Su importancia radica en que los errores de integridad pueden producir indicadores incorrectos y afectar las decisiones administrativas sustentadas en dichos resultados.

8.3.3. Posición ética del equipo

El equipo considera que la seguridad, la privacidad y la calidad de los datos deben incorporarse desde el diseño de las soluciones tecnológicas y no únicamente como controles posteriores.

Por ello, se prioriza el uso responsable de los datos abiertos, la restricción de accesos según funciones, la trazabilidad de operaciones y la documentación de las transformaciones aplicadas a las fuentes originales.

Asimismo, se reconoce que la implementación de controles técnicos no garantiza por sí sola el cumplimiento integral de todas las obligaciones legales y profesionales, por lo que estas medidas deben complementarse con procedimientos organizativos y una revisión continua de los riesgos.

8.4. Articulación curricular

8.4.1. Saberes previos del curso Base de Datos

El desarrollo de DataSalud Perú permitió retomar y profundizar conocimientos adquiridos en el curso Base de Datos, principalmente en el diseño de modelos entidad-relación, identificación de claves primarias y foráneas, establecimiento de relaciones, normalización y elaboración de consultas SQL.

Estos conocimientos sirvieron como fundamento para construir la base de datos relacional, organizar los registros de las fuentes seleccionadas y establecer mecanismos de integridad referencial.

A partir de esta base, el proyecto incorporó procedimientos almacenados, transacciones, triggers, auditoría, administración de privilegios y optimización mediante índices, ampliando el alcance de los conocimientos previamente adquiridos.

8.4.2. Proyección hacia el curso Ciencia de Datos

El proyecto proporciona una base para el desarrollo de competencias vinculadas con la preparación, transformación, exploración y análisis de conjuntos de datos.

Las actividades de diagnóstico de calidad, implementación de procesos ETL, construcción de indicadores y procesamiento mediante PySpark permiten comprender la importancia de contar con información confiable antes de aplicar técnicas analíticas más avanzadas.

Asimismo, la experiencia con Power BI y el Data Warehouse facilita la interpretación de patrones y la comunicación de resultados, competencias que podrán profundizarse posteriormente mediante técnicas estadísticas, modelos predictivos y aprendizaje automático.

8.4.3. Proyección hacia el curso Ingeniería de Software

DataSalud Perú también contribuye al desarrollo de competencias relacionadas con el análisis de requerimientos, diseño de arquitecturas, integración de componentes, documentación técnica, pruebas y mantenimiento de sistemas.

La organización de scripts y artefactos mediante Git favorece la trazabilidad de los cambios y el trabajo colaborativo, mientras que la implementación de mecanismos de validación, auditoría y recuperación permite comprender la importancia de los requisitos no funcionales.

Estos conocimientos servirán como base para desarrollar soluciones de software más completas, seguras, mantenibles y orientadas a necesidades reales de los usuarios.

8.5. Conclusiones de la integración

El proyecto DataSalud Perú permitió integrar conocimientos de bases de datos relacionales, tecnologías NoSQL, almacenamiento dimensional, inteligencia de negocios y procesamiento Big Data en una arquitectura orientada al análisis de información sanitaria.

La experiencia demuestra que cada tecnología responde a necesidades diferentes y que su selección debe justificarse considerando la estructura de los datos, los requerimientos de consulta, el rendimiento, la seguridad y la facilidad de mantenimiento.

Asimismo, el proyecto permitió reconocer que las decisiones técnicas pueden tener consecuencias sobre la privacidad, la confiabilidad de los resultados y la disponibilidad de información relacionada con las personas.

Finalmente, la integración de los componentes fortalece las competencias adquiridas en Base de Datos y establece una base de aprendizaje para cursos posteriores como Ciencia de Datos e Ingeniería de Software.

ANEXOS

Las evidencias gráficas y los bloques de código/query se presentan a continuación, organizados según el capítulo al que pertenecen.

CAPÍTULO I

Evidencias gráficas

Figura 1. Identificación de valores faltantes en el dataset de Diabetes Mellitus.

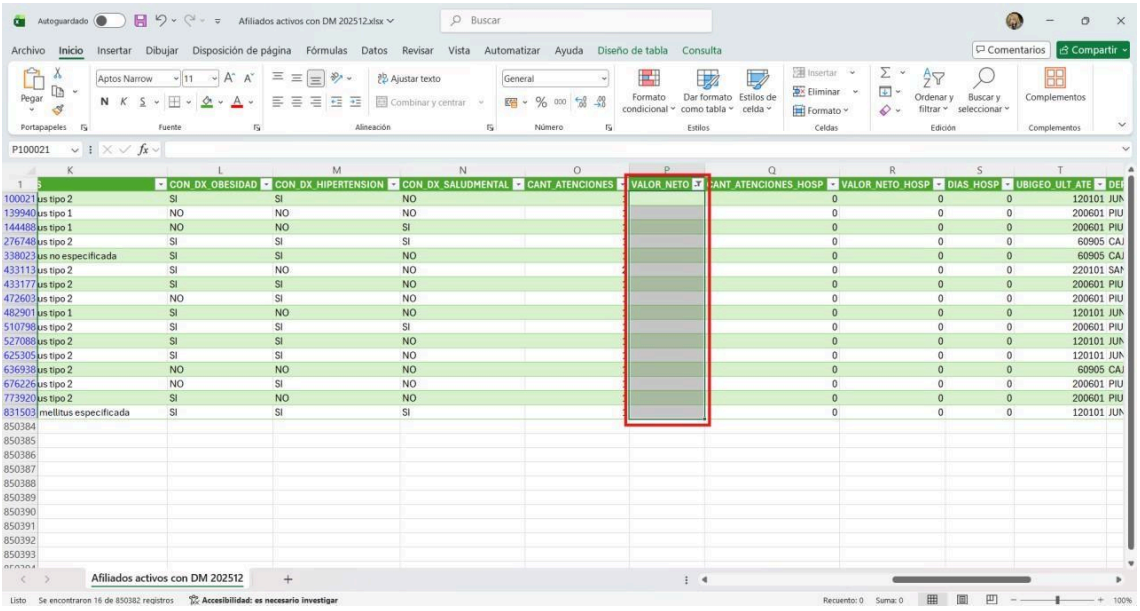
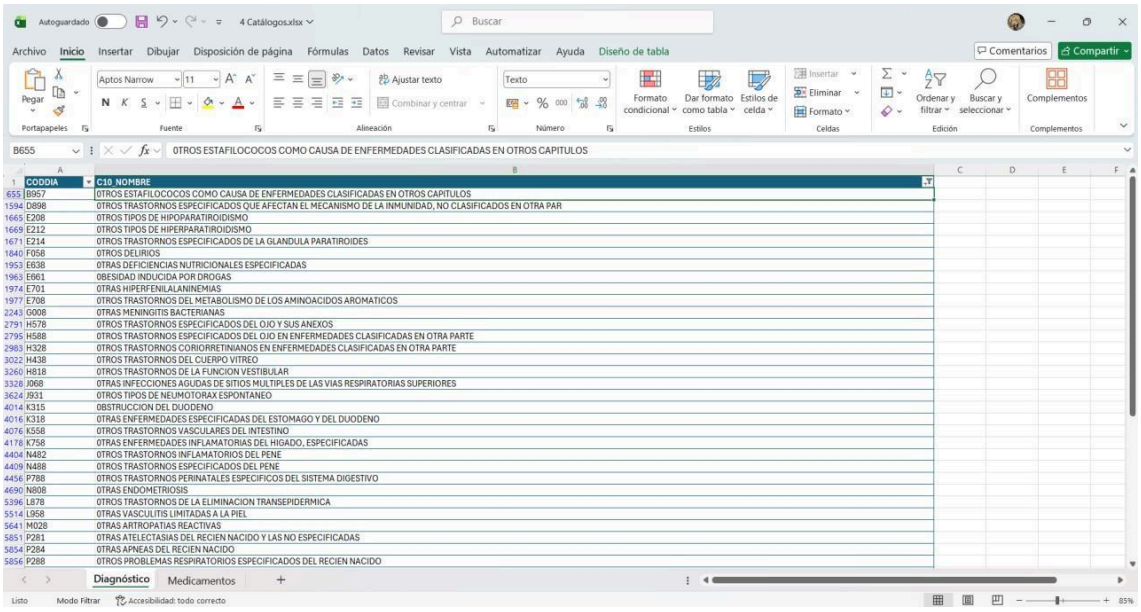


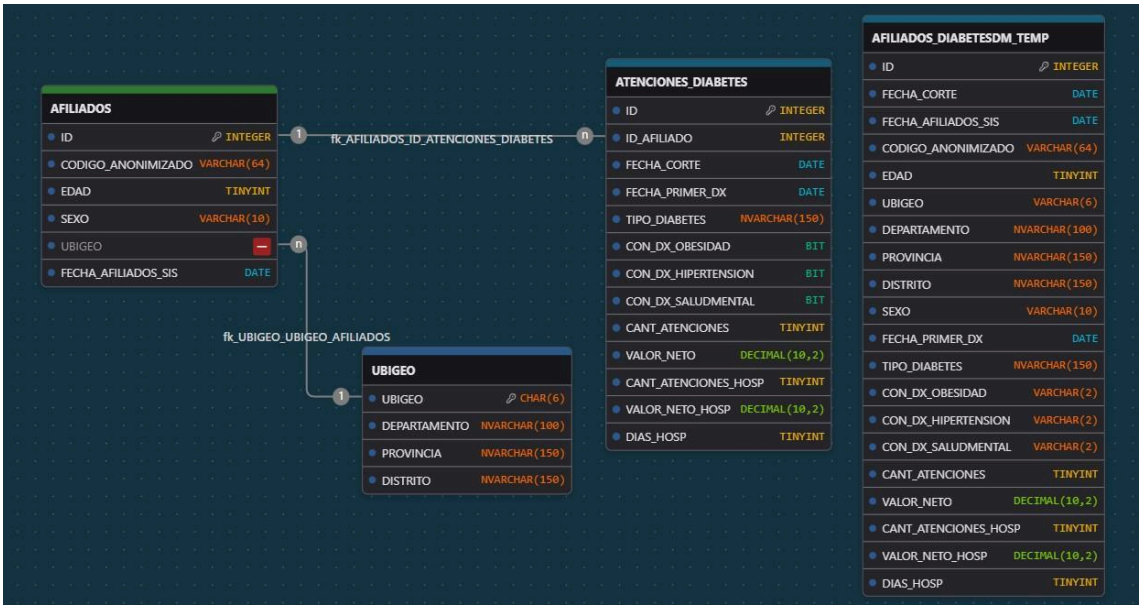
Figura 2. Identificación y corrección de inconsistencias textuales en el catálogo de diagnósticos del dataset de Recién Nacidos.



CAPÍTULO II

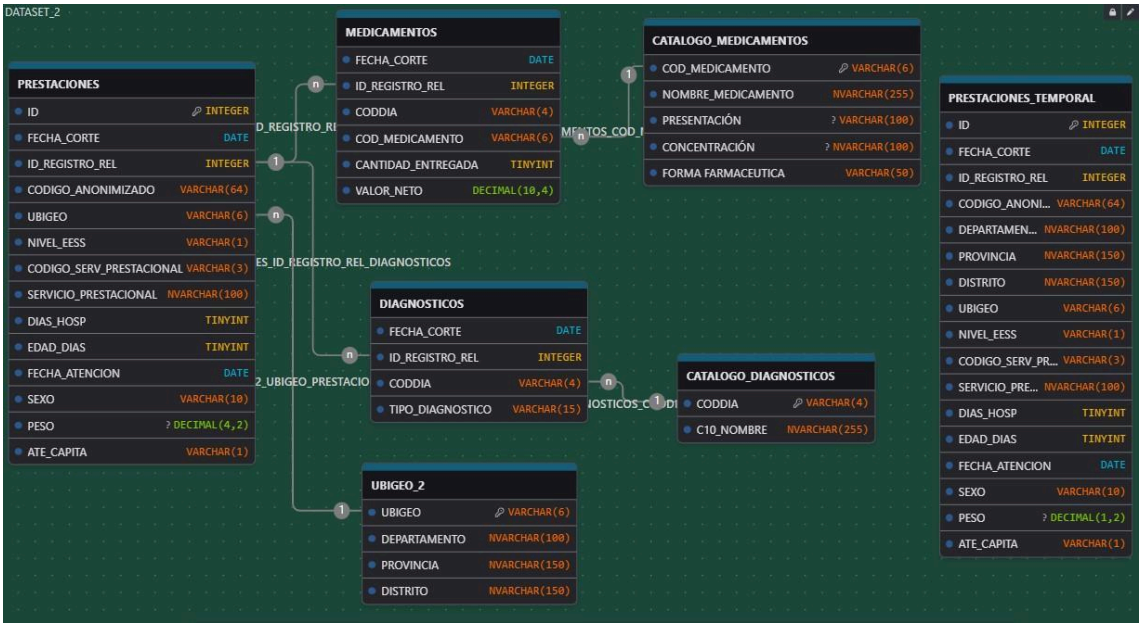
Evidencias gráficas

Figura 1. Modelo de datos del Dataset 1 – Diabetes Mellitus



Fuente: Elaboración propia.

Figura 2. Modelo de datos del Dataset 2 – Recién Nacidos



Fuente: Elaboración propia.

Figura 3. Ejecución satisfactoria de SP_INGRESAR_PRESTACION_RN.

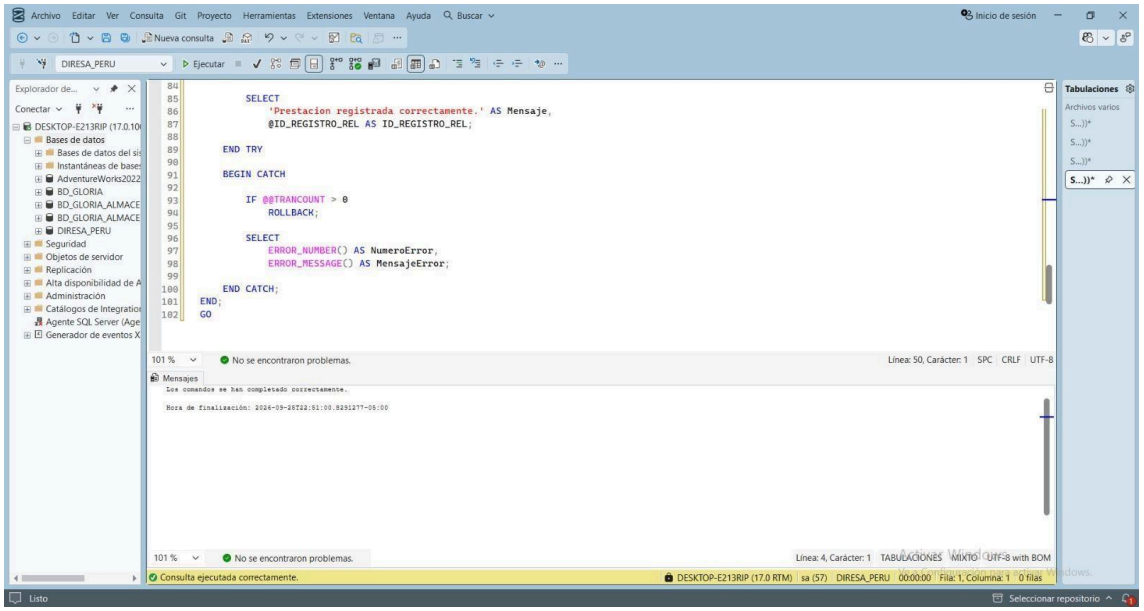


Figura 4. Ejecución del procedimiento SP_INGRESAR_AFILIADO_DIABETES

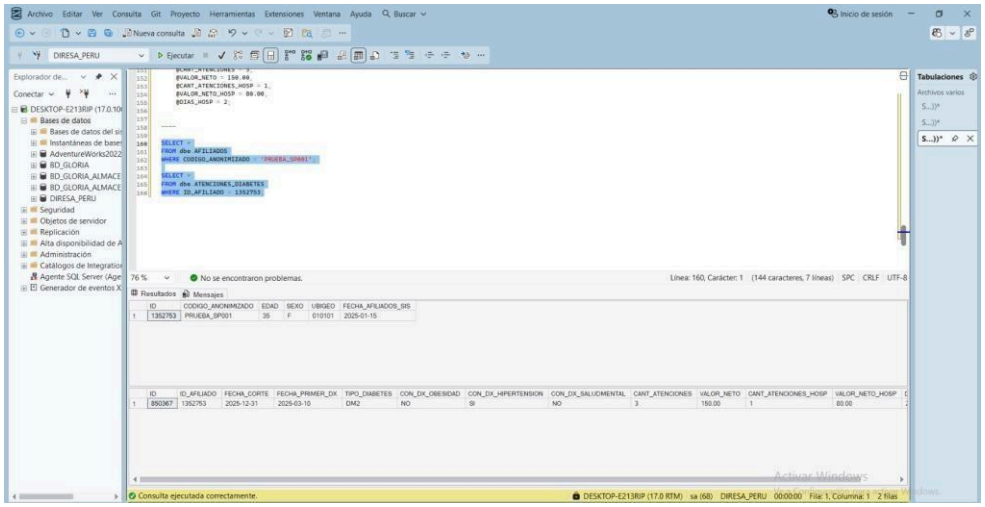


Figura 4. Ejecución del procedimiento SP_INGRESAR_AFILIADO_DIABETES

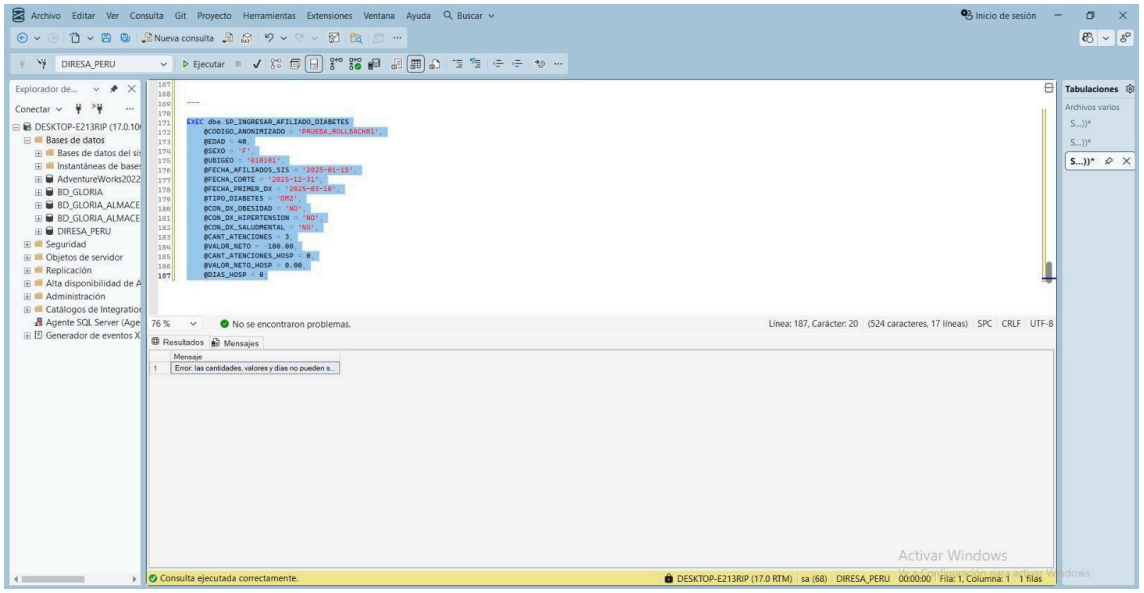


Figura 5. Prueba de ROLLBACK ante una excepción durante el procedimiento.

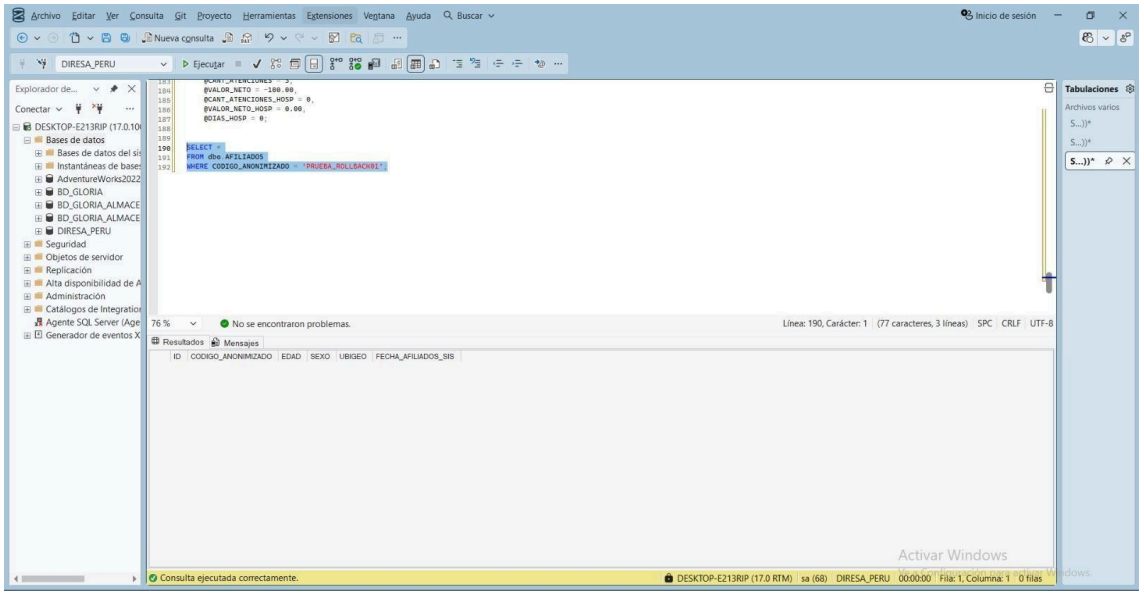


Figura 6. Prueba de SAVEPOINT y ROLLBACK parcial.

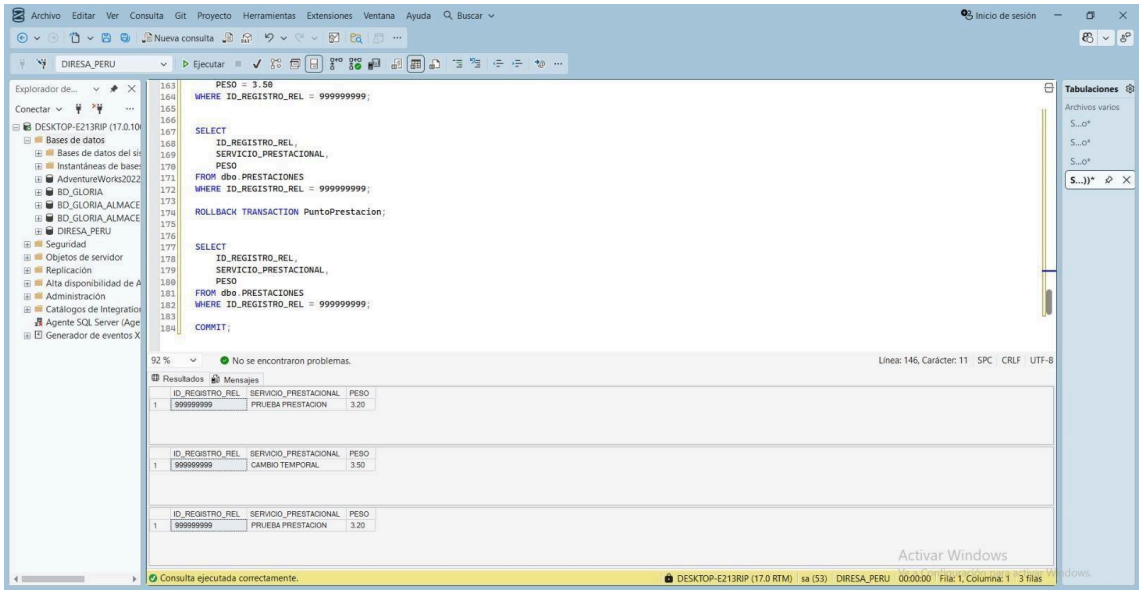


Figura 7. Registro generado automáticamente en LOG_AUDITORIA.

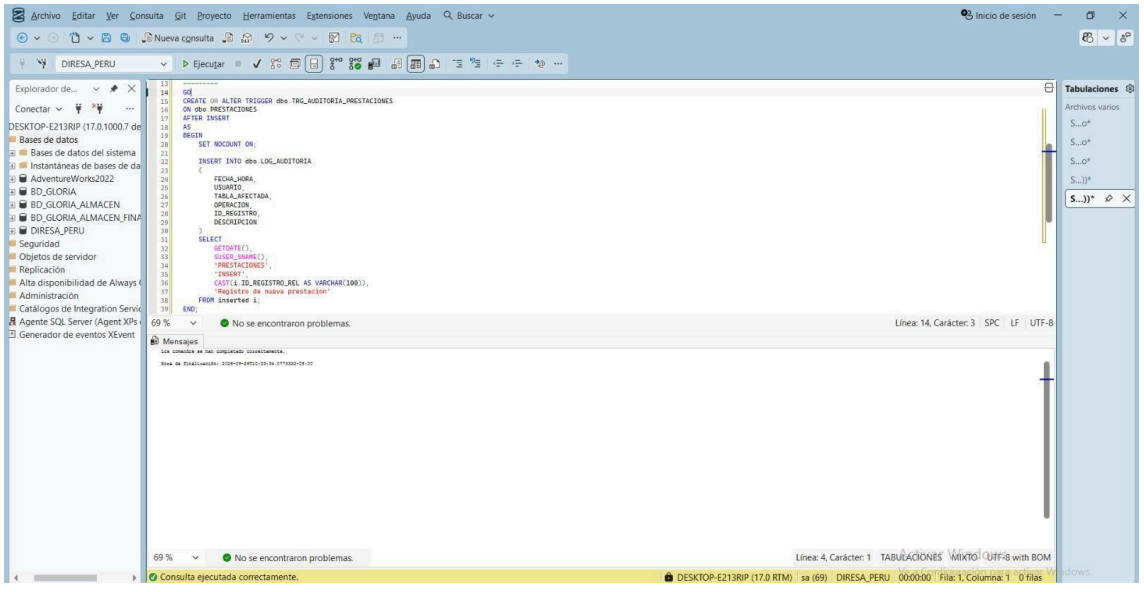


Figura 8. Rechazo de un valor negativo mediante TRG_VALIDAR_PRESTACIONES.

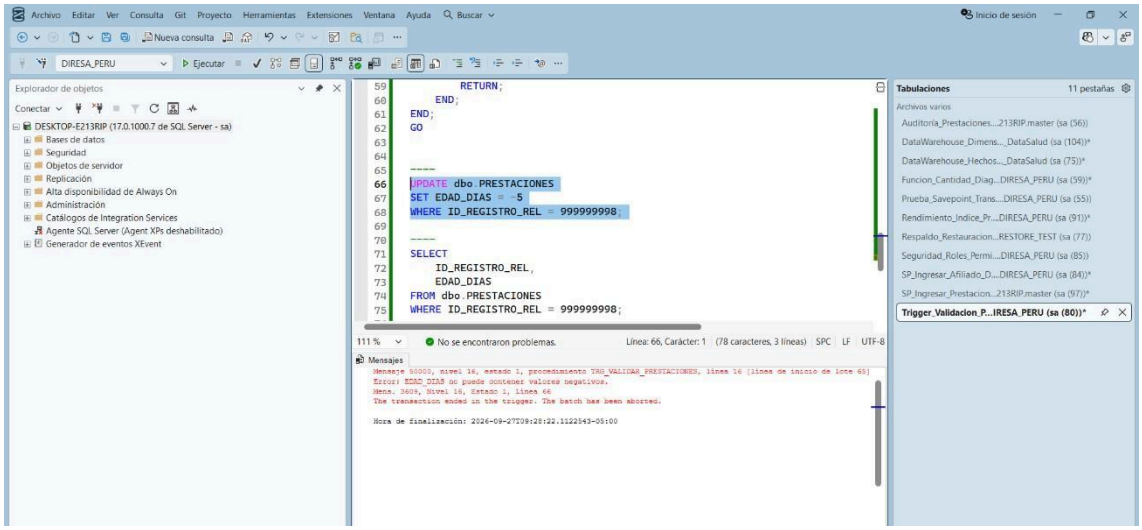


Figura 9. Ejecución de FN_CANTIDAD_DIAGNOSTICOS.

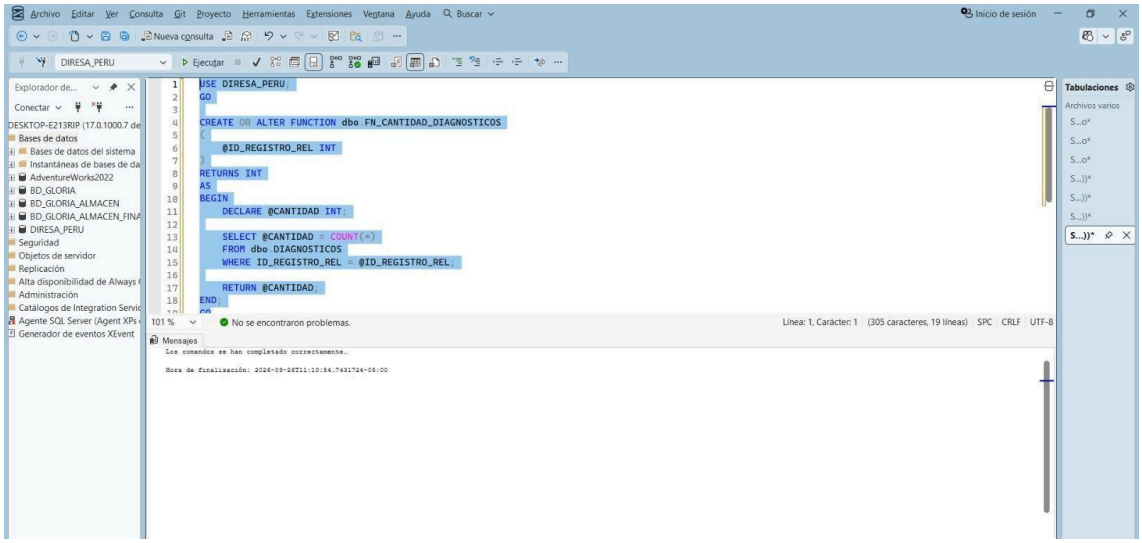
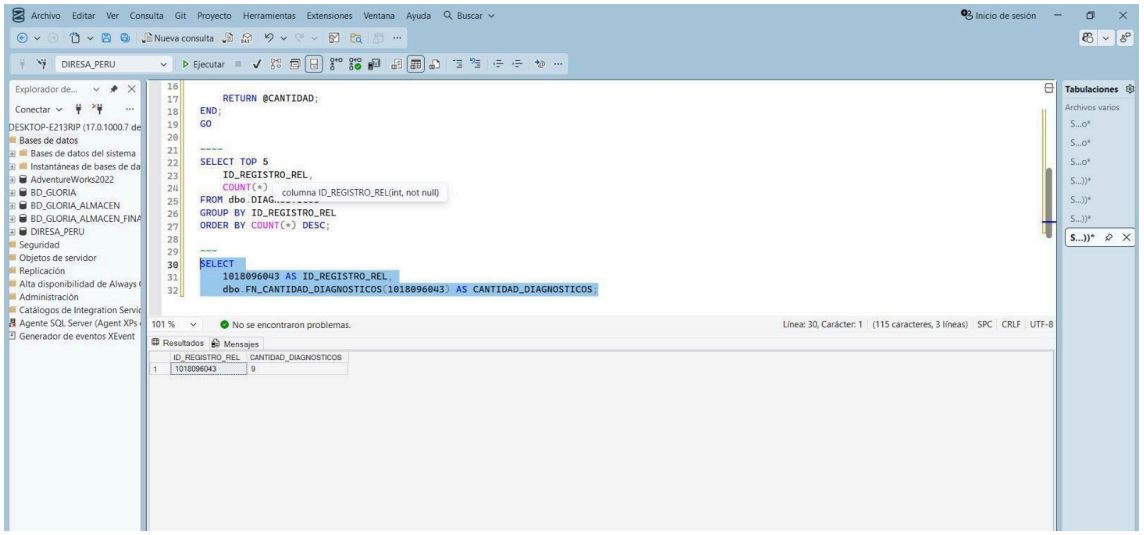


Figura 10. Prueba de funcionamiento de la función FN_CANTIDAD_DIAGNOSTICOS.



CAPÍTULO II

Códigos y consultas

SP_INGRESAR_PRESTACION_RN y pruebas transaccionales

```
USE DIRESA_PERU;
GO


CREATE OR ALTER PROCEDURE dbo.SP_INGRESAR_PRESTACION_RN
    @FECHA_CORTE DATE,
    @ID_REGISTRO_REL INT,
    @CODIGO_ANONIMIZADO VARCHAR(64),
    @UBIGEO VARCHAR(6),
    @NIVEL_EESS VARCHAR(1),
    @CODIGO_SERV_PRESTACIONAL VARCHAR(3),
    @SERVICIO_PRESTACIONAL NVARCHAR(100),
    @DIAS_HOSP INT,
    @EDAD_DIAS INT,
    @FECHA_ATENCION DATE,
    @SEXO VARCHAR(10),
    @PESO DECIMAL(4,2) = NULL,
    @ATE_CAPITA VARCHAR(1)
AS
BEGIN
```

```
SET NOCOUNT ON;

BEGIN TRY
  BEGIN TRANSACTION;


  IF EXISTS (
    SELECT 1
    FROM dbo.PRESTACIONES
    WHERE ID_REGISTRO_REL = @ID_REGISTRO_REL
  )
  BEGIN
    ROLLBACK;
    SELECT 'Error: el ID_REGISTRO_REL ya se encuentra registrado.' AS Mensaje;
    RETURN;
  END;


  IF @DIAS_HOSP < 0
    OR @EDAD_DIAS < 0
    OR @PESO < 0
  BEGIN
    ROLLBACK;
    SELECT 'Error: los dias, edad o peso no pueden ser negativos.' AS Mensaje;
    RETURN;
  END;


  SAVE TRANSACTION PuntoPrestacion;

  INSERT INTO dbo.PRESTACIONES
```

```
(
    FECHA_CORTE,
    ID_REGISTRO_REL,
    CODIGO_ANONIMIZADO,
    UBIGEO,
    NIVEL_EESS,
    CODIGO_SERV_PRESTACIONAL,
    SERVICIO_PRESTACIONAL,
    DIAS_HOSP,
    EDAD_DIAS,
    FECHA_ATENCION,
    SEXO,
    PESO,
    ATE_CAPITA
)
VALUES
(
    @FECHA_CORTE,
    @ID_REGISTRO_REL,
    @CODIGO_ANONIMIZADO,
    @UBIGEO,
    @NIVEL_EESS,
    @CODIGO_SERV_PRESTACIONAL,
    @SERVICIO_PRESTACIONAL,
    @DIAS_HOSP,
    @EDAD_DIAS,
    @FECHA_ATENCION,
    @SEXO,
    @PESO,
    @ATE_CAPITA
);
```

```
        COMMIT;

        SELECT
            'Prestacion registrada correctamente.' AS Mensaje,
            @ID_REGISTRO_REL AS ID_REGISTRO_REL;

    END TRY

    BEGIN CATCH

        IF @@TRANCOUNT > 0
            ROLLBACK;

        SELECT
            ERROR_NUMBER() AS NumeroError,
            ERROR_MESSAGE() AS MensajeError;

    END CATCH;
END;
GO



SELECT *
FROM dbo.PRESTACIONES
WHERE ID_REGISTRO_REL = 999999999;



EXEC dbo.SP_INGRESAR_PRESTACION_RN
    @FECHA_CORTE = '2025-09-18',
```

@ID_REGISTRO_REL = 999999999,

@CODIGO_ANONIMIZADO = 'PRUEBA_RN001',

@UBIGEO = '010101',

@NIVEL_EESS = '1',

@CODIGO_SERV_PRESTACIONAL = '001',

@SERVICIO_PRESTACIONAL = 'PRUEBA PRESTACION',

@DIAS_HOSP = 2,

@EDAD_DIAS = 10,

@FECHA_ATENCION = '2025-09-10',

@SEXO = 'F',

@PESO = 3.20,

@ATE_CAPITA = 'N';

EXEC dbo.SP_INGRESAR_PRESTACION_RN

@FECHA_CORTE = '2025-09-18',

@ID_REGISTRO_REL = 999999999,

@CODIGO_ANONIMIZADO = 'PRUEBA_RN001',

@UBIGEO = '100101',

@NIVEL_EESS = '1',

@CODIGO_SERV_PRESTACIONAL = '001',

@SERVICIO_PRESTACIONAL = 'PRUEBA PRESTACION',

@DIAS_HOSP = 2,

@EDAD_DIAS = 10,

@FECHA_ATENCION = '2025-09-10',

@SEXO = 'F',

@PESO = 3.20,

@ATE_CAPITA = 'N';

```
BEGIN TRANSACTION;

SELECT
   ID_REGISTRO_REL,
   SERVICIO_PRESTACIONAL,
   PESO
FROM dbo.PRESTACIONES
WHERE ID_REGISTRO_REL = 999999999;


SAVE TRANSACTION PuntoPrestacion;


UPDATE dbo.PRESTACIONES
SET SERVICIO_PRESTACIONAL = 'CAMBIO TEMPORAL',
   PESO = 3.50
WHERE ID_REGISTRO_REL = 999999999;


SELECT
   ID_REGISTRO_REL,
   SERVICIO_PRESTACIONAL,
   PESO
FROM dbo.PRESTACIONES
WHERE ID_REGISTRO_REL = 999999999;

ROLLBACK TRANSACTION PuntoPrestacion;
```

```
SELECT
   ID_REGISTRO_REL,
   SERVICIO_PRESTACIONAL,
   PESO
FROM dbo.PRESTACIONES
WHERE ID_REGISTRO_REL = 999999999;


COMMIT;
```

SP_INGRESAR_AFILIADO_DIABETES y pruebas

```
CREATE OR ALTER PROCEDURE dbo.SP_INGRESAR_AFILIADO_DIABETES
   @CODIGO_ANONIMIZADO VARCHAR(64),
   @EDAD TINYINT,
   @SEXO VARCHAR(10),
   @UBIGEO CHAR(6),
   @FECHA_AFILIADOS_SIS DATE,
   @FECHA_CORTE DATE,
   @FECHA_PRIMER_DX DATE,
   @TIPO_DIABETES NVARCHAR(300),
   @CON_DX_OBESIDAD VARCHAR(2),
   @CON_DX_HIPERTENSION VARCHAR(2),
   @CON_DX_SALUDMENTAL VARCHAR(2),
   @CANT_ATENCIONES INT,
   @VALOR_NETO DECIMAL(18,2),
   @CANT_ATENCIONES_HOSP INT,
   @VALOR_NETO_HOSP DECIMAL(18,2),
   @DIAS_HOSP INT
AS
BEGIN
   SET NOCOUNT ON;
```

```
BEGIN TRY
  BEGIN TRANSACTION;
  IF @CODIGO_ANONIMIZADO IS NULL
    OR LTRIM(RTRIM(@CODIGO_ANONIMIZADO)) = ''
  BEGIN
    ROLLBACK;
    SELECT 'Error: el codigo anonimizado es obligatorio.' AS Mensaje;
    RETURN;
  END;


  IF NOT EXISTS (
    SELECT 1
    FROM dbo.UBIGEO
    WHERE UBIGEO = @UBIGEO
  )
  BEGIN
    ROLLBACK;
    SELECT 'Error: el UBIGEO indicado no existe.' AS Mensaje;
    RETURN;
  END;


  IF EXISTS (
    SELECT 1
    FROM dbo.AFILIADOS
    WHERE CODIGO_ANONIMIZADO = @CODIGO_ANONIMIZADO
  )
  BEGIN
```

```
    ROLLBACK;
    SELECT 'Error: el codigo anonimizado ya esta registrado.' AS Mensaje;
    RETURN;
END;



IF @CANT_ATENCIONES < 0
   OR @VALOR_NETO < 0
   OR @CANT_ATENCIONES_HOSP < 0
   OR @VALOR_NETO_HOSP < 0
   OR @DIAS_HOSP < 0
BEGIN
    ROLLBACK;
    SELECT 'Error: las cantidades, valores y dias no pueden ser negativos.' AS Mensaje;
                                                                        RETURN;
END;

INSERT INTO dbo.AFILIADOS
(
    CODIGO_ANONIMIZADO,
    EDAD,
    SEXO,
    UBIGEO,
    FECHA_AFILIADOS_SIS
)
VALUES
(
    @CODIGO_ANONIMIZADO,
    @EDAD,
    @SEXO,
    @UBIGEO,
```

```
    @FECHA_AFILIADOS_SIS
);

DECLARE @ID_AFILIADO INT;
SET @ID_AFILIADO = SCOPE_IDENTITY();

INSERT INTO dbo.ATENCIONES_DIABETES
(
    ID_AFILIADO,
    FECHA_CORTE,
    FECHA_PRIMER_DX,
    TIPO_DIABETES,
    CON_DX_OBESIDAD,
    CON_DX_HIPERTENSION,
    CON_DX_SALUDMENTAL,
    CANT_ATENCIONES,
    VALOR_NETO,
    CANT_ATENCIONES_HOSP,
    VALOR_NETO_HOSP,
    DIAS_HOSP
)
VALUES
(
    @ID_AFILIADO,
    @FECHA_CORTE,
    @FECHA_PRIMER_DX,
    @TIPO_DIABETES,
    @CON_DX_OBESIDAD,
    @CON_DX_HIPERTENSION,
    @CON_DX_SALUDMENTAL,
    @CANT_ATENCIONES,
```

```
            @VALOR_NETO,
            @CANT_ATENCIONES_HOSP,
            @VALOR_NETO_HOSP,
            @DIAS_HOSP
        );

        COMMIT;

        SELECT
            'Registro ingresado correctamente.' AS Mensaje,
            @ID_AFILIADO AS ID_AFILIADO;

    END TRY

    BEGIN CATCH

        IF @@TRANCOUNT > 0
            ROLLBACK;

        SELECT
            ERROR_NUMBER() AS NumeroError,
            ERROR_MESSAGE() AS MensajeError;

    END CATCH;
END;
GO



----Prueba de ejecución con éxito

EXEC dbo.SP_INGRESAR_AFILIADO_DIABETES
```

```
    @CODIGO_ANONIMIZADO = 'PRUEBA_SP001',
    @EDAD = 35,
    @SEXO = 'F',
    @UBIGEO = '010101',
    @FECHA_AFILIADOS_SIS = '2025-01-15', @FECHA_CORTE = '2025-12-31',
    @FECHA_PRIMER_DX = '2025-03-10',
    @TIPO_DIABETES = 'DM2',
    @CON_DX_OBESIDAD = 'NO',
    @CON_DX_HIPERTENSION = 'SI',
    @CON_DX_SALUDMENTAL = 'NO',
    @CANT_ATENCIONES = 3,
    @VALOR_NETO = 150.00,
    @CANT_ATENCIONES_HOSP = 1,
    @VALOR_NETO_HOSP = 80.00,
    @DIAS_HOSP = 2;


---- Verificamos el registro insertado

SELECT *
FROM dbo.AFILIADOS
WHERE CODIGO_ANONIMIZADO = 'PRUEBA_SP001';

SELECT *
FROM dbo.ATENCIONES_DIABETES
WHERE ID_AFILIADO = 1352753;


--- Validación y Rollback

EXEC dbo.SP_INGRESAR_AFILIADO_DIABETES
```

```
    @CODIGO_ANONIMIZADO = 'PRUEBA_ROLLBACK01', @EDAD = 40,
    @SEXO = 'F',
    @UBIGEO = '010101',
    @FECHA_AFILIADOS_SIS = '2025-01-15',
    @FECHA_CORTE = '2025-12-31',
    @FECHA_PRIMER_DX = '2025-03-10',
    @TIPO_DIABETES = 'DM2',
    @CON_DX_OBESIDAD = 'NO',
    @CON_DX_HIPERTENSION = 'NO',
    @CON_DX_SALUDMENTAL = 'NO',
    @CANT_ATENCIONES = 3,
    @VALOR_NETO = -100.00,
    @CANT_ATENCIONES_HOSP = 0,
    @VALOR_NETO_HOSP = 0.00,
    @DIAS_HOSP = 0;



SELECT *
FROM dbo.AFILIADOS
WHERE CODIGO_ANONIMIZADO = 'PRUEBA_ROLLBACK01';



---PRUEBA DE EXCEPCIÓN CON TRY...CATCH Y ROLLBACK
DECLARE @TIPO_LARGO NVARCHAR(300);
SET @TIPO_LARGO = REPLICATE(N'X', 200);

EXEC dbo.SP_INGRESAR_AFILIADO_DIABETES
    @CODIGO_ANONIMIZADO = 'PRUEBA_ROLLBACK02', @EDAD = 40,
    @SEXO = 'F',
    @UBIGEO = '100101',
    @FECHA_AFILIADOS_SIS = '2025-01-15',
```

```
    @FECHA_CORTE = '2025-12-31',
    @FECHA_PRIMER_DX = '2025-03-10',
    @TIPO_DIABETES = @TIPO_LARGO,
    @CON_DX_OBESIDAD = 'NO',
    @CON_DX_HIPERTENSION = 'NO',
    @CON_DX_SALUDMENTAL = 'NO',
    @CANT_ATENCIONES = 3,
    @VALOR_NETO = 100.00,
    @CANT_ATENCIONES_HOSP = 0,
    @VALOR_NETO_HOSP = 0.00,
    @DIAS_HOSP = 0;



SELECT *
FROM dbo.AFILIADOS
WHERE CODIGO_ANONIMIZADO = 'PRUEBA_ROLLBACK02';
```

Creación de LOG_AUDITORIA

```
USE DIRESA_PERU;
GO
CREATE TABLE dbo.LOG_AUDITORIA
(
    ID_LOG INT IDENTITY(1,1) PRIMARY KEY,
    FECHA_HORA DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    USUARIO NVARCHAR(128) NOT NULL DEFAULT SUSER_SNAME(),
    TABLA_AFECTADA VARCHAR(100) NOT NULL,
    OPERACION VARCHAR(20) NOT NULL,
    ID_REGISTRO VARCHAR(100) NULL,
    DESCRIPCION NVARCHAR(500) NULL
);
GO
```

Trigger TRG_AUDITORIA_PRESTACIONES

```
CREATE OR ALTER TRIGGER dbo.TRG_AUDITORIA_PRESTACIONES
ON dbo.PRESTACIONES
AFTER INSERT
AS
BEGIN
   SET NOCOUNT ON;


   INSERT INTO dbo.LOG_AUDITORIA
   (
      FECHA_HORA,
      USUARIO,
      TABLA_AFECTADA,
      OPERACION,
      ID_REGISTRO,
      DESCRIPCION
   )
   SELECT
      GETDATE(),
      SUSER_SNAME(),
      'PRESTACIONES',
      'INSERT',
      CAST(i.ID_REGISTRO_REL AS VARCHAR(100)),
      'Registro de nueva prestacion'
   FROM inserted i;
END;
GO
```

Trigger TRG_VALIDAR_PRESTACIONES

```
USE DIRESA_PERU;
GO
```

```
CREATE OR ALTER TRIGGER dbo.TRG_VALIDAR_PRESTACIONES
ON dbo.PRESTACIONES
AFTER INSERT, UPDATE
AS
BEGIN
  SET NOCOUNT ON;

  IF EXISTS (
    SELECT 1
    FROM inserted
    WHERE EDAD_DIAS < 0
  )
  BEGIN
    RAISERROR(
      'Error: EDAD_DIAS no puede contener valores negativos.',
      16,
      1
    );

    ROLLBACK TRANSACTION;
    RETURN;
  END;

  IF EXISTS (
    SELECT 1
    FROM inserted
    WHERE DIAS_HOSP < 0
  )
  BEGIN
    RAISERROR(
      'Error: DIAS_HOSP no puede contener valores negativos.',
      16,
      1
    );

    ROLLBACK TRANSACTION;
    RETURN;
  END;

  IF EXISTS (
    SELECT 1
```

```
        FROM inserted
        WHERE PESO IS NOT NULL
          AND PESO <= 0
    )
    BEGIN
        RAISERROR(
            'Error: PESO debe ser mayor que cero cuando se encuentre registrado.',
            16,
            1
        );

        ROLLBACK TRANSACTION;
        RETURN;
    END;
END;
GO
```

Función FN_CANTIDAD_DIAGNOSTICOS

```
CREATE OR ALTER FUNCTION dbo.FN_CANTIDAD_DIAGNOSTICOS
(
    @ID_REGISTRO_REL INT
)
RETURNS INT
AS
BEGIN
    DECLARE @CANTIDAD INT;

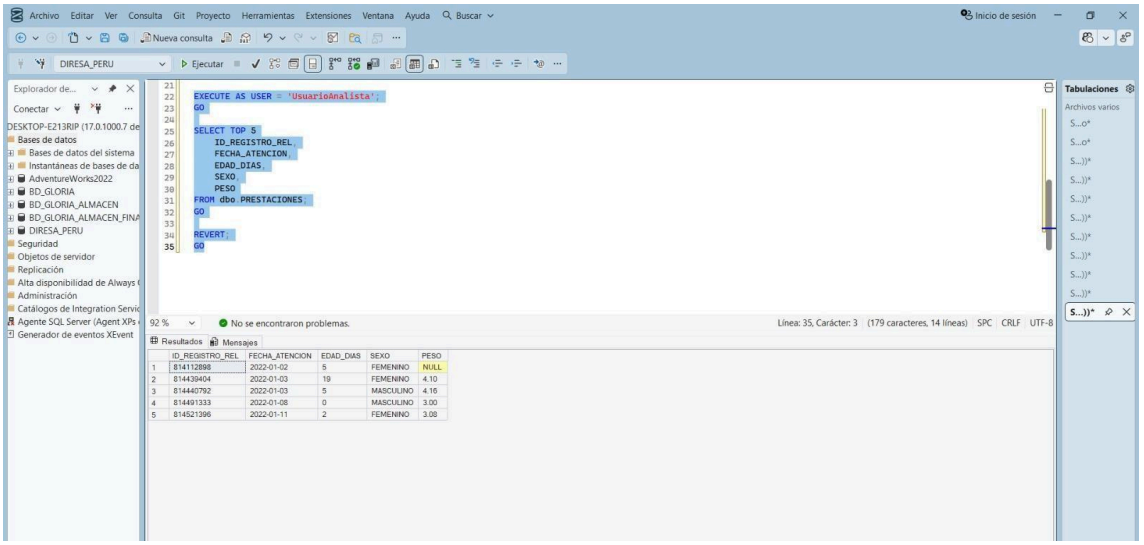
    SELECT @CANTIDAD = COUNT(*)
    FROM dbo.DIAGNOSTICOS
    WHERE ID_REGISTRO_REL = @ID_REGISTRO_REL;


    RETURN @CANTIDAD;
END;
GO
```

CAPÍTULO III

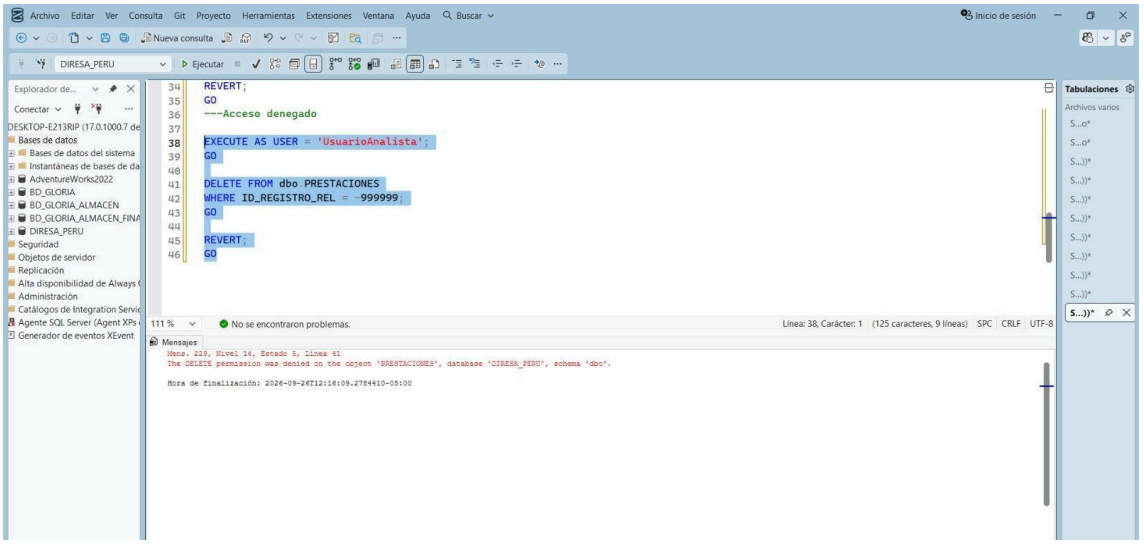
Evidencias gráficas

Figura 10. Consulta permitida para el usuario UsuarioAnalista.



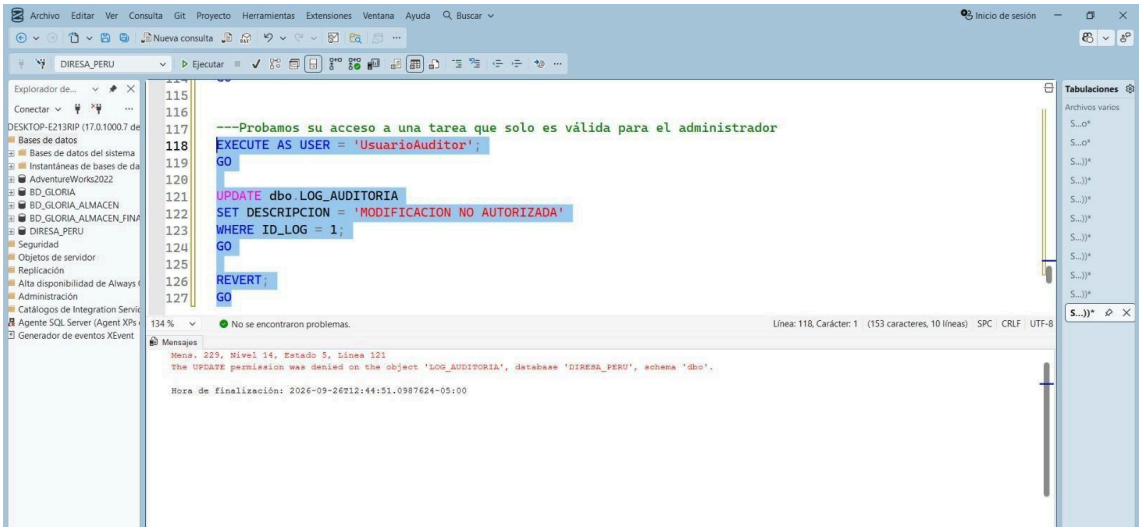
Fuente: Elaboración propia.

Figura 11. Operación DELETE denegada para el usuario UsuarioAnalista.



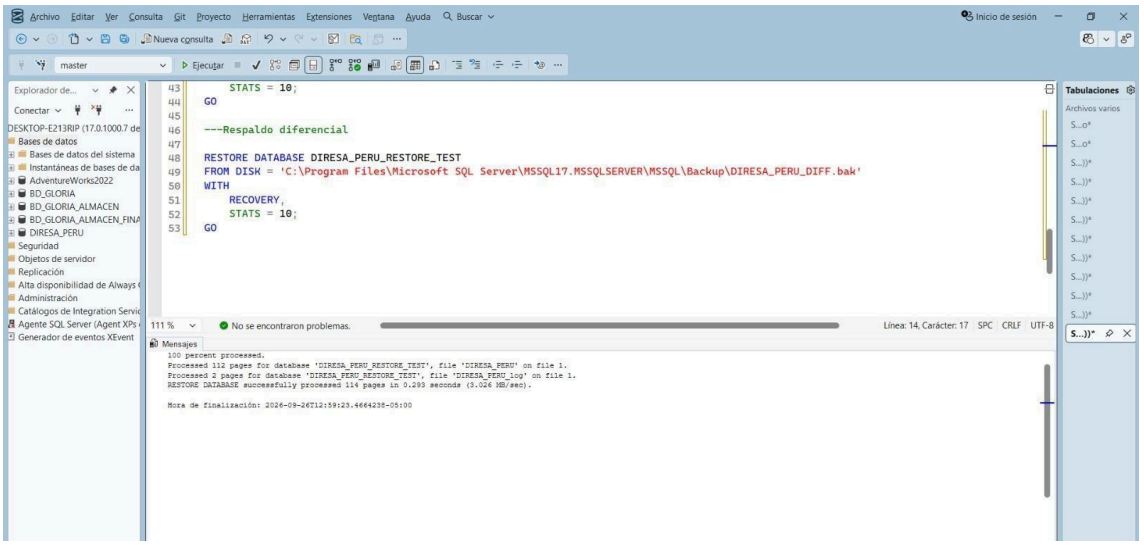
Fuente: Elaboración propia.

Figura 12. Operación UPDATE denegada para el usuario UsuarioAuditor.



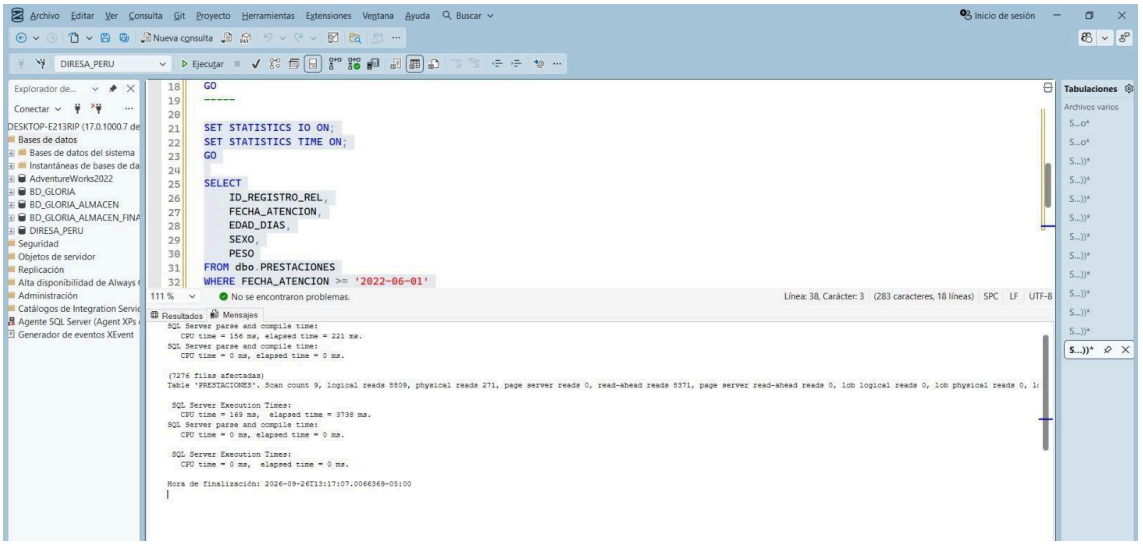
Fuente: Elaboración propia.

Figura 13. Verificación de la restauración exitosa de la base de datos DIRESA_PERU.



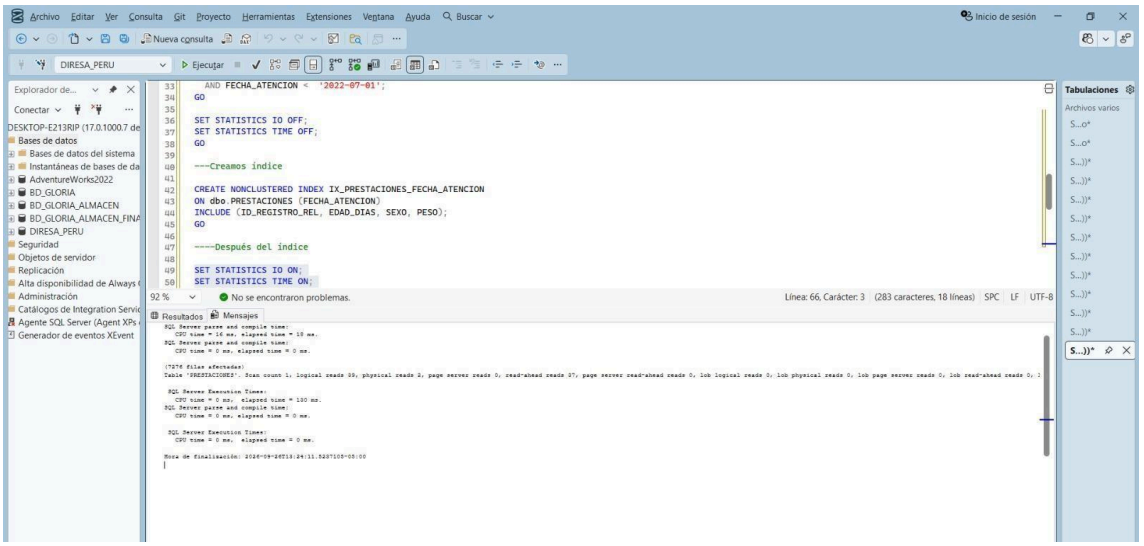
Fuente: Elaboración propia.

Figura 14. Métricas de rendimiento antes de implementar el índice.



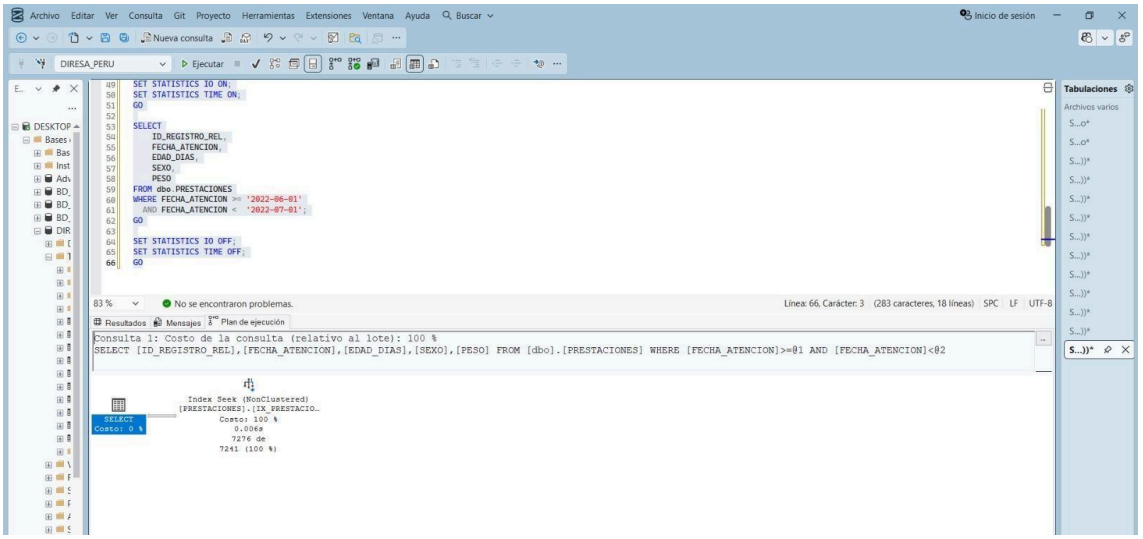
Fuente: Elaboración propia.

Figura 15. Métricas de rendimiento después de implementar el índice.



Fuente: Elaboración propia.

Figura 16. Plan de ejecución posterior a la implementación del índice no agrupado.



CAPÍTULO III

Códigos y consultas

Creación de usuarios, roles y privilegios

```
USE DIRESA_PERU;

GO


CREATE USER UsuarioAnalista WITHOUT LOGIN;

CREATE ROLE RolAnalista;

ALTER ROLE RolAnalista ADD MEMBER UsuarioAnalista;
```

```
GRANT SELECT ON dbo.AFILIADOS TO RolAnalista;
GRANT SELECT ON dbo.ATENCIONES_DIABETES TO RolAnalista;
GRANT SELECT ON dbo.PRESTACIONES TO RolAnalista;
GRANT SELECT ON dbo.DIAGNOSTICOS TO RolAnalista;
GRANT SELECT ON dbo.MEDICAMENTOS TO RolAnalista;
GO

CREATE USER UsuarioAdministrador WITHOUT LOGIN;
CREATE USER UsuarioAuditor WITHOUT LOGIN;
GO

CREATE ROLE RolAdministrador;
CREATE ROLE RolAuditor;
GO

ALTER ROLE RolAdministrador ADD MEMBER UsuarioAdministrador;
ALTER ROLE RolAuditor ADD MEMBER UsuarioAuditor;
GO

GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::dbo TO RolAdministrador;
GO

GRANT SELECT ON dbo.LOG_AUDITORIA TO RolAuditor;
GO
```

Índice IX_PRESTACIONES_FECHA_ATENCION

```
CREATE NONCLUSTERED INDEX IX_PRESTACIONES_FECHA_ATENCION
ON dbo.PRESTACIONES (FECHA_ATENCION) INCLUDE
(ID_REGISTRO_REL, EDAD_DIAS, SEXO, PESO);
```

Consulta/índice para comparación de rendimiento

```
CREATE NONCLUSTERED INDEX IX_PRESTACIONES_FECHA_ATENCION
```

ON dbo.PRESTACIONES (FECHA_ATENCION)

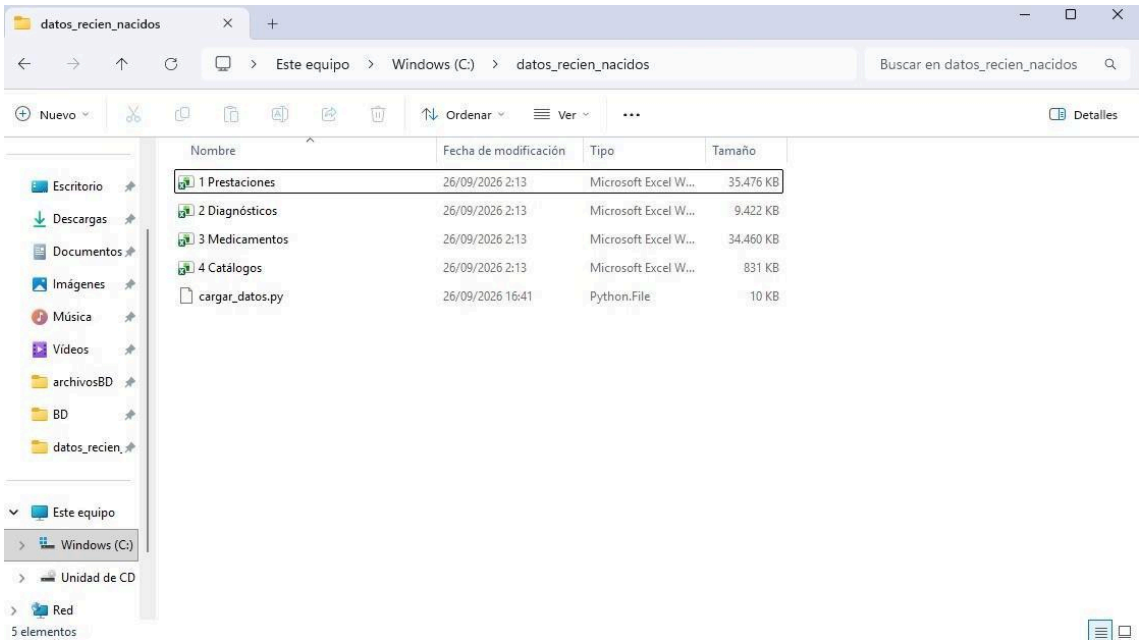
INCLUDE (ID_REGISTRO_REL, EDAD_DIAS, SEXO, PESO);

GO

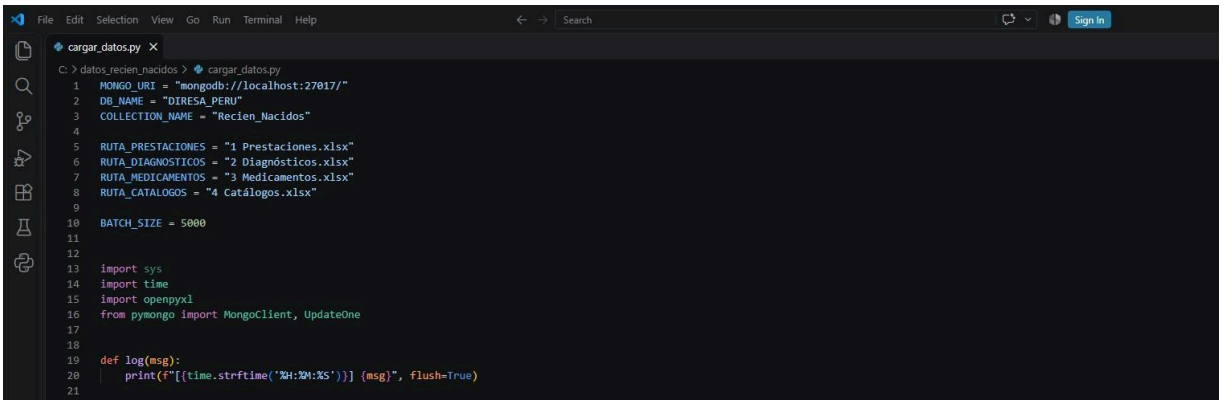
CAPÍTULO IV

Evidencias gráficas

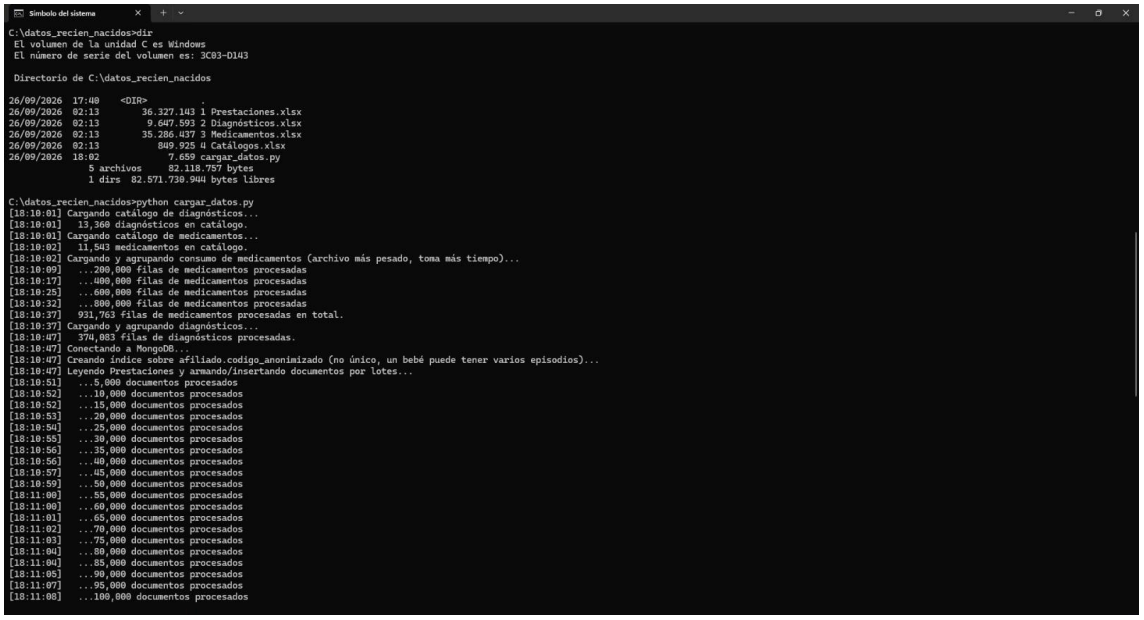
Evidencia gráfica originalmente ubicada cerca del párrafo 881.



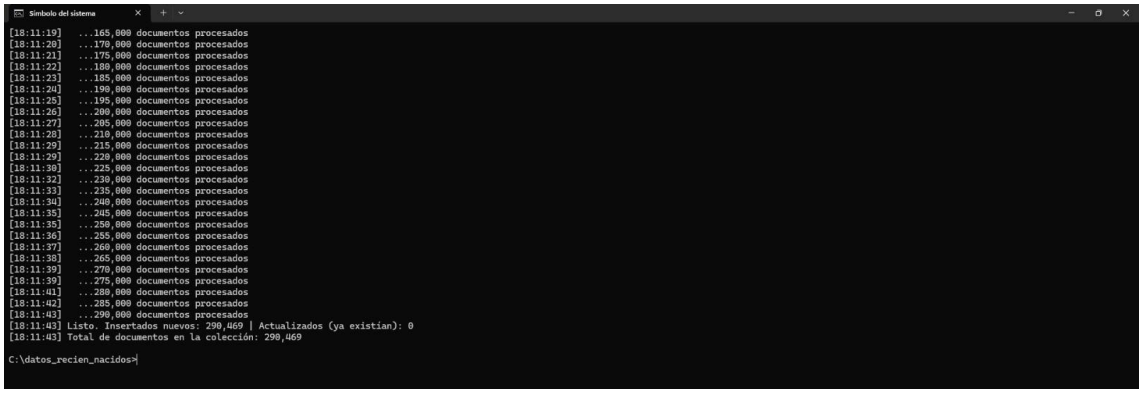
Evidencia gráfica originalmente ubicada cerca del párrafo 886.



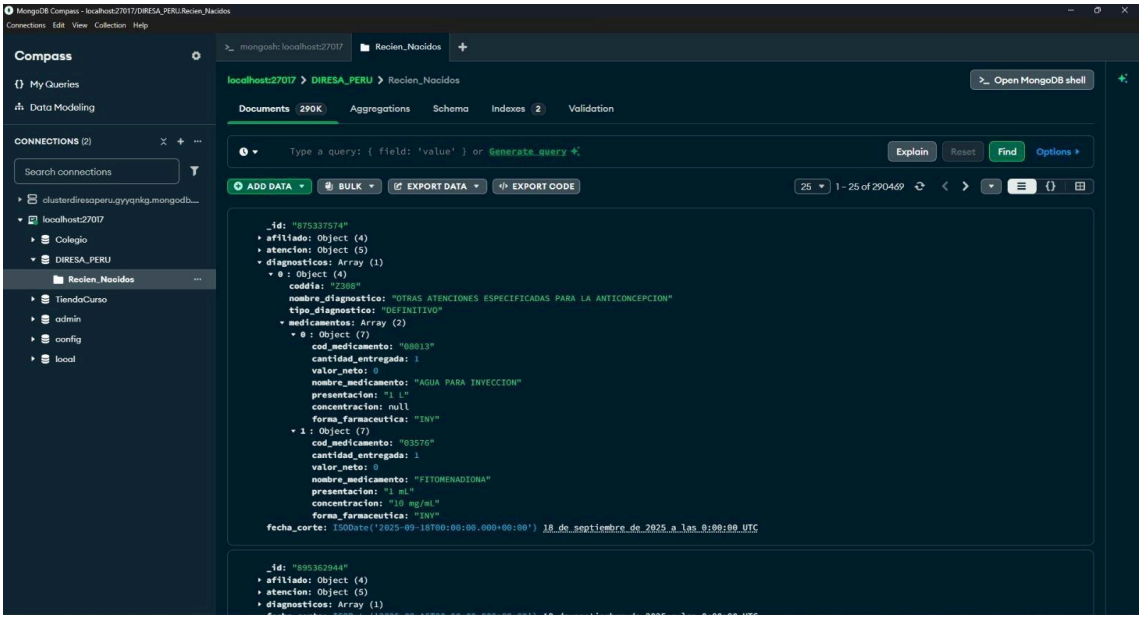
Evidencia gráfica originalmente ubicada cerca del párrafo 887.



Evidencia gráfica originalmente ubicada cerca del párrafo 888.



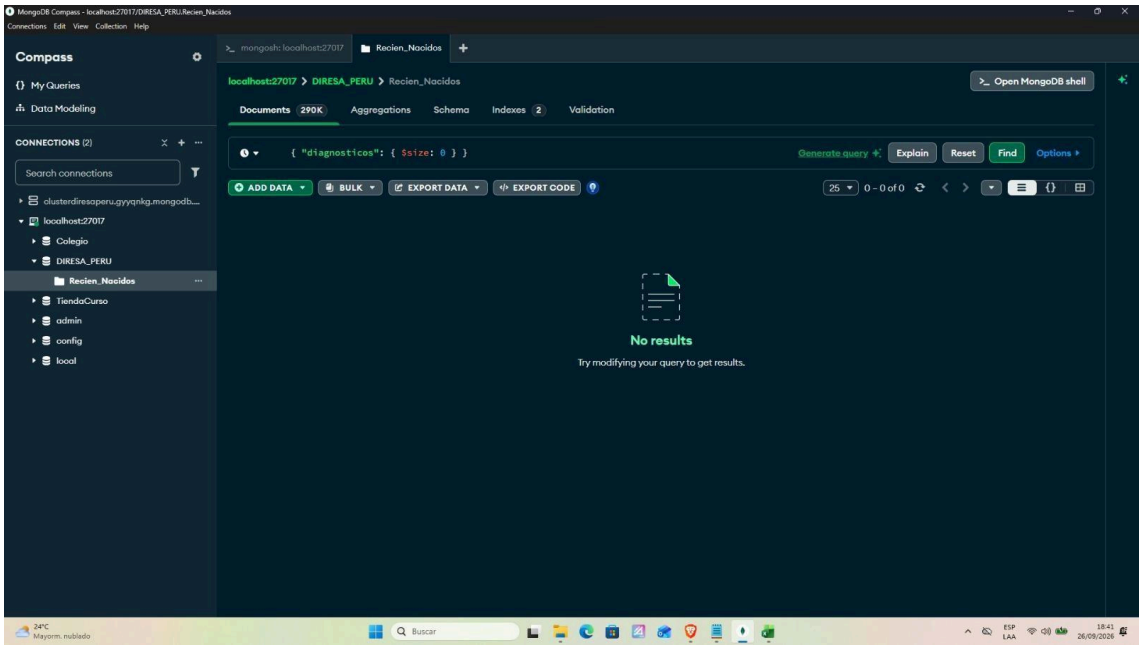
Evidencia gráfica originalmente ubicada cerca del párrafo 890.



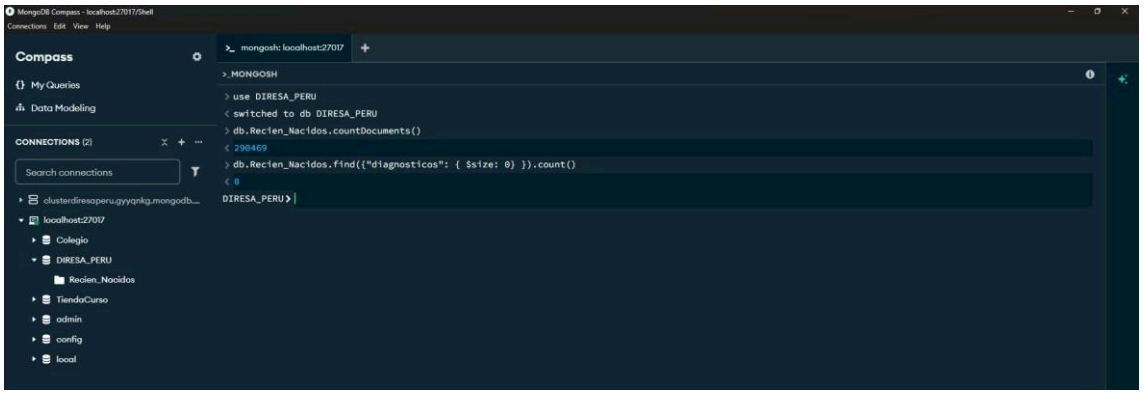
Se realizaron las siguientes verificaciones de integridad sobre la colección resultante:

Contraste manual, campo por campo, entre documentos individuales de la colección y sus filas de origen en los cuatro archivos Excel, confirmando correspondencia exacta en los datos del episodio, del diagnóstico y de los medicamentos, incluyendo el enriquecimiento correcto desde ambos catálogos.

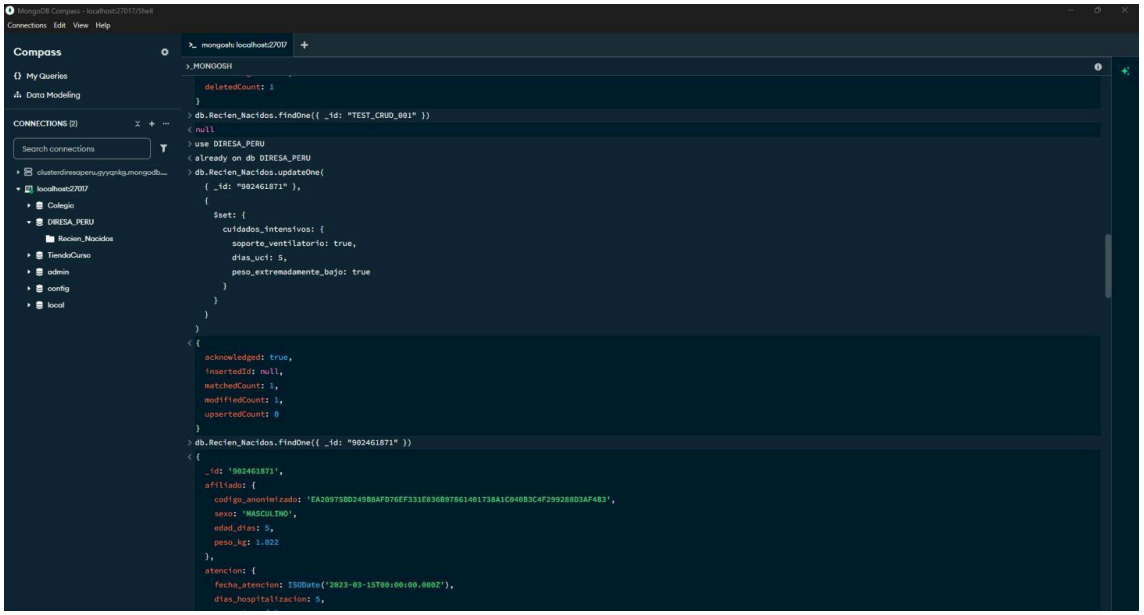
Evidencia gráfica originalmente ubicada cerca del párrafo 893.



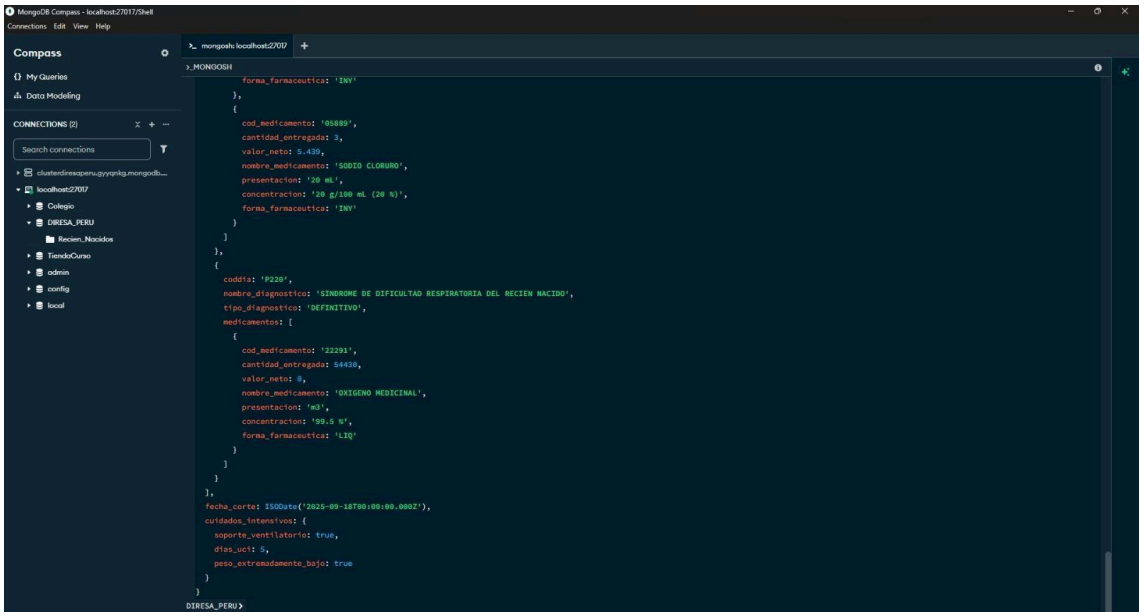
Evidencia gráfica originalmente ubicada cerca del párrafo 897.



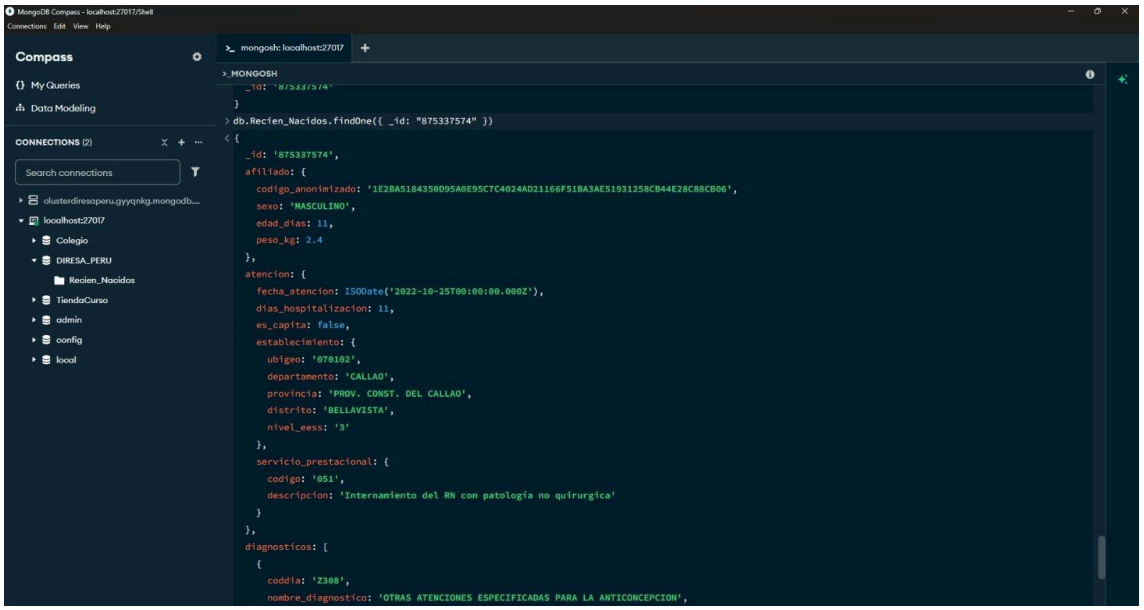
Evidencia gráfica originalmente ubicada cerca del párrafo 903.



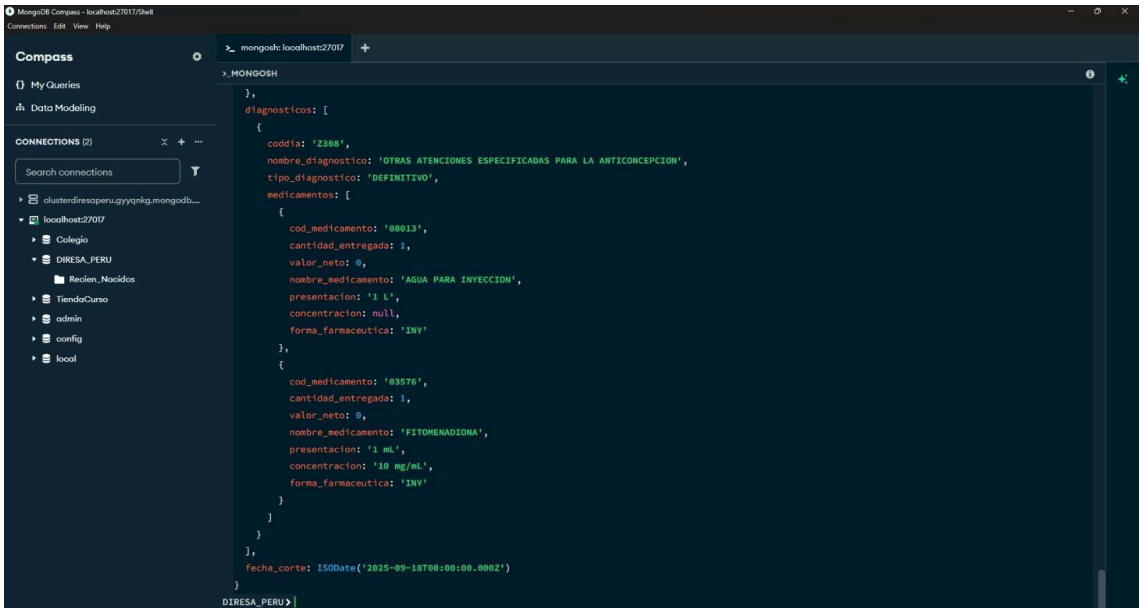
Evidencia gráfica originalmente ubicada cerca del párrafo 913.



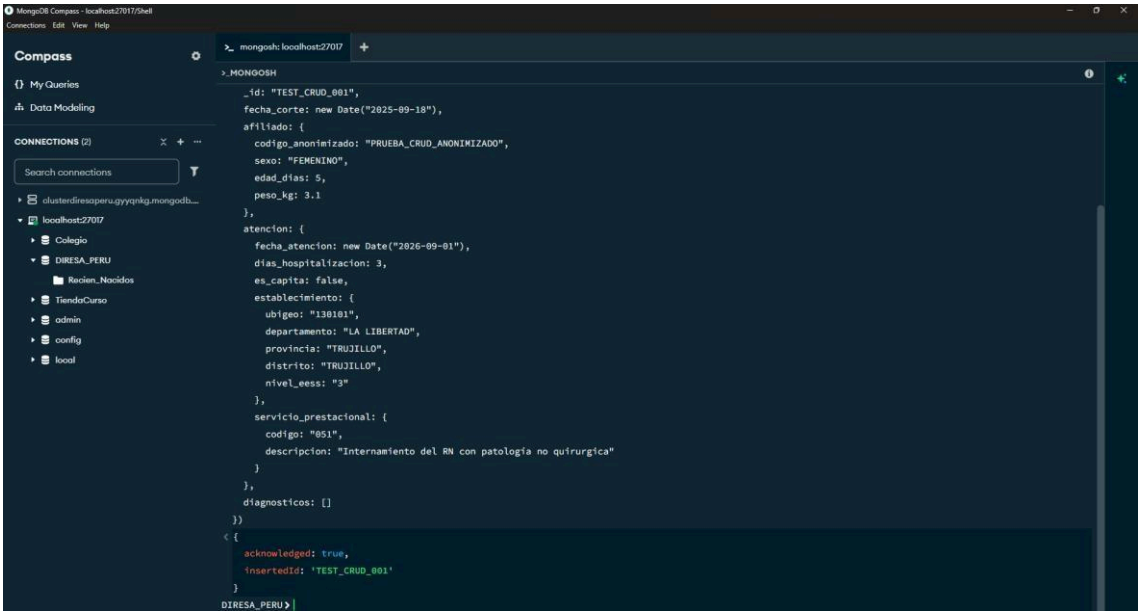
Evidencia gráfica originalmente ubicada cerca del párrafo 916.



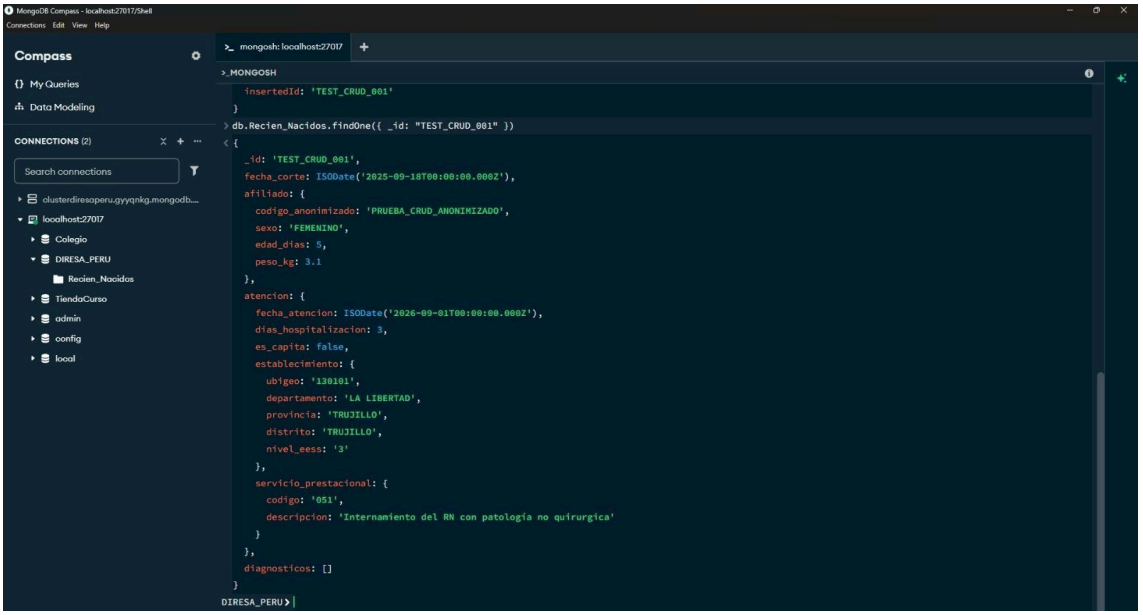
Evidencia gráfica originalmente ubicada cerca del párrafo 929.



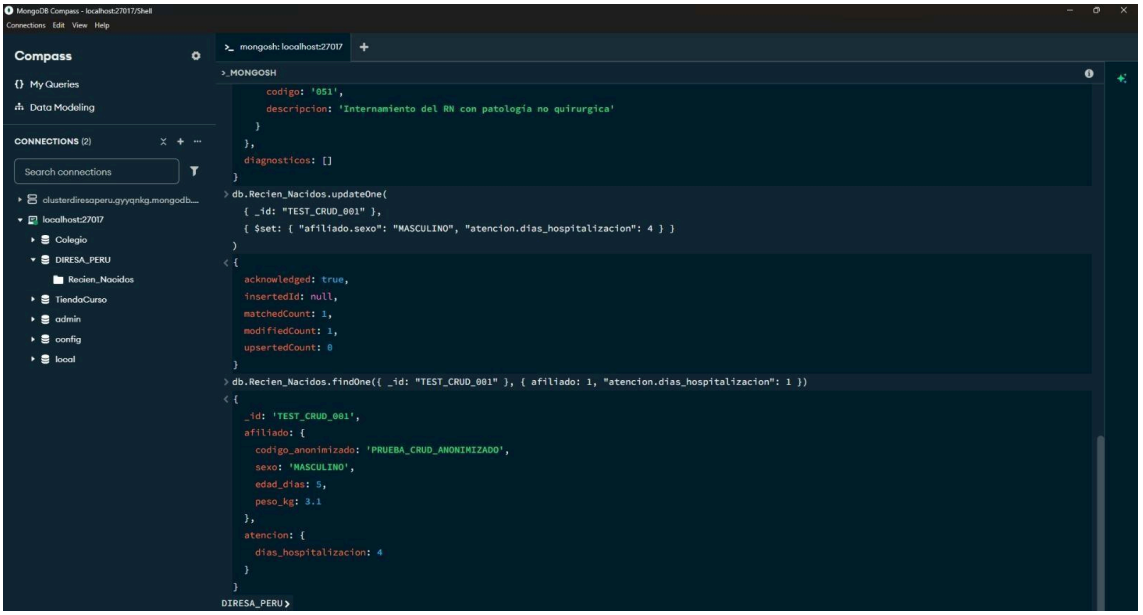
Evidencia gráfica originalmente ubicada cerca del párrafo 967.



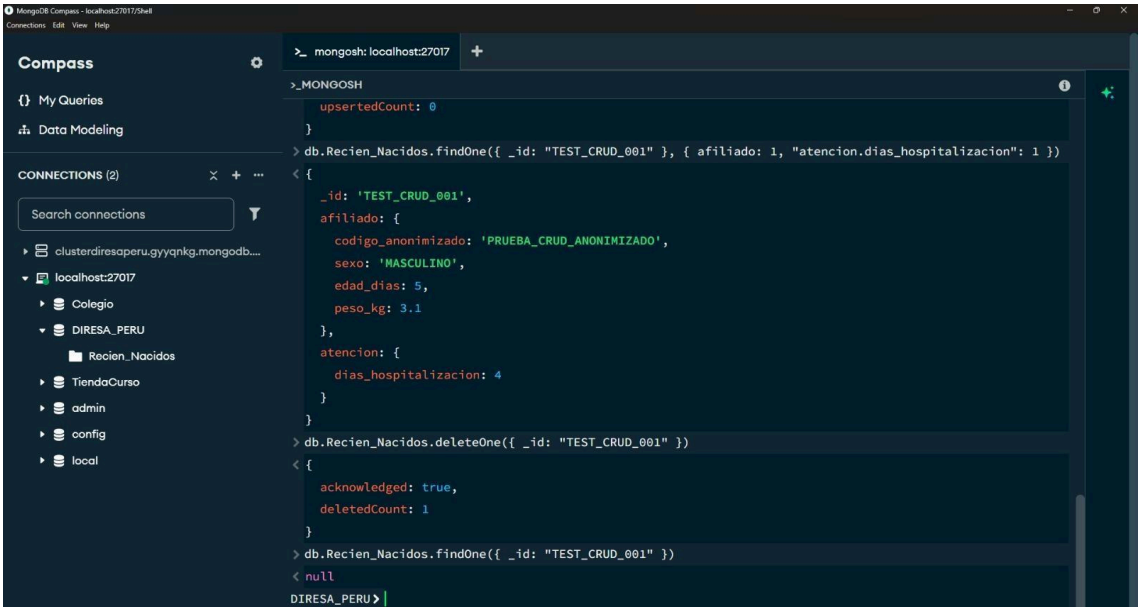
Evidencia gráfica originalmente ubicada cerca del párrafo 972.



Evidencia gráfica originalmente ubicada cerca del párrafo 983.



Evidencia gráfica originalmente ubicada cerca del párrafo 994.



CAPÍTULO IV

Códigos y consultas

CRUD MongoDB — CREATE

```
db.Recien_Nacidos.insertOne({
_id: "TEST_CRUD_001",
 fecha_corte: new Date("2025-09-18"),
 afiliado: {
  codigo_anonimizado: "PRUEBA_CRUD_ANONIMIZADO",
  sexo: "FEMENINO",
  edad_dias: 5,
  peso_kg: 3.1
 },
 atencion: {
  fecha_atencion: new Date("2026-09-01"),
  dias_hospitalizacion: 3,
  es_capita: false,
  establecimiento: {
   ubigeo: "130101",
   departamento: "LA LIBERTAD",
   provincia: "TRUJILLO",
   distrito: "TRUJILLO",
   nivel_eess: "3"
  },
  servicio_prestacional: {
   codigo: "051",
```

descripcion: "Internamiento del RN con patología no quirurgica"

}

},

diagnosticos: []

})

CRUD MongoDB — READ

db.Recien_Nacidos.findOne({ _id: "TEST_CRUD_001" }) **CRUD**

MongoDB — UPDATE y verificación

db.Recien_Nacidos.updateOne(

{ _id: "TEST_CRUD_001" },

{ $set: { "afiliado.sexo": "MASCULINO", "atencion.dias_hospitalizacion": 4 } }

)

Verificación:

db.Recien_Nacidos.findOne({ _id: "TEST_CRUD_001" }, { afiliado: 1, "atencion.dias_hospitalizacion": 1 })

CRUD MongoDB — DELETE y verificación

db.Recien_Nacidos.deleteOne({ _id: "TEST_CRUD_001" })

Verificación:

db.Recien_Nacidos.findOne({ _id: "TEST_CRUD_001" })

CAPÍTULO V

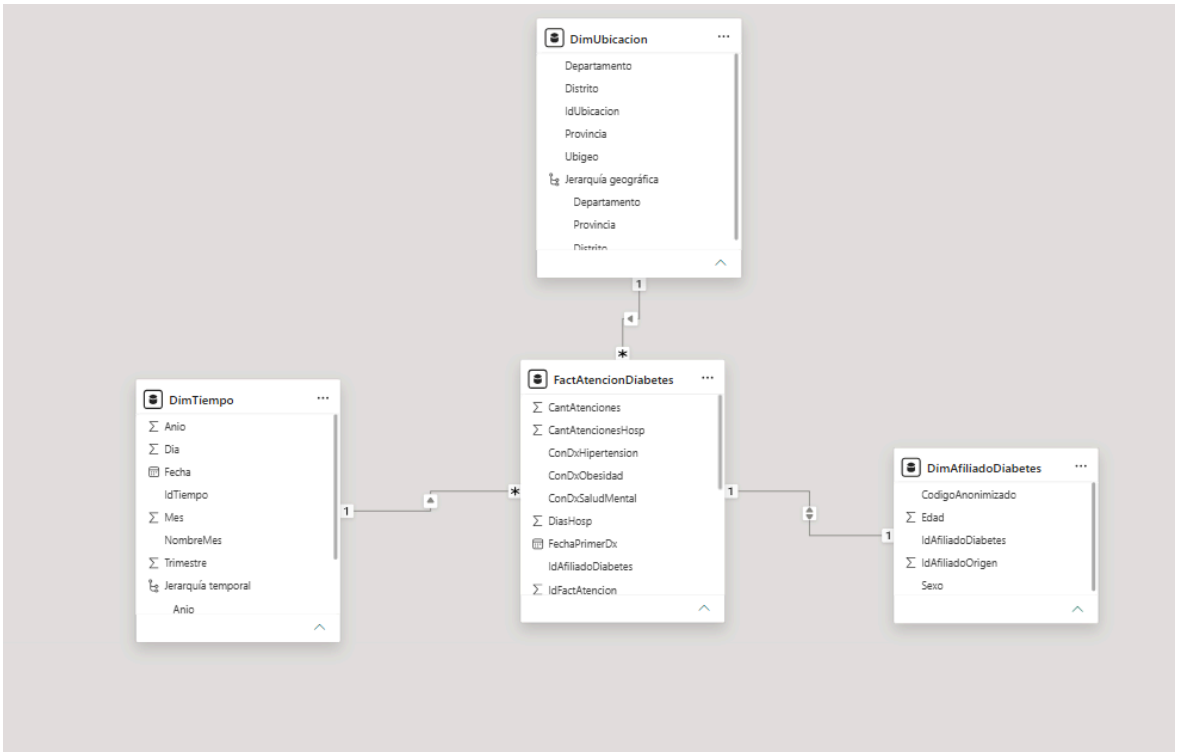
Evidencias gráficas

Figura 17. Modelo dimensional en estrella de Recién Nacidos en DW_DataSalud.



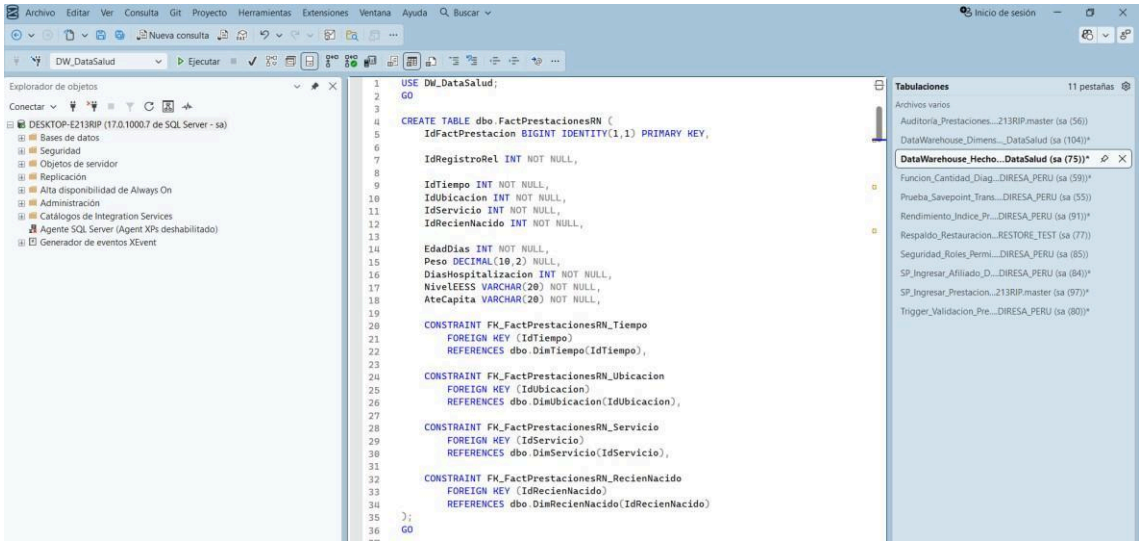
Fuente: Elaboración propia.

Figura 18. Modelo dimensional en estrella de Diabetes Mellitus en DW_DataSalud.



Fuente: Elaboración propia.

Figura 19. Implementación DDL de una tabla del modelo dimensional en SQL Server.



```
USE DW_DataSalud;
GO

CREATE TABLE dbo.FactPrestacionesRN (
    IdFactPrestacion BIGINT IDENTITY(1,1) PRIMARY KEY,

    IdRegistroRel INT NOT NULL,

    IdTiempo INT NOT NULL,
    IdUbicacion INT NOT NULL,
    IdServicio INT NOT NULL,
    IdRecienNacido INT NOT NULL,

    EdadDias INT NOT NULL,
    Peso DECIMAL(10,2) NULL,
    DiasHospitalizacion INT NOT NULL,
    NivelEESS VARCHAR(20) NOT NULL,
    AteCapita VARCHAR(20) NOT NULL,

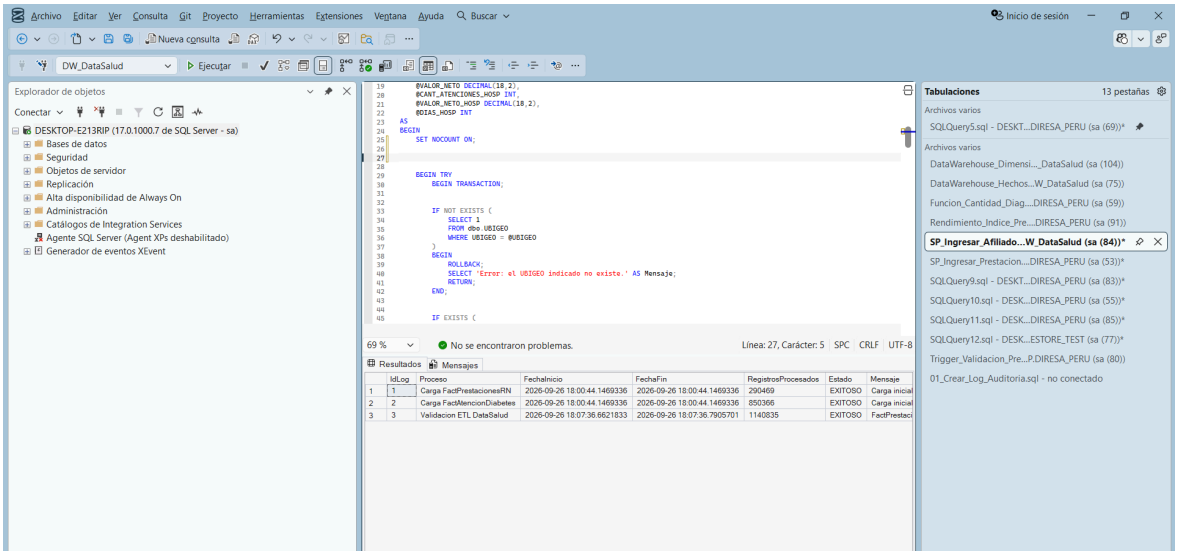
    CONSTRAINT FK_FactPrestacionesRN_Tiempo
        FOREIGN KEY (IdTiempo)
        REFERENCES dbo.DimTiempo(IdTiempo),

    CONSTRAINT FK_FactPrestacionesRN_Ubicacion
        FOREIGN KEY (IdUbicacion)
        REFERENCES dbo.DimUbicacion(IdUbicacion),

    CONSTRAINT FK_FactPrestacionesRN_Servicio
        FOREIGN KEY (IdServicio)
        REFERENCES dbo.DimServicio(IdServicio),

    CONSTRAINT FK_FactPrestacionesRN_RecienNacido
        FOREIGN KEY (IdRecienNacido)
        REFERENCES dbo.DimRecienNacido(IdRecienNacido)
);
GO
```

Figura 20: Registro de control y validación del proceso ETL en SQL Server.



```
    @VALOR_NETO DECIMAL(18,2),
    @CANT_ATENCIONES_HOSP INT,
    @VALOR_NETO_HOSP DECIMAL(18,2),
    @DIAS_HOSP INT
AS
BEGIN
    SET NOCOUNT ON;


    BEGIN TRY
        BEGIN TRANSACTION;


        IF NOT EXISTS (
            SELECT 1
            FROM dbo.UBIGEO
            WHERE UBIGEO = @UBIGEO
        )
        BEGIN
            ROLLBACK;
            SELECT 'Error: el UBIGEO indicado no existe.' AS Mensaje;
            RETURN;
        END;


        IF EXISTS (
```

	IdLog	Proceso	FechaInicio	FechaFin	RegistrosProcesados	Estado	Mensaje
1	1	Carga FactPrestacionesRN	2026-09-26 18:00:44.1469336	2026-09-26 18:00:44.1469336	290469	EXITOSO	Carga inicial
2	2	Carga FactAtencionDiabetes	2026-09-26 18:00:44.1469336	2026-09-26 18:00:44.1469336	850366	EXITOSO	Carga inicial
3	3	Validacion ETL DataSalud	2026-09-26 18:07:36.6621833	2026-09-26 18:07:36.7905701	1140835	EXITOSO	FactPrestaci

CAPÍTULO VI

Evidencias gráficas

Figura 21. Modelo dimensional integrado de DW_DataSalud implementado en Power BI.

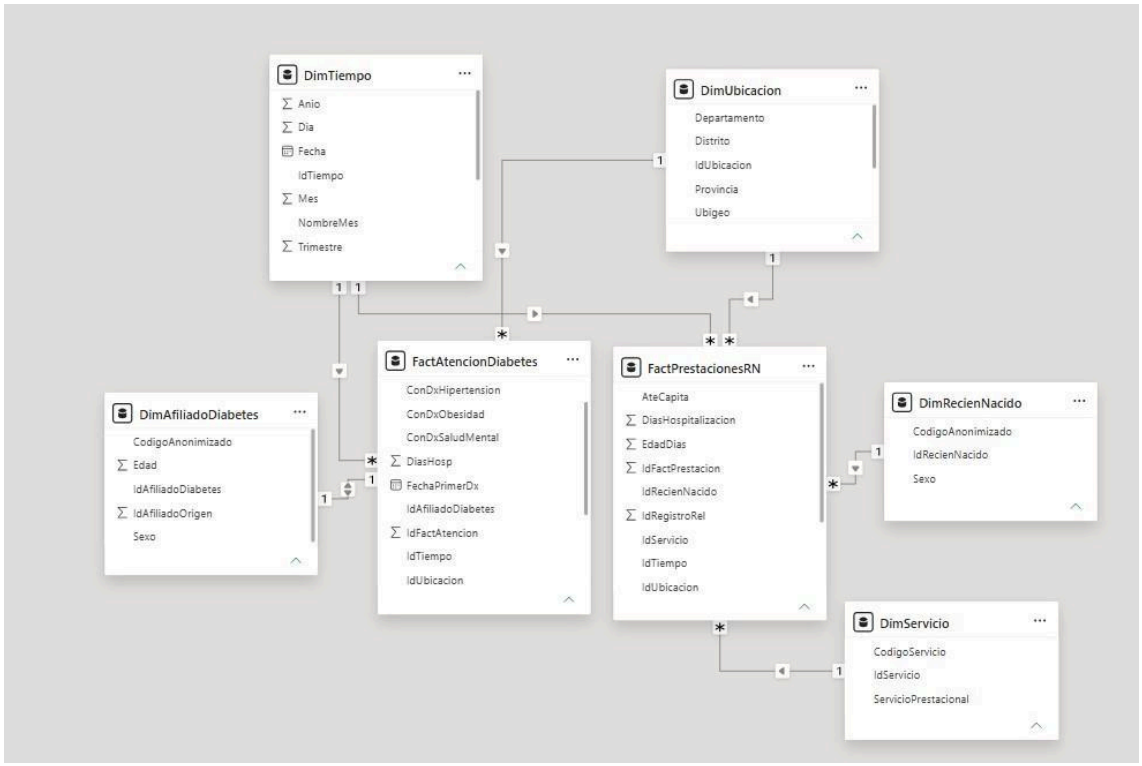


Figura 22. Indicadores clave de desempeño de Recién Nacidos implementados en Power BI.

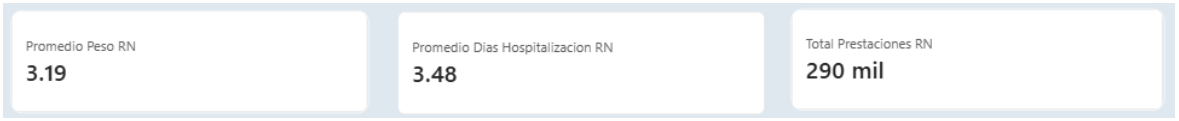


Figura 23. Indicadores clave de desempeño de Diabetes Mellitus implementados en Power BI.

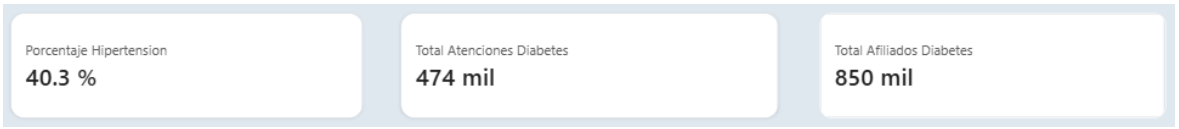


Figura 24. Indicadores clave de desempeño de Recién Nacidos implementados en Power BI.



Figura 25. Indicadores clave de desempeño de Diabetes Mellitus implementados en Power BI.

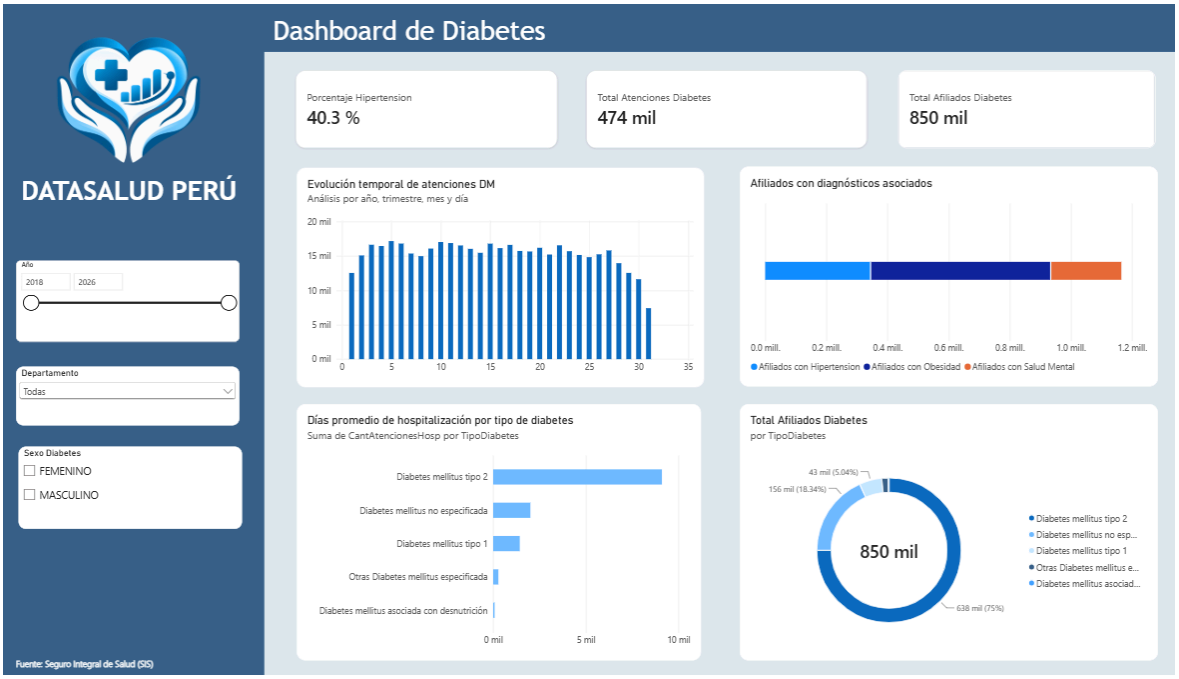


Figura 26. Operación OLAP mediante drill-down aplicada al análisis temporal.

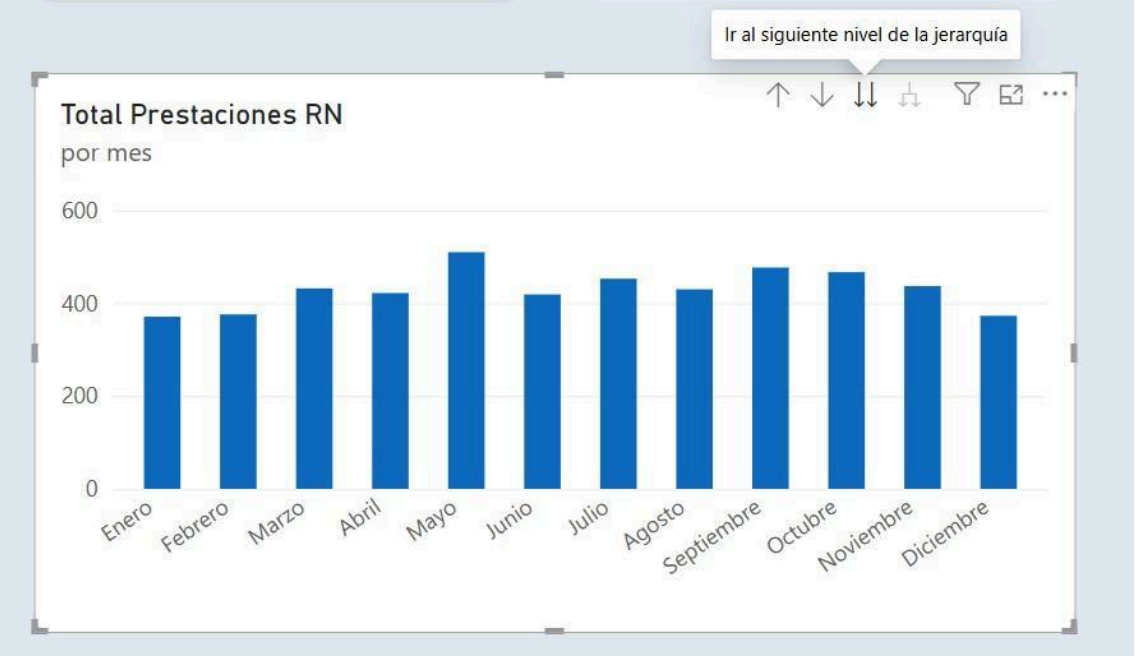


Figura 27. Cantidad de prestaciones de recién nacidos por provincia.

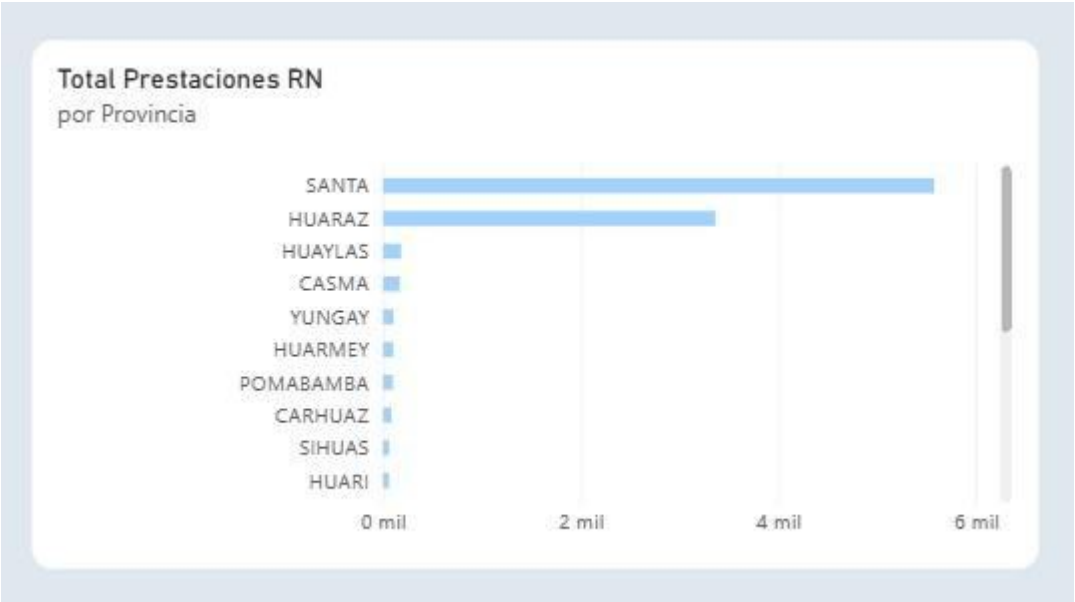
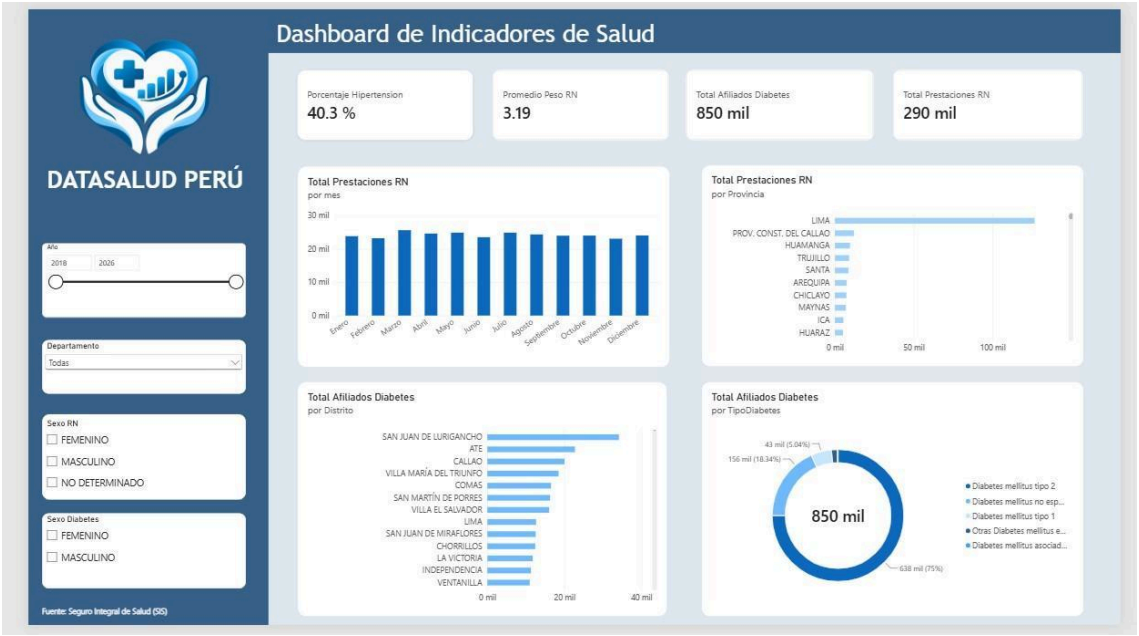


Figura 28. Distribución de los registros de diabetes según diagnóstico de hipertensión.



Figura 29. Distribución de los registros de diabetes según diagnóstico de hipertensión.



CAPÍTULO VI

Códigos y consultas

Medida DAX — Total Prestaciones RN

Medida DAX:

Total Prestaciones RN = COUNTROWS(FactPrestacionesRN)

Medida DAX — Total Afiliados Diabetes

Medida DAX:

Total Afiliados Diabetes = DISTINCTCOUNT(FactAtencionDiabetes[IdAfiliadoDiabetes])

Medida DAX — Promedio Peso RN

Medida DAX:

Promedio Peso RN = AVERAGE(FactPrestacionesRN[Peso])

Medida DAX — Porcentaje Hipertensión

Medida DAX:

Porcentaje Hipertension = DIVIDE(CALCULATE(COUNTROWS(FactAtencionDiabetes), FactAtencionDiabetes[ConDxHipertension] = "SI"), COUNTROWS(FactAtencionDiabetes), 0)

CAPÍTULO VII

Evidencias gráficas

Figura 30. Configuración del entorno PySpark en Google Colab.

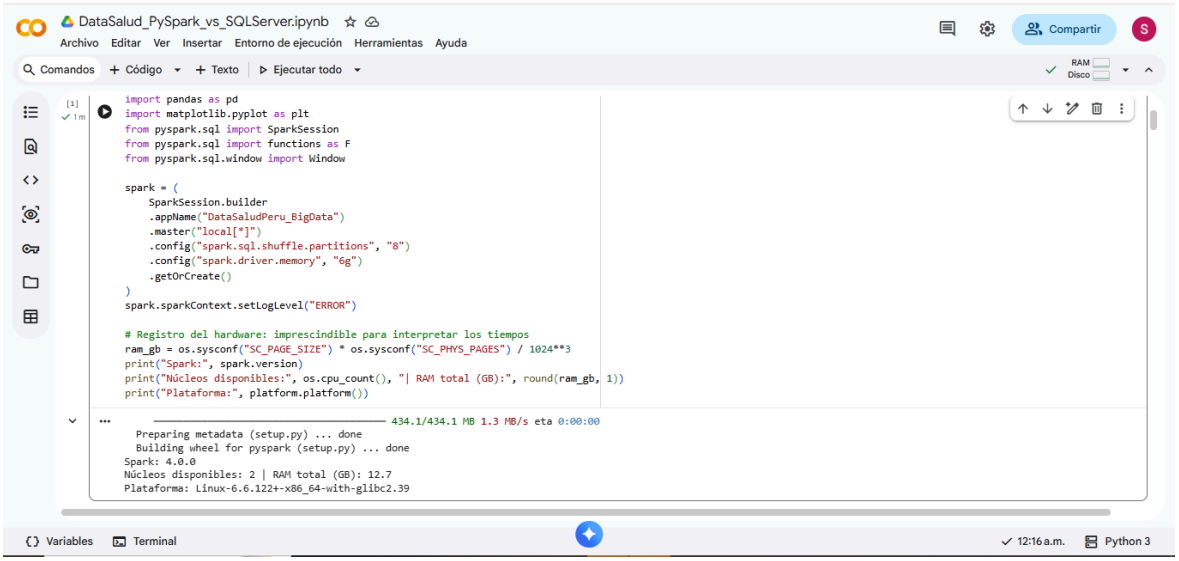
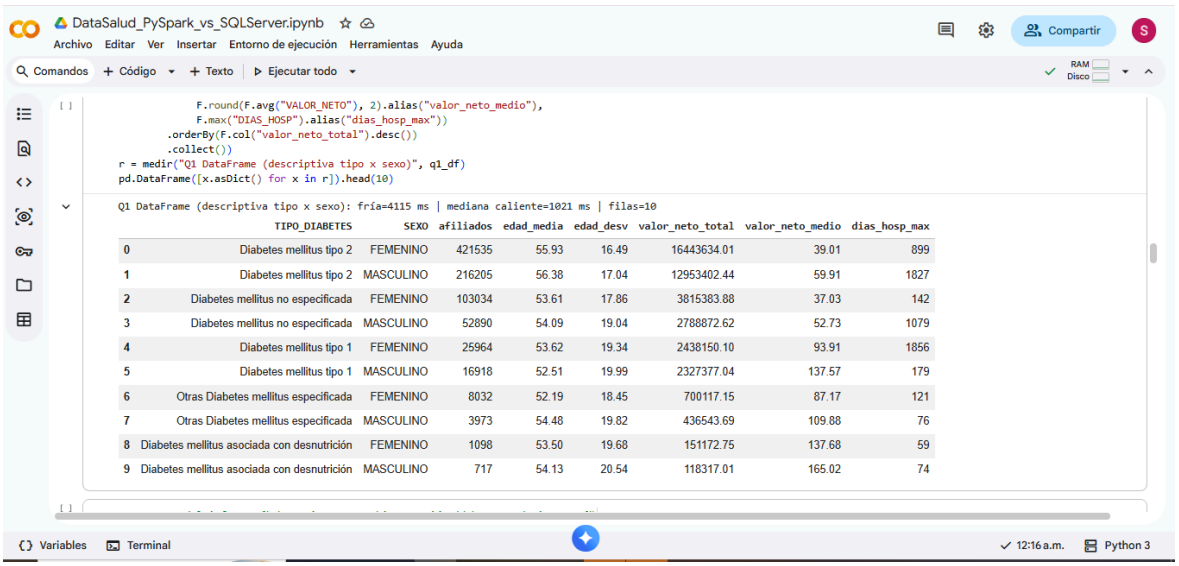


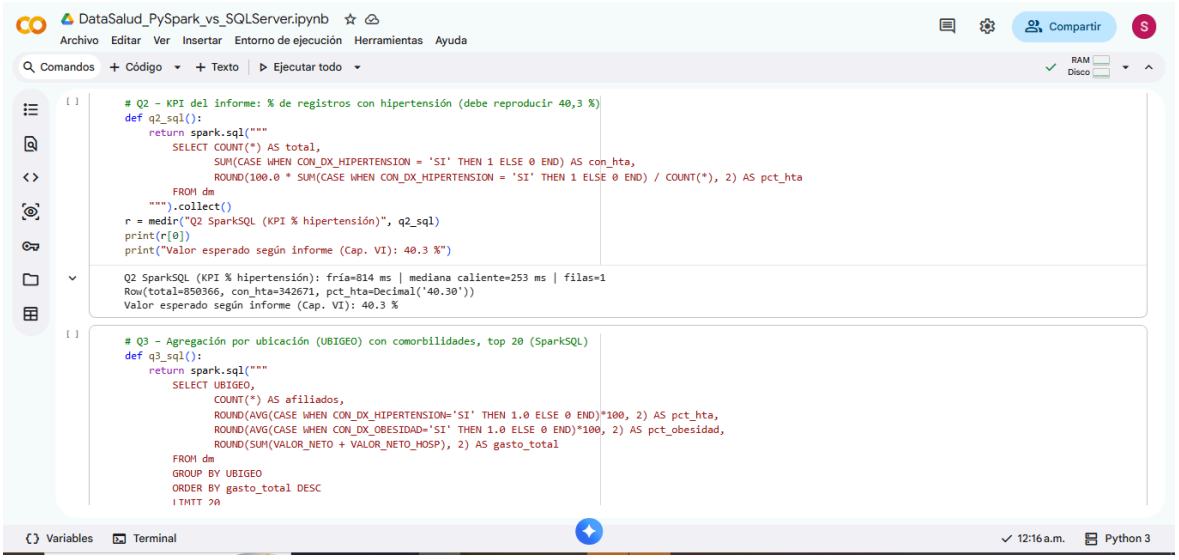
Figura 31. Ejecución de operaciones analíticas mediante DataFrames y SparkSQL.

a) Consulta Q1 mediante DataFrames de PySpark, con agrupación por tipo de diabetes y sexo.



	TIPO_DIABETES	SEXO	afiliados	edad_media	edad_desv	valor_neto_total	valor_neto_medio	dias_hosp_max
0	Diabetes mellitus tipo 2	FEMENINO	421535	55.93	16.49	16443634.01	39.01	899
1	Diabetes mellitus tipo 2	MASCULINO	216205	56.38	17.04	12953402.44	59.91	1827
2	Diabetes mellitus no especificada	FEMENINO	103034	53.61	17.86	3815383.88	37.03	142
3	Diabetes mellitus no especificada	MASCULINO	52890	54.09	19.04	2788872.62	52.73	1079
4	Diabetes mellitus tipo 1	FEMENINO	25964	53.62	19.34	2438150.10	93.91	1856
5	Diabetes mellitus tipo 1	MASCULINO	16918	52.51	19.99	2327377.04	137.57	179
6	Otras Diabetes mellitus especificada	FEMENINO	8032	52.19	18.45	700117.15	87.17	121
7	Otras Diabetes mellitus especificada	MASCULINO	3973	54.48	19.82	436543.69	109.88	76
8	Diabetes mellitus asociada con desnutrición	FEMENINO	1098	53.50	19.68	151172.75	137.68	59
9	Diabetes mellitus asociada con desnutrición	MASCULINO	717	54.13	20.54	118317.01	165.02	74

b) Consulta analítica equivalente mediante SparkSQL, mostrando el código ejecutado y los resultados obtenidos.



Fuente: Elaboración propia a partir de las pruebas realizadas en Google Colab.

Figura 31. Comparación de tiempos de ejecución entre PySpark y SQL Server.

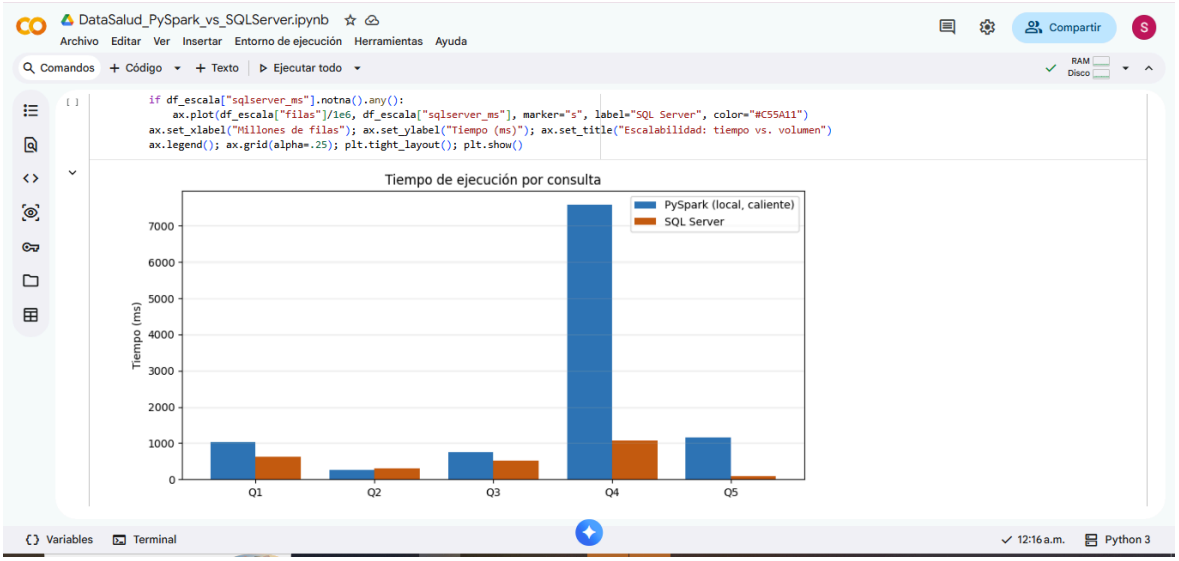
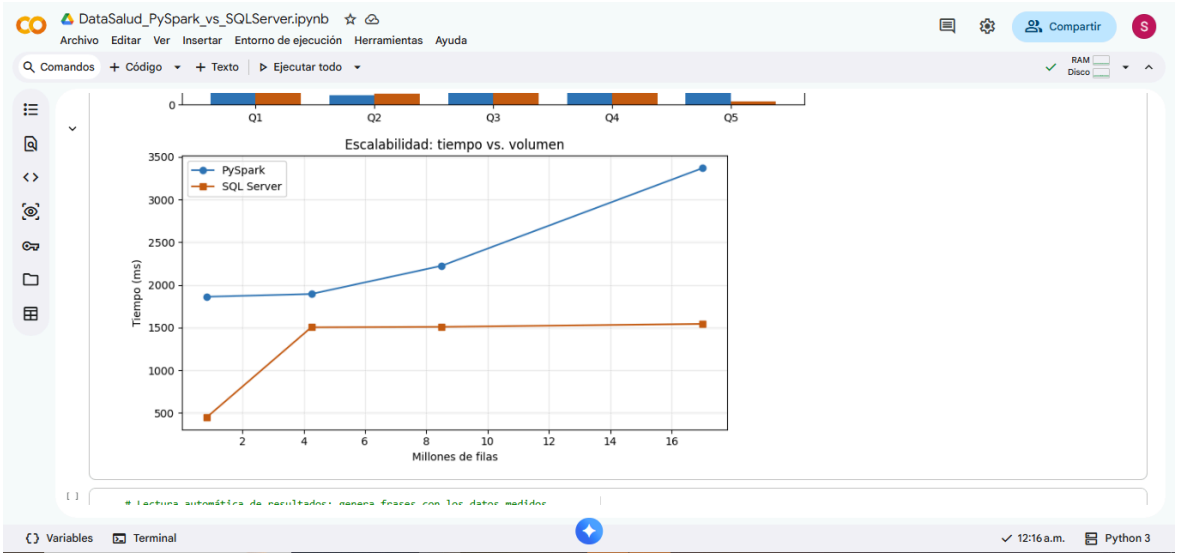


Figura 32. Evaluación experimental de escalabilidad según el volumen lógico de registros.



Fuente: Elaboración propia a partir de las pruebas realizadas en Google Colab y SQL Server.

Código y documentación complementaria

El código utilizado para las operaciones de procesamiento, agregación, filtrado y evaluación de escalabilidad se encuentra documentado en el notebook **DataSalud_PySpark_vs_SQLServer.ipynb**, que se presenta como archivo complementario del proyecto.

El notebook contiene las operaciones implementadas mediante DataFrames y SparkSQL, las mediciones experimentales, las tablas comparativas y los gráficos de resultados.